All books — Block Programming

Four books: Drawing, Maze, Space, Break & Build (blocks and Python)

📘 Learn Programming with Turtle Drawing 2

From blocks to Python — 18 drawing challenges with a turtle

📘 Learn Programming with Mazes 54

From blocks to Python — all 41 maze stages

📘 Learn Programming in Space 164

From blocks to Python — 13 star-hunting stages

📘 Learn Programming by Breaking & Building 207

From blocks to Python — 14 adventure stages

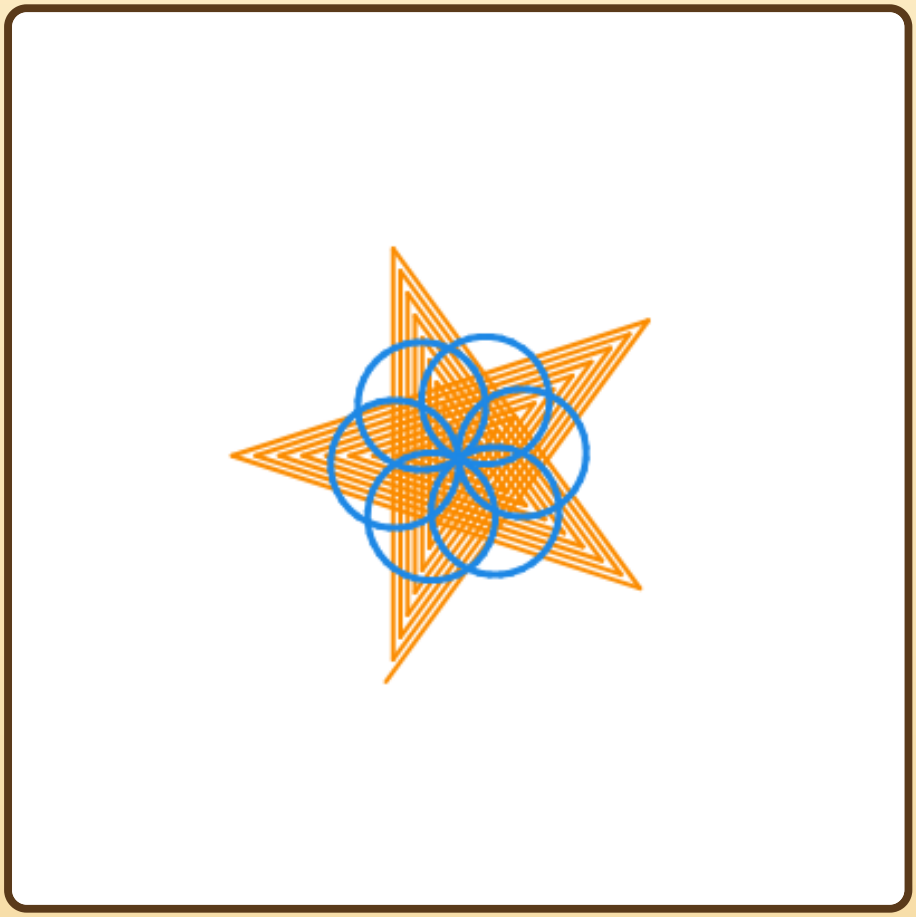
Numbers are PDF pages (each book numbers its own pages from 1).

National Institute of Technology, Maizuru College — Yasuhito Inoue

For kids and teachers

Learn Programming with Turtle Drawing

From blocks to Python — 18 drawing challenges with a turtle



National Institute of Technology, Maizuru College — Yasuhito Inoue

Contents

How to use this book 2

Part 1 The drawing book for kids 3

Chapter 1 Draw by giving commands to a turtle 4

Chapter 2 Block Drawing 5

Chapter 3 Drawing in Python 25

Well done! 45

Part 2 Guide for teachers and parents 46

About this guide 47

Overview of the challenges 48

Suggested order and teaching points 49

Assessment (example) 50

Using the apps and related materials 51

How to use this book

Use this book together with the “Block Drawing” app on a computer or tablet. You will learn how to think like a programmer by building programs with blocks and with Python.

1. Read the **problem** and look at the picture.
2. Think about the order: trace it with your finger or write it in your notebook.
3. Build the program in the app and press “▶ Run”. If it doesn’t work, find what was different and fix it.
4. Stuck? Look at the 💡 **Hint**. When you are done, turn the page and check the 📖 **Answer** and the ✨ **Key point**.

In Drawing, you are correct when your picture matches the light gray “model”.

There is more than one right answer. If it works, your program is correct, even if it is different from the book. Mistakes are hints — try again and again!

Apps in this book

App	Tasks	Grade
Block Drawing	9	Grade 3+ (angles, polygons)
Python Drawing	9	Grade 5 – junior high

PART 1

Part 1 The drawing book for kids

Chapter 1

Draw by giving commands to a turtle

In this app you give commands to a turtle. The turtle holds a pen, and it leaves a line wherever it moves.

Command	What the turtle does
move forward ○ steps	Moves ○ steps the way it is facing (draws a line)
turn right ○ degrees	Turns right ○ degrees on the spot (does not move)
repeat ○ times	Repeats the commands inside ○ times
count with i from ○ to ○ by ○	Repeats while changing the number i (a variable)

The secret of the turn angle

When the turtle turns all the way around, it turns **360 degrees**. To draw a square, the turtle turns 4 times and faces the same way again, so each turn is 360 ÷ 4 = **90 degrees**. For a hexagon, 360 ÷ 6 = 60 degrees.

> The turn angle is not the corner inside the shape. It is how much the turtle **changes the direction it walks**. Try turning your own body to check.

When it doesn’t work

- Compare with the model and find where it starts to go wrong.
- Check that (repeat times) × (turn angle) is 360 (or 720).
- Finding and fixing mistakes is called **debugging**.

Chapter 2

🐢 Block Drawing

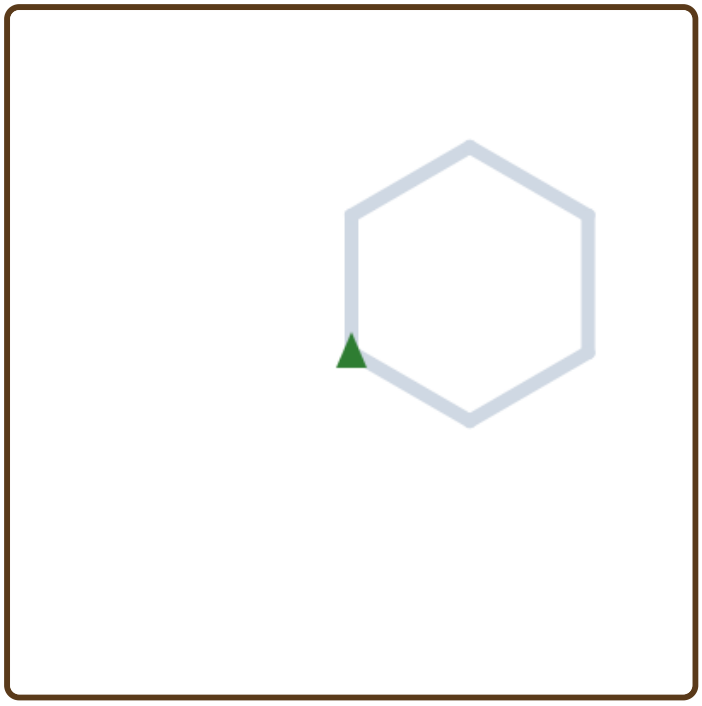
Snap blocks together to draw hexagons, stars and circles, then spirals and snowflakes. You will go from **loops → nested loops → variables → breaking a task into parts**.

Commands in this chapter

move forward / move back / turn right, left ◯ degrees / repeat ◯ times / count with i from ◯ to ◯ by ◯ / math (+ − × ÷) / pen color

① Hexagon ★

Loops Angles



Model (draw over the light lines)

Problem

Draw a hexagon with sides of 80 steps.

🎯 **Goal**：Draw a regular polygon with a loop

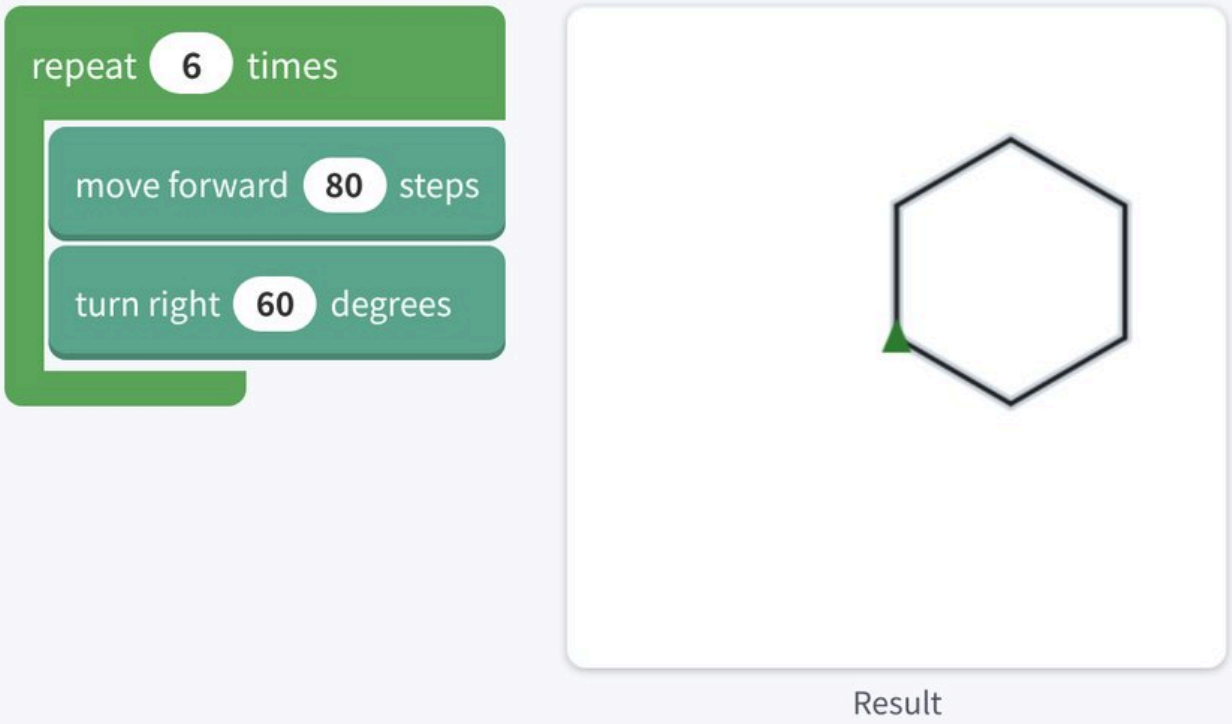
💡 **Hint (look if you get stuck)**

Repeat 6 times. How many degrees to turn? 360 ÷ 6 = ?

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ① Hexagon ★



✨ Key point

Repeat 6 times and turn 60° each time: 360 ÷ 6 = 60.

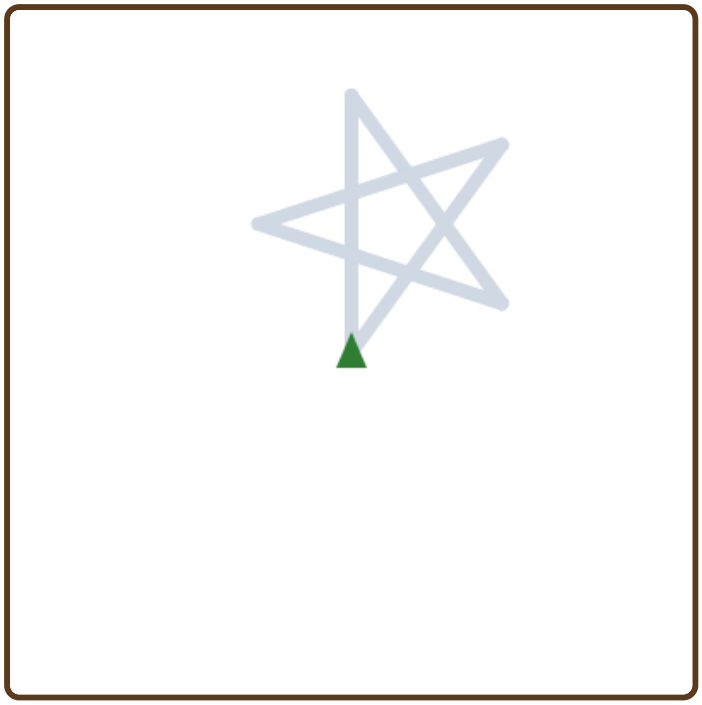
⚠ Common mistake

Turning 120° (the inside angle). The turtle turns the outside angle.

🎬 Video: `videos/en/draw-01.mp4`

② Star ★★

Loops Angles



Model (draw over the light lines)

Problem

Draw a star with lines of 150 steps.

🎯 **Goal**：Draw a shape that turns around twice

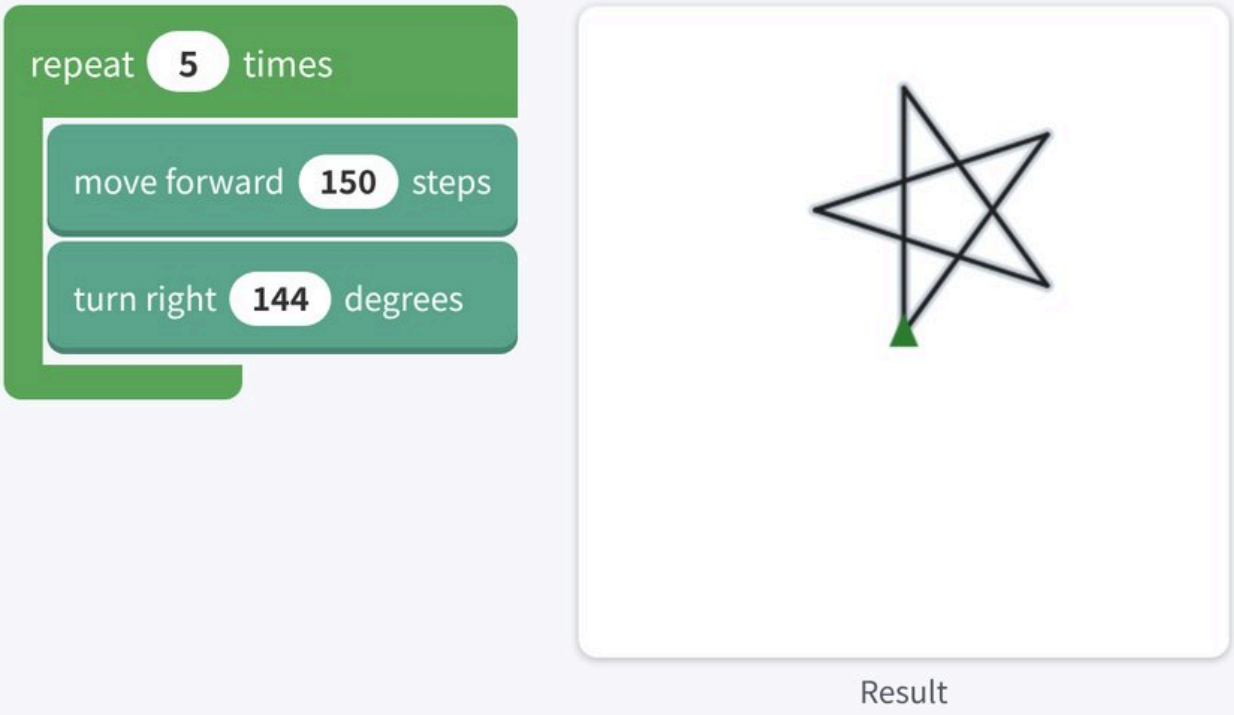
💡 **Hint (look if you get stuck)**

After 5 repeats the turtle turns around twice (720 degrees). 720 ÷ 5 = ?

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ② Star ★★



✨ Key point

It turns 720° (twice around) in 5 repeats, so each turn is 720 ÷ 5 = 144°.

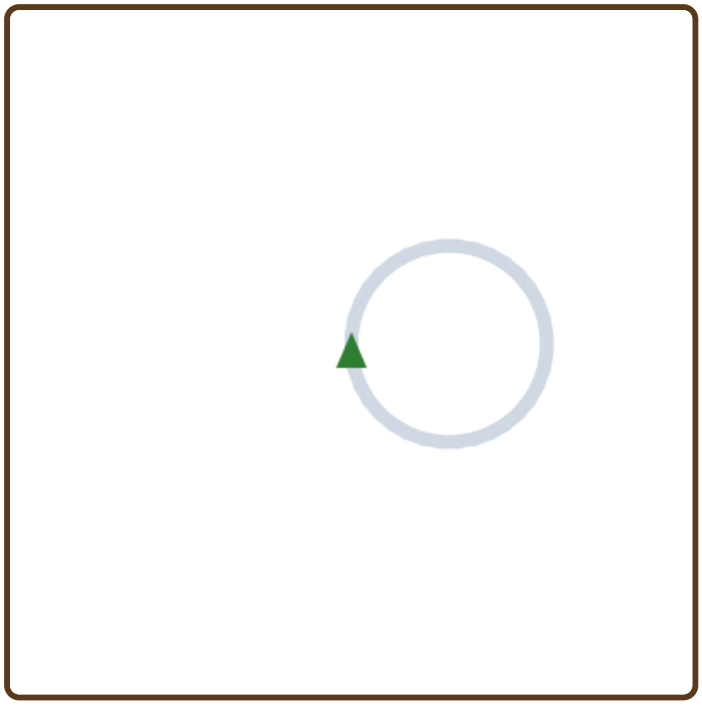
⚠ Common mistake

Using 72° (360 ÷ 5) draws a pentagon, not a star.

🎬 Video: `videos/en/draw-02.mp4`

③ Circle ★★

Loops Angles



Model (draw over the light lines)

Problem

Draw a circle by repeating: move 10 steps, then turn a little.

🎯 **Goal**：Make a circle from short lines

💡 **Hint (look if you get stuck)**

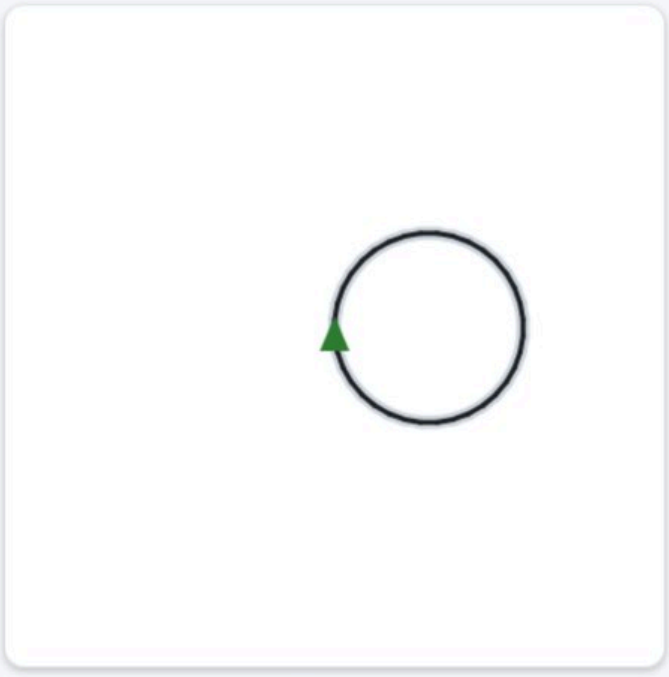
One full turn is 360 degrees. If you turn 10 degrees each time, how many repeats do you need?

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ③ Circle ★★





Result

✨ **Key point**

Move 10 and turn 10°, 36 times. Many small turns look like a circle.

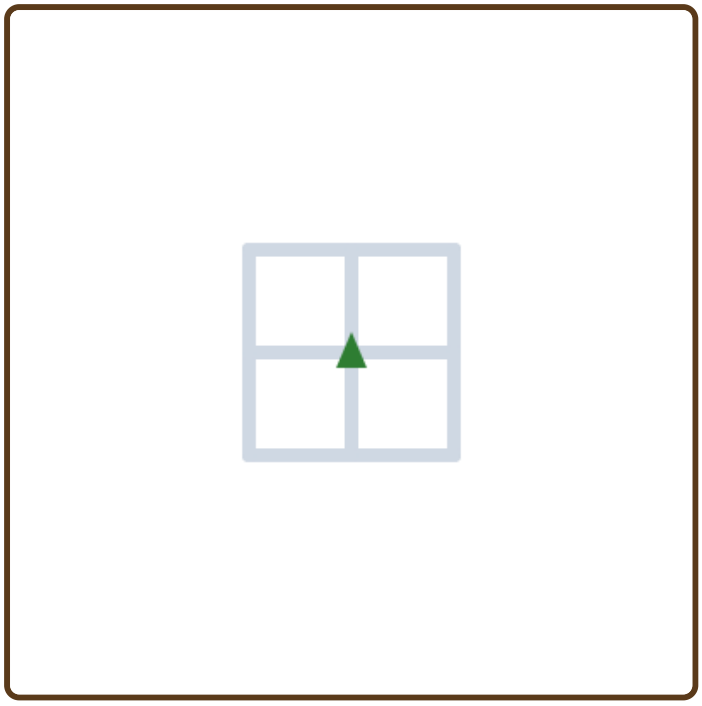
⚠ **Common mistake**

Too few repeats and the circle stops partway (360 ÷ 10 = 36).

🎬 Video: `videos/en/draw-03.mp4`

④ Window ★★★

Nesting



Model (draw over the light lines)

Problem

Draw 4 squares (sides of 60 steps) next to each other to make a window.

🎯 **Goal**：Put a loop inside a loop

💡 **Hint (look if you get stuck)**

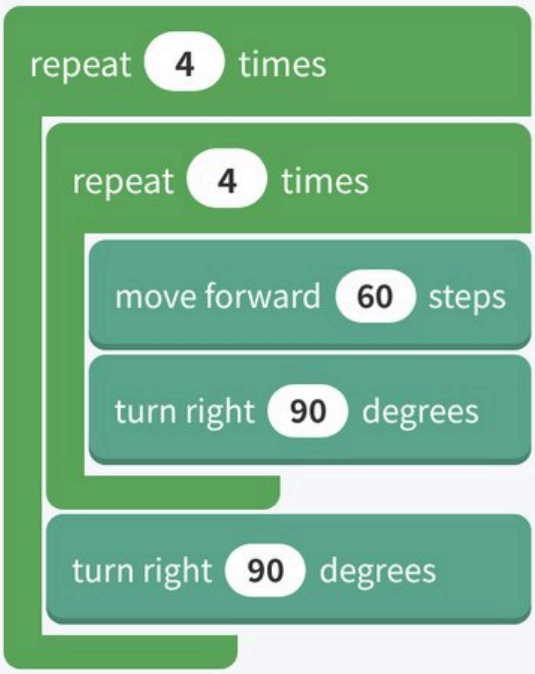
Repeat "draw a square" 4 times. After each square, turn right 90 degrees.

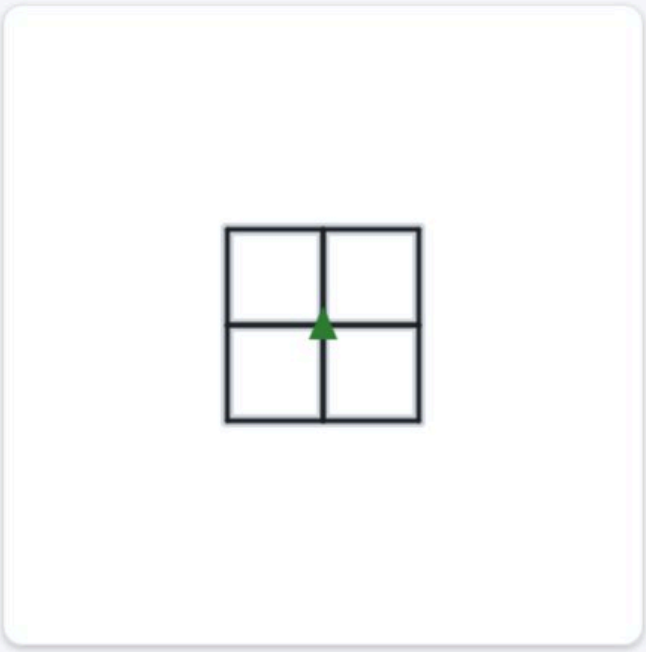
Put a repeat inside a repeat!

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ④ Window ★★★





Result

✨ Key point

Repeat the square-drawing loop 4 more times.

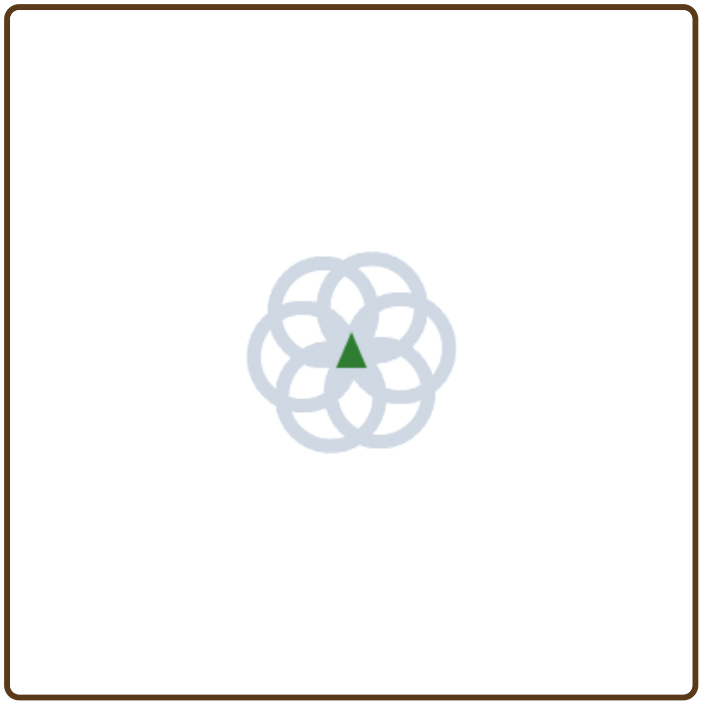
⚠ Common mistake

Forget “turn right 90” after each square, and the squares overlap into one.

🎬 Video: `videos/en/draw-04.mp4`

⑤ Flower ★★★

Nesting Angles



Model (draw over the light lines)

Problem

Draw 6 circles, turning between them, to make a flower. (Circle: 5 steps and 10 degrees, 36 times)

🎯 **Goal**：Nesting + rotating

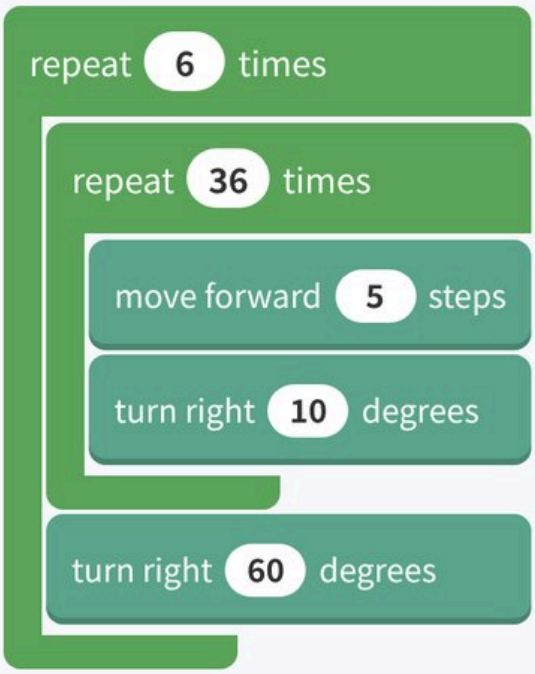
💡 **Hint (look if you get stuck)**

After each circle, turn 360 ÷ 6 = 60 degrees. Repeat this 6 times.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑤ Flower ★★★





Result

✨ **Key point**

After each circle, turn 60°. Do this 6 times.

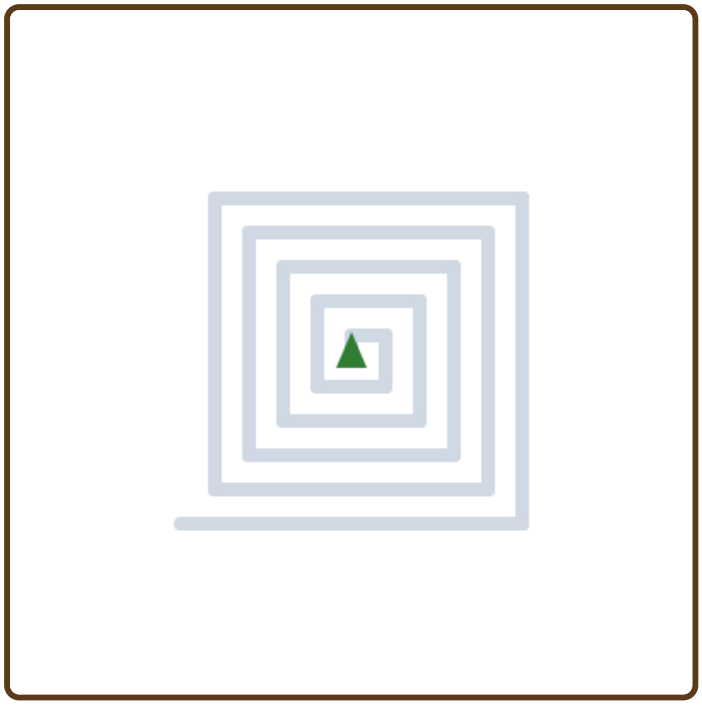
⚠ **Common mistake**

If the turn is not 60°, the petals are uneven.

🎬 Video: `videos/en/draw-05.mp4`

⑥ Square spiral ★★★★

Variables Loops



Model (draw over the light lines)

Problem

Move 10, 20, 30 … up to 200 steps, turning right 90 degrees each time.

🎯 **Goal**：Use a variable as a length

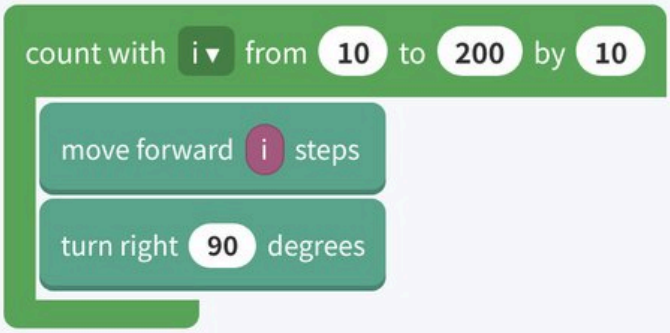
💡 **Hint (look if you get stuck)**

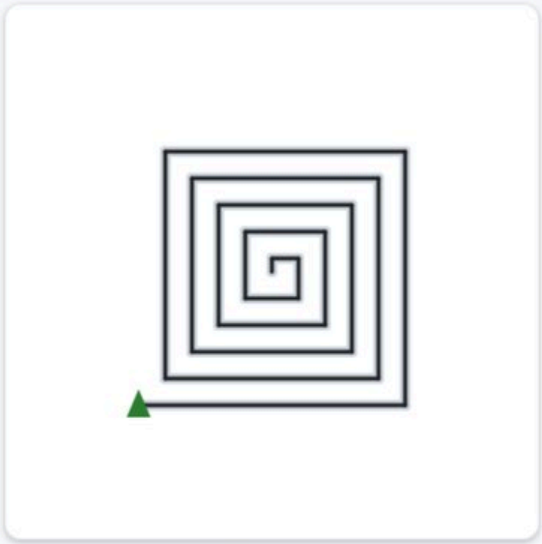
Use the loop "count with i from 10 to 200 by 10". Put the variable i into the "move" block.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑥ Square spiral ★★★★





Result

✨ Key point

i changes to 10, 20, 30..., so the line gets longer and longer.

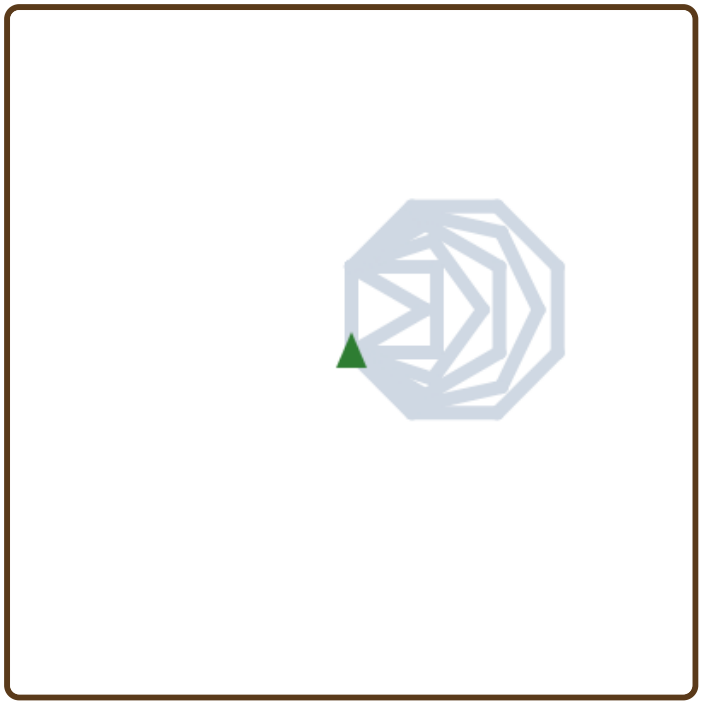
⚠ Common mistake

With a fixed number in “move”, the length never changes. Put the variable i in.

🎬 Video: `videos/en/draw-06.mp4`

⑦ Polygons in a row ★★★★

Variables | Math | Nesting



Model (draw over the light lines)

Problem

From the same spot, draw a triangle, square, pentagon … up to an octagon, each with sides of 50 steps.

🎯 **Goal**：Variable + math

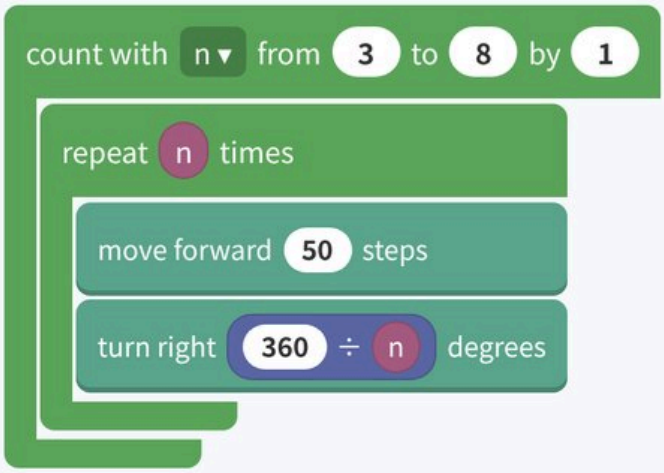
💡 **Hint (look if you get stuck)**

Change n from 3 to 8: "repeat n times: move 50 steps, turn right 360 ÷ n degrees". Use the math block!

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑦ Polygons in a row ★★★★





Result

✨ Key point

An n-sided shape turns 360 ÷ n each time. Change n from 3 to 8 to draw 6 shapes.

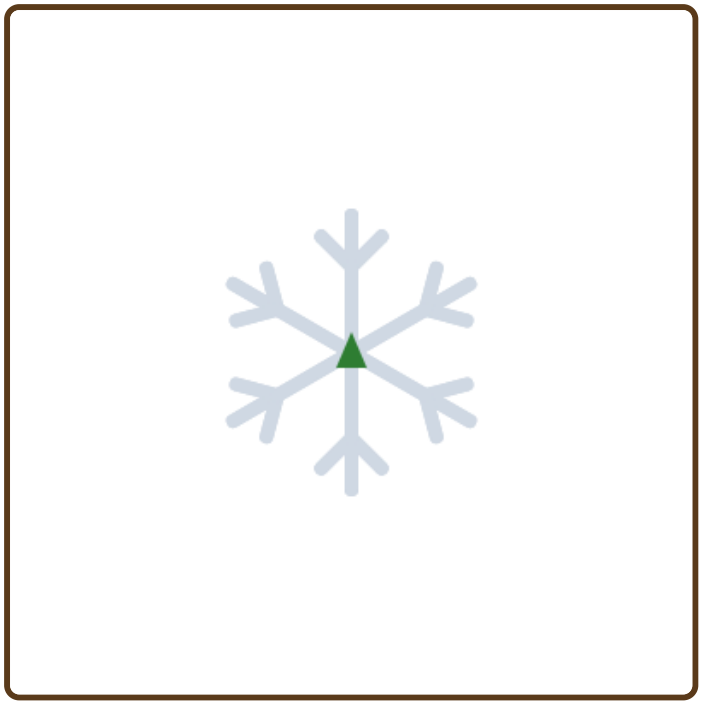
⚠ Common mistake

With a fixed turn (like 60), every shape is the same.

🎬 Video: `videos/en/draw-07.mp4`

⑧ Snowflake ★★★★★

Decomposition Loops



Model (draw over the light lines)

Problem

Draw one branch, come back to where you started, and turn 60 degrees. Repeat this 6 times.

🎯 **Goal**：Break the task into steps

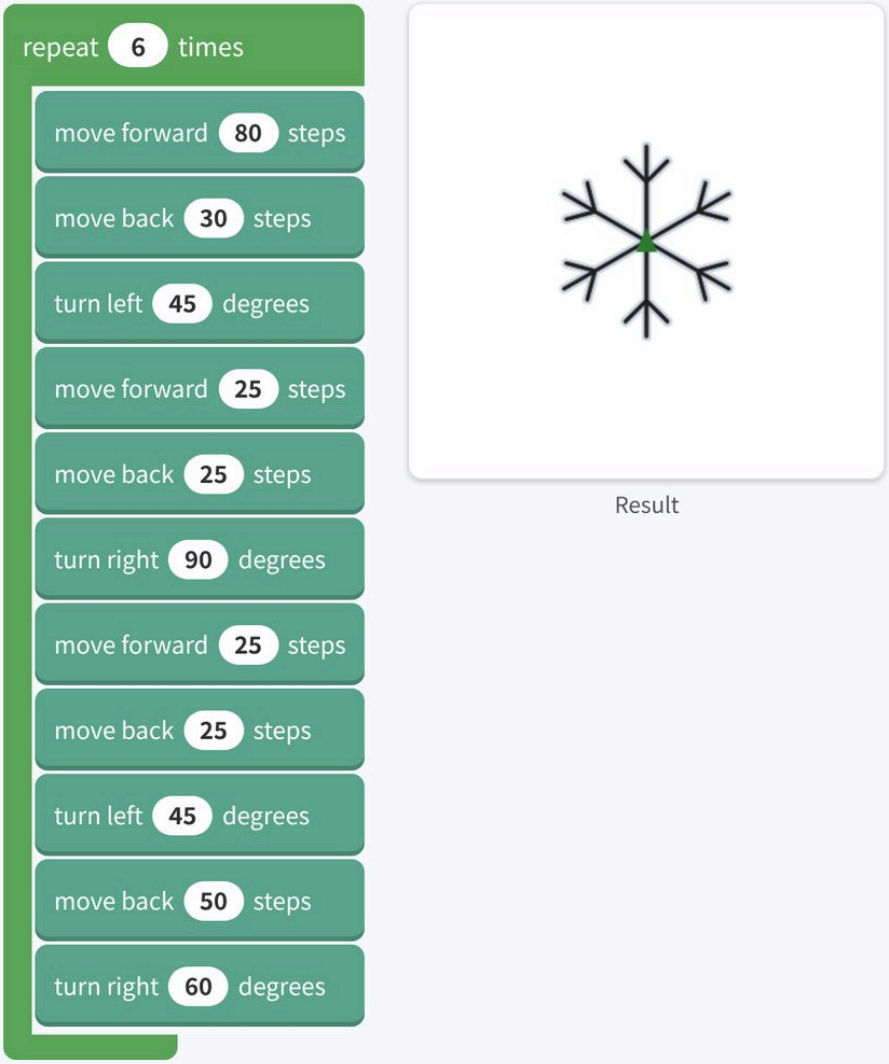
💡 **Hint (look if you get stuck)**

Branch: forward 80 → back 30 → left 45 → forward 25 → back 25 → right 90 → forward 25 → back 25 → left 45 → back 50.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑧ Snowflake ★★★★★



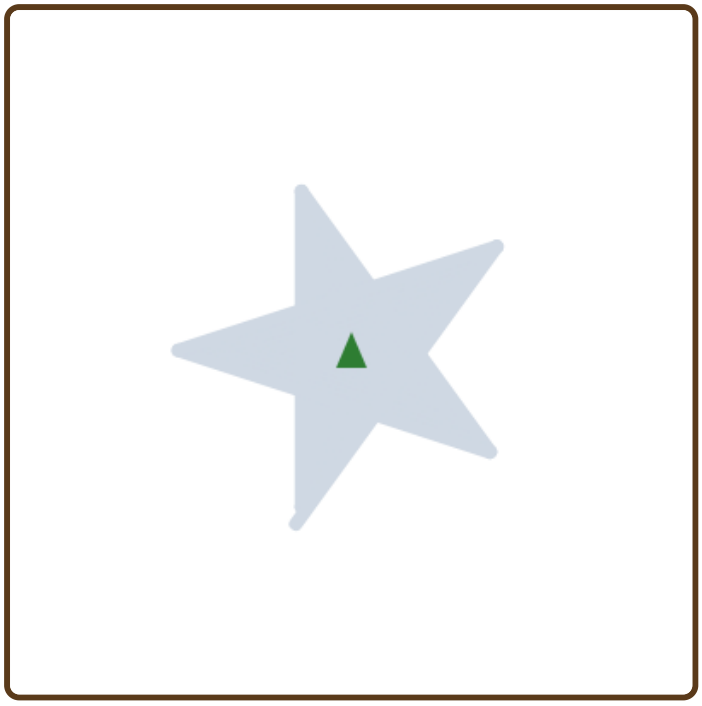
> ✨ **Key point**
>
> The trick: after each branch, put the position and the direction back.

> ⚠ **Common mistake**
>
> Forget the final “move back 50”, and the branches drift.

🎬 Video: `videos/en/draw-08.mp4`

⑨ Star spiral ★★★★★

Variables Math



Model (draw over the light lines)

Problem

Move 4, 8, 12 … steps (4 more each time, 50 times), turning right 144 degrees.

🎯 **Goal**：Variable × math + turning

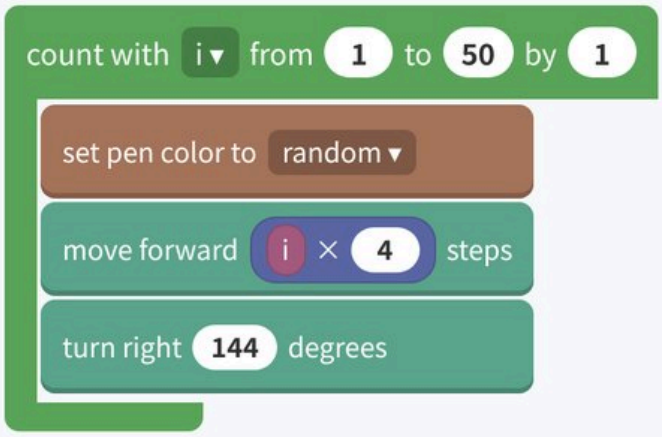
💡 **Hint (look if you get stuck)**

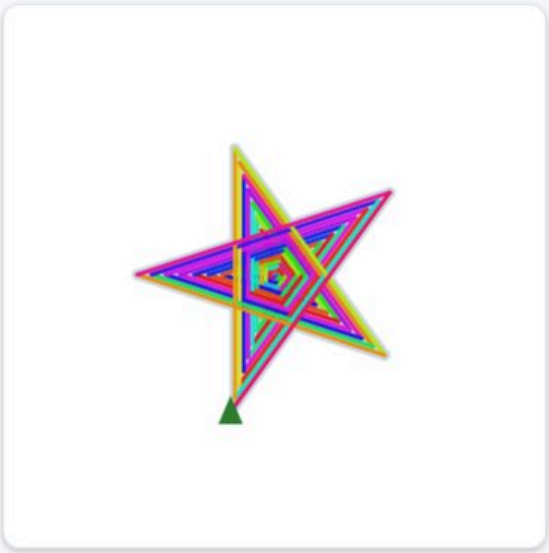
Count i from 1 to 50 by 1. Move i × 4 steps. Try a "random" pen color!

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑨ Star spiral ★★★★★





Result

✨ Key point

The length i × 4 grows, and turning 144° each time makes a star spiral.

⚠ Common mistake

Without the variable i in the multiply block, the length does not grow.

🎬 Video: `videos/en/draw-09.mp4`

Column

Corners and turn angles

A regular polygon is “move the same length → turn the same angle”, repeated. The turn angle is always **360 ÷ number of corners**. A star turns around twice, so 720 ÷ 5 = 144 degrees. As you add more corners, the shape gets closer to a circle. With a program, you can check this right away just by changing a number.

Chapter 3

🐍 Drawing in Python

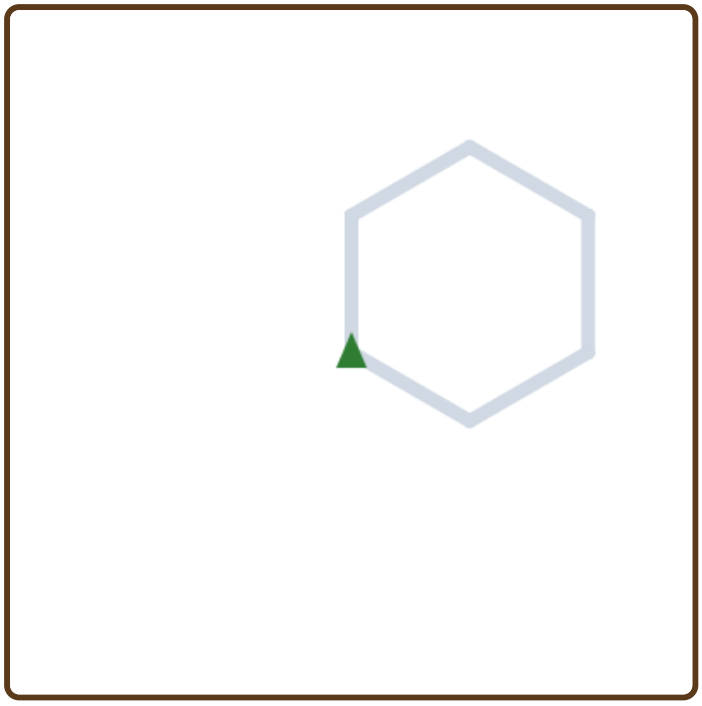
Draw the same challenges in **Python**. “move forward 80 steps” is `forward(80)`, “turn right 60 degrees” is `right(60)`, and “repeat 6 times” is `for i in range(6):`.

Commands in this chapter

forward(80) / right(60) / left(45) / for i in range(6): / for i in range(10, 201, 10): / 360 / n

① Hexagon ★

Loops Angles Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 1

Problem

forward(80) moves 80 steps and right(60) turns 60 degrees. Repeat them 6 times to draw a hexagon.

🎯 **Goal** : Draw a regular polygon with for

💡 **Hint (look if you get stuck)**

Put forward(80) and right(60) inside for i in range(6):. 360 ÷ 6 = 60

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ① Hexagon ★

```
for i in range(6):
    forward(80)
    right(60)
```

✨ Key point

Write numbers inside forward() and right(). The 2 lines inside for repeat 6 times.

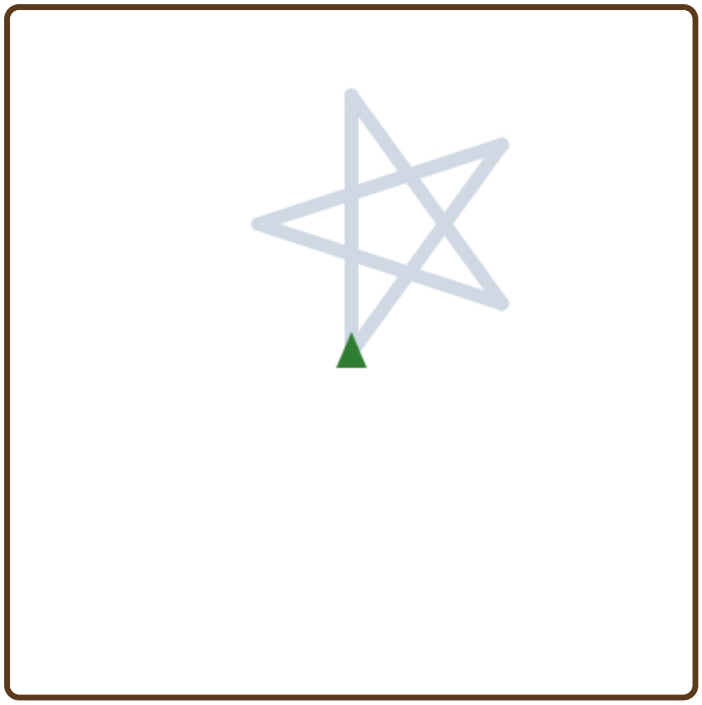
⚠ Common mistake

If right(60) is not indented, it turns only once and draws a straight line.

🎬 Video: `videos/en/pydraw-01.mp4`

② Star ★★

Loops | Angles | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 2

Problem

Draw a star with lines of 150 steps: repeat forward(150) and right(144) 5 times.

🎯 **Goal**：Think about turning around twice

💡 **Hint (look if you get stuck)**

The turtle turns around twice (720 degrees). 720 ÷ 5 = 144

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ② Star ★★

```
for i in range(5):
    forward(150)
    right(144)
```

✨ Key point

5 turns make 720 degrees (twice around), so each turn is 720 ÷ 5 = 144.

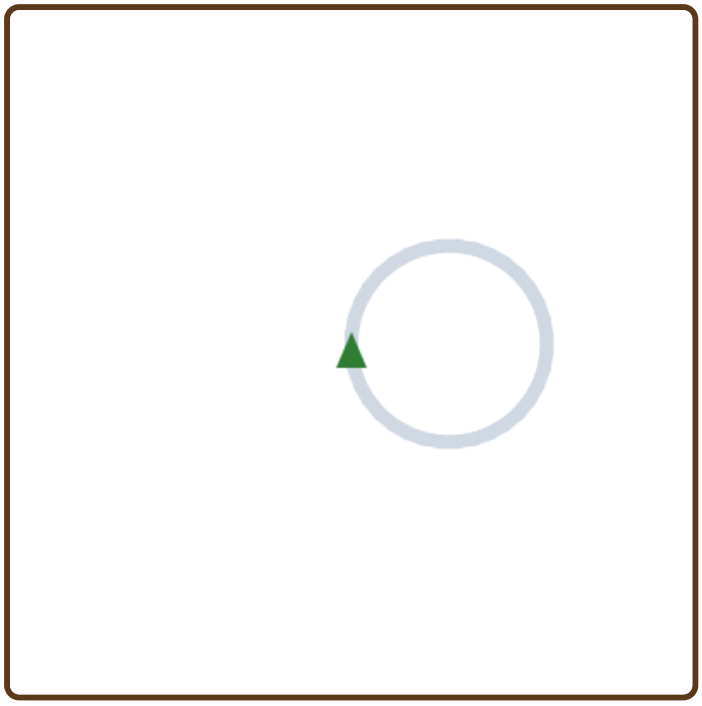
⚠ Common mistake

right(72) (360 ÷ 5) makes a pentagon, not a star.

🎬 Video: `videos/en/pydraw-02.mp4`

③ Circle ★★

Loops | Angles | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 3

Problem

Draw a circle by repeating forward(10) and right(10).

🎯 **Goal**：Make a circle from short lines

💡 **Hint (look if you get stuck)**

One full turn is 360 degrees. 360 ÷ 10 = 36, so use range(36).

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ③ Circle ★★

```
for i in range(36):
    forward(10)
    right(10)
```

✨ Key point

36 turns of 10 degrees make 360. Many short lines look like a circle.

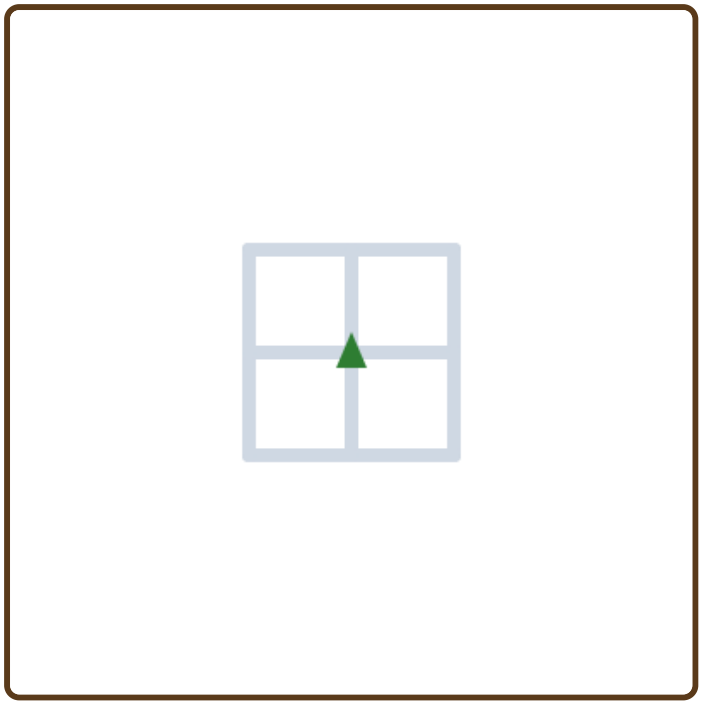
⚠ Common mistake

range(35) leaves a small gap. Check 360 ÷ 10 = 36.

🎬 Video: `videos/en/pydraw-03.mp4`

④ Window ★★★

Loops | Nesting | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 4

Problem

Draw 4 squares (sides of 60 steps) to make a window. Put a for loop inside a for loop.

🎯 **Goal**：Put a for inside a for

💡 **Hint (look if you get stuck)**

Outer loop 4 times: the inner for k in range(4): draws a square, then right(90).

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ④ Window ★★★

```
for i in range(4):
    for k in range(4):
        forward(60)
        right(90)
    right(90)
```

✨ Key point

The inner for is indented 8 spaces. After a square, right(90) inside the outer for.

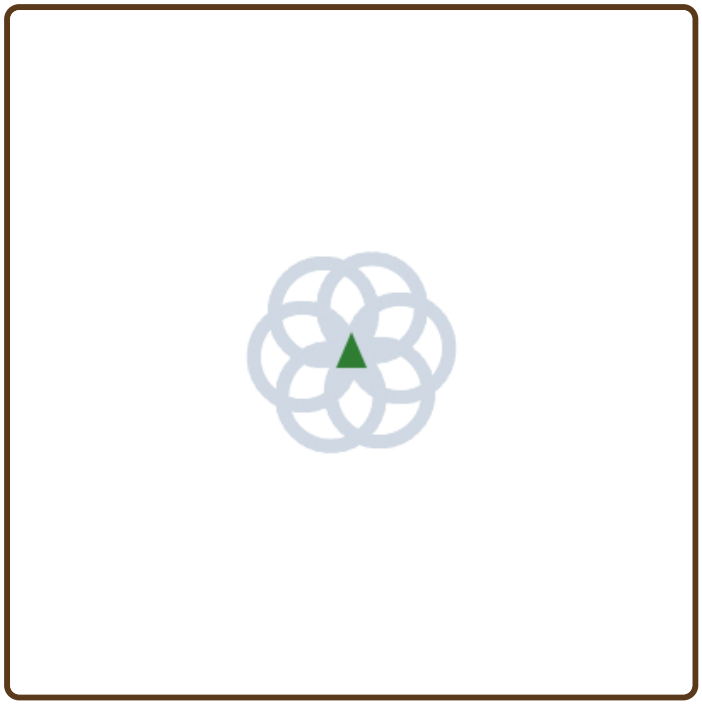
⚠ Common mistake

If the last right(90) is at the inner for’s depth, the squares overlap and look like one.

🎬 Video: `videos/en/pydraw-04.mp4`

⑤ Flower ★★★

Loops | Nesting | Angles | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 5

Problem

Draw 6 circles (forward(5) and right(10), 36 times), turning 60 degrees between them, to make a flower.

🎯 **Goal**：Nesting + turning around

💡 **Hint (look if you get stuck)**

for i in range(6): → for k in range(36): forward(5), right(10) → right(60)

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑤ Flower ★★★

```
for i in range(6):
    for k in range(36):
        forward(5)
        right(10)
    right(60)
```

✨ **Key point**

One circle is the inner for. The outer for turns 360 ÷ 6 = 60 degrees, 6 times.

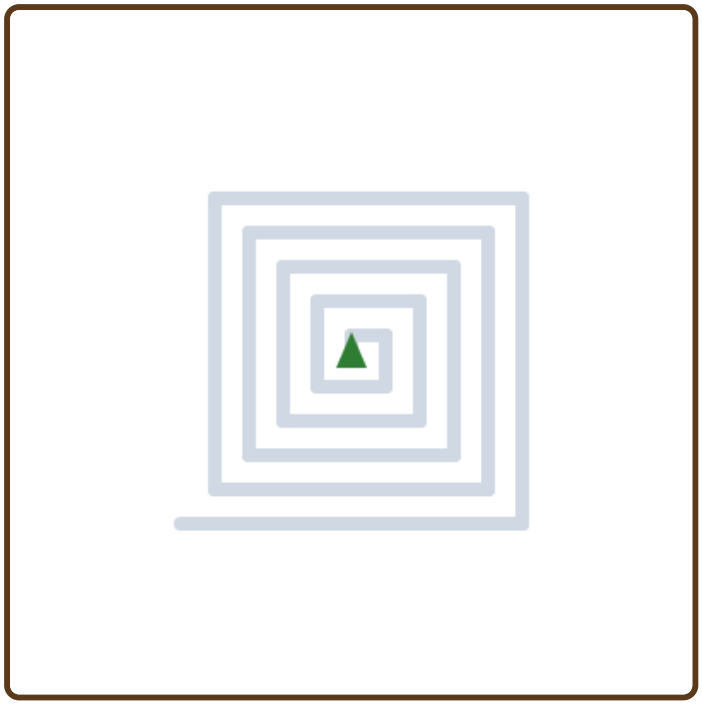
⚠ **Common mistake**

If the turn is not 60, the petals are uneven.

🎬 Video: `videos/en/pydraw-05.mp4`

⑥ Square spiral ★★★★

Variables | Loops | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 6

Problem

Use for i in range(10, 201, 10): so i becomes 10, 20, 30 ... 200, and draw a spiral with forward(i) and right(90).

🎯 **Goal**：Use the variable i as the length

💡 **Hint (look if you get stuck)**

range(start, end, step). The end number is not included, so use 201.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑥ Square spiral ★★★★

```
for i in range(10, 201, 10):
    forward(i)
    right(90)
```

✨ Key point

range(10, 201, 10) gives 10, 20, ... 200. The end is not included, so use 201. forward(i) gets longer each time.

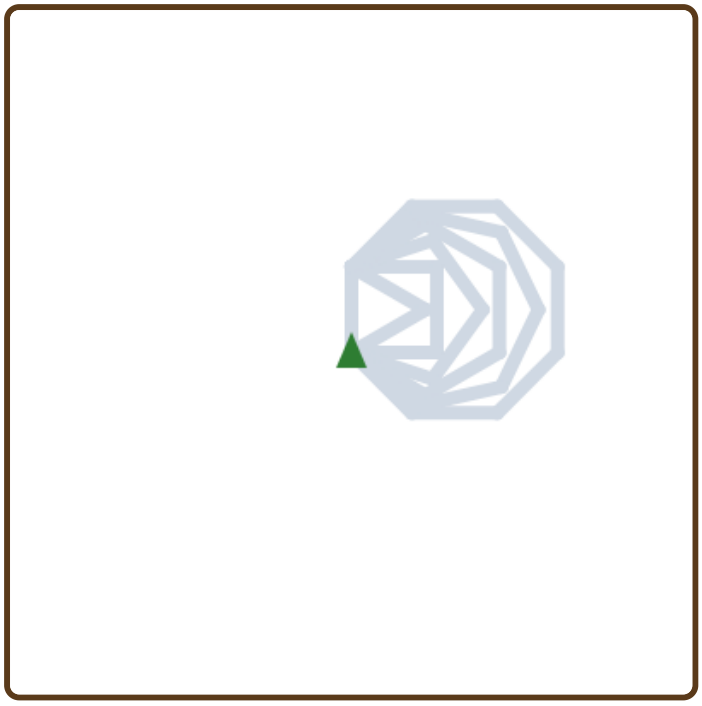
⚠ Common mistake

range(10, 200, 10) stops at 190.

🎬 Video: `videos/en/pydraw-06.mp4`

⑦ Polygons in a row ★★★★

Variables | Math | Nesting | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 7

Problem

Use for n in range(3, 9): to change n from 3 to 8, and draw an n-sided shape (sides of 50 steps) from the same spot.

🎯 **Goal** : Variables and division

💡 **Hint (look if you get stuck)**

Inside, for i in range(n): forward(50) and right(360 / n). / means divide.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑦ Polygons in a row ★★★★

```
for n in range(3, 9):
    for i in range(n):
        forward(50)
        right(360 / n)
```

✨ **Key point**

An n-sided shape turns 360 / n. range(3, 9) gives n = 3 to 8.

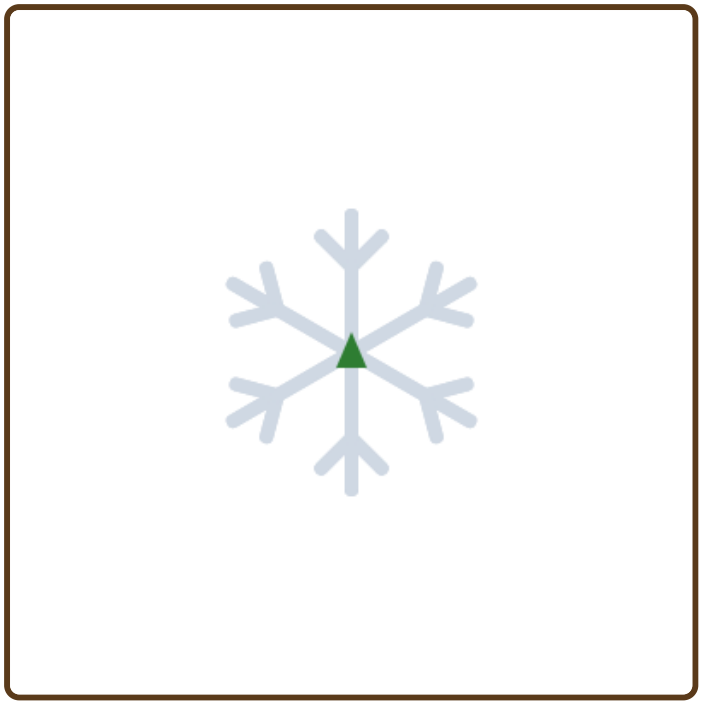
⚠ **Common mistake**

360 // n (floor division) has no block. Use / for division.

🎬 Video: `videos/en/pydraw-07.mp4`

⑧ Snowflake ★★★★★

Sequence | Decomposition | Loops | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 8

Problem

Draw one branch, come back, and right(60). Repeat this 6 times to draw a snowflake.

🎯 **Goal**：Break the steps into parts

💡 **Hint (look if you get stuck)**

Branch: forward(80) → back(30) → left(45) → forward(25) → back(25) → right(90) → forward(25) → back(25) → left(45) → back(50)

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑧ Snowflake ★★★★★

```
for i in range(6):
    forward(80)
    back(30)
    left(45)
    forward(25)
    back(25)
    right(90)
    forward(25)
    back(25)
    left(45)
    back(50)
    right(60)
```

✨ Key point

One branch is 10 lines. It returns to the start, so right(60) moves to the next branch.

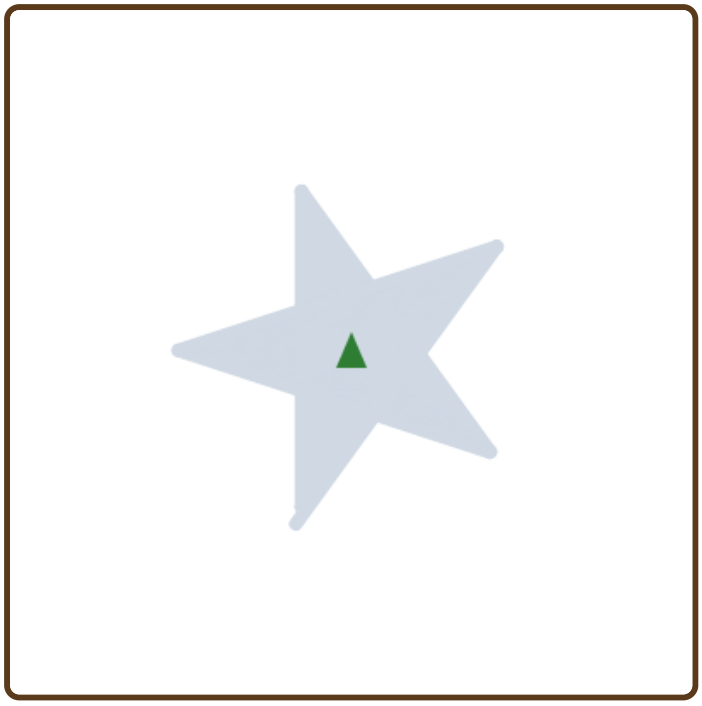
⚠ Common mistake

Forgetting the last back(50) shifts each branch.

🎬 Video: `videos/en/pydraw-08.mp4`

⑨ Star spiral ★★★★★

Variables | Math | Loops | Python syntax (colon & indent)



Model (draw over the light lines)

Same shape as block challenge 9

Problem

Use for i in range(1, 51): (i from 1 to 50), then forward(i * 4) and right(144). Change colors with pen_color('random').

🎯 **Goal** : Variable × math + turning

💡 **Hint (look if you get stuck)**

* means multiply. The line gets 4 steps longer each time.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer ⑨ Star spiral ★★★★★

```
for i in range(1, 51):
    pen_color('random')
    forward(i * 4)
    right(144)
```

✨ **Key point**

The length i * 4 grows by 4 each time, and turning 144 makes a star spiral.

⚠ **Common mistake**

range(1, 50) stops after 49. Use range(1, 51) for 50 times.

🎬 Video: `videos/en/pydraw-09.mp4`

Column

How to read range

`range(6)` gives 0, 1, 2, 3, 4, 5 (six numbers). `range(10, 201, 10)` starts at 10, adds 10 each time, and stops before 201 (at 200). In Python, the last number is not included. In `360 / n`, “/” means division.

Well done!

You worked hard to the end! Let’s look back at the ideas you used in this book.

Idea	Where did it appear?
Loops	Pack “move, turn” together (① – ③)
Angles	360 ÷ corners; a star is 720 ÷ 5 (① – ③)
Nested loops	A loop inside a loop (④ ⑤)
Variables	Repeat while changing i (⑥ ⑦ ⑨)
Breaking into parts	One branch → six times (⑧)
Python	Write the same thing in words (code)

Try Free drawing to make your own patterns! Next, try “Block Maze” and “Space Star Hunt”.

PART 2

Part 2 Guide for teachers and parents

About this guide

This book covers “Block Drawing” (9 challenges) and “Python Drawing” (the same 9). Part 1 is written in simple language for students.

In Japanese elementary mathematics, drawing regular polygons with programming is an example activity in Grade 5 (“Regular polygons and circles”). Challenges ①–③ fit that lesson directly; ④ onward extends to nested loops, variables and decomposition. In junior high, the Python version can introduce text-based programming with the same shapes.

Aims

- Express properties of shapes (side length, turn angle) with ordered commands and loops.
- Reason that the turn angle is “360 ÷ number of sides” and explain it.
- Use variables to draw shapes that change step by step (spirals, etc.).

Overview of the challenges

App	#	Stage	Ideas
🐢 Block Drawing	1	① Hexagon ★	Loops, Angles
	2	② Star ★★	Loops, Angles
	3	③ Circle ★★	Loops, Angles
	4	④ Window ★★★	Nesting
	5	⑤ Flower ★★★	Nesting, Angles
	6	⑥ Square spiral ★★★★	Variables, Loops
	7	⑦ Polygons in a row ★★★★	Variables, Math, Nesting
	8	⑧ Snowflake ★★★★★	Decomposition, Loops
	9	⑨ Star spiral ★★★★★	Variables, Math
🐍 Python Drawing	1	① Hexagon ★	Loops, Angles, Python syntax (colon & indent)
	2	② Star ★★	Loops, Angles, Python syntax (colon & indent)
	3	③ Circle ★★	Loops, Angles, Python syntax (colon & indent)
	4	④ Window ★★★	Loops, Nesting, Python syntax (colon & indent)
	5	⑤ Flower ★★★	Loops, Nesting, Angles, Python syntax (colon & indent)
	6	⑥ Square spiral ★★★★	Variables, Loops, Python syntax (colon & indent)
	7	⑦ Polygons in a row ★★★★	Variables, Math, Nesting, Python syntax (colon & indent)
	8	⑧ Snowflake ★★★★★	Sequence, Decomposition, Loops, Python syntax (colon & indent)
	9	⑨ Star spiral ★★★★★	Variables, Math, Loops, Python syntax (colon & indent)

Suggested order and teaching points

Students	Challenges	Lessons	Key ideas
Grades 3–4	① – ③	1–2	Loops, angles
Grade 5 (math)	① – ③ (+ ④ ⑤)	2	Regular polygons, 360 ÷ sides
Grades 5–6	④ – ⑨	2–3	Nesting, variables, math, decomposition
Grade 6 – junior high	Python ① – ⑨	2–4	for/range, variables, syntax

Common difficulties and support

- **Inside angle vs. turn angle:** turning 60° for a triangle. → Have students walk the shape and feel “the angle that changes the direction”.
- **Times and angle don’t match:** check that times × angle = 360 (720 for the star).
- **Where to put the turn in nested loops:** confirm in words that “after each square” means outside the inner loop.
- **Variables:** for ⑥, have students write in a table how i changes (10, 20, 30…) before building.
- **Python:** post the three rules (brackets, colon, indent). Note that the end number of `range` is not included.

Example 45-minute lesson (① – ③)

Time	Activity	Support
Intro 5 min	Review regular polygons; share the goal.	Show a triangle, square and hexagon.
Main 30 min	Draw a square and triangle; predict and check the hexagon’s angle; find the rule in a table.	Predict → run → fix. Use the Worksheet 2 table.
Wrap-up 10 min	Summarize “360 ÷ sides”; write a reflection.	Compare 2–3 programs.

Assessment (example)

Area	Meets expectations (B)	Exceeds (A), e.g.
Knowledge & skills	Combines blocks (code) correctly and completes the task.	Uses loops and conditions to complete it with a short, clear program.
Thinking, judgment & expression	Finds where the error is from the result and fixes it.	Explains the rule or steps in words and makes a program that works in other cases too.
Attitude toward learning	Keeps trying even when it doesn’t work.	Shares ideas with friends and looks for better ways on their own.

Use the worksheets (1 reflection; 2, 5, 7 etc.) and the app progress (8 check card, certificate) as evidence.

Using the apps and related materials

Requirements and tips

- Works in the browser on computers, tablets and smartphones. No installation needed. An internet connection is required.
- Cleared stages are saved in that device’s browser. If devices are shared, check progress at the start of each lesson.
- **Teacher mode:** add `?sensei` to the end of the address to show an “Answer” button.
- From “⚙ Settings” you can switch Japanese/English, use the pixel font, switch to the Python version and show the certificate.

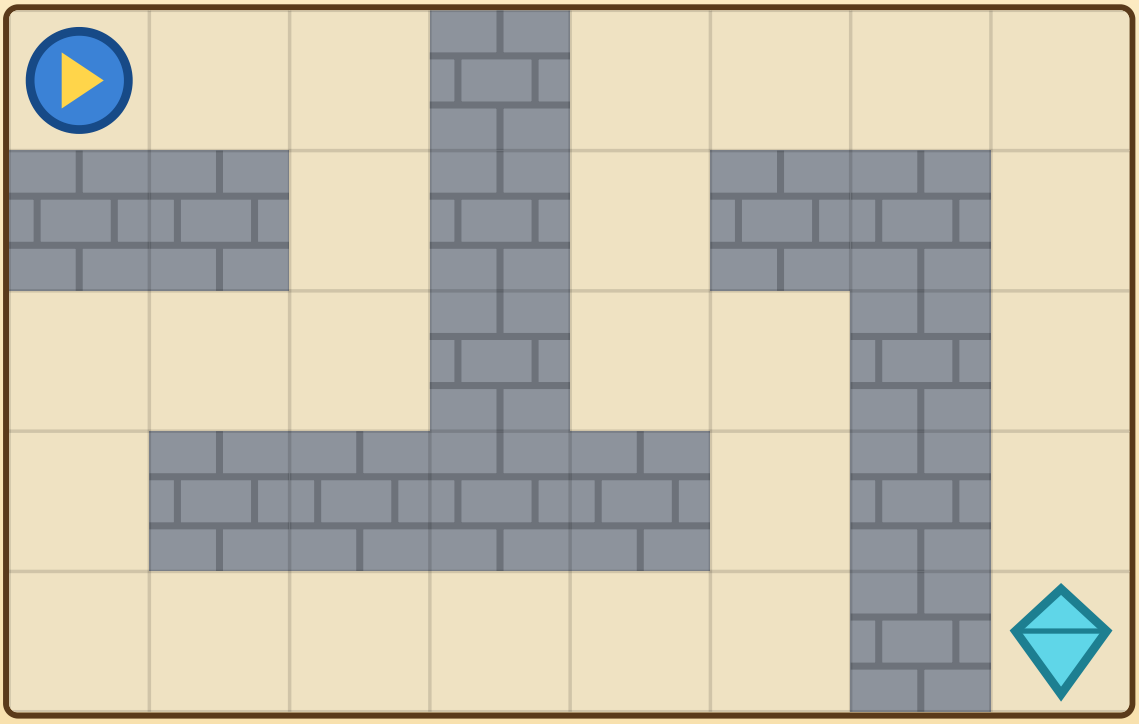
Related materials

Material	Contents
Guide (guide.html)	Goals, common mistakes, hints and answers for every task. This book is built from the same data.
Worksheets (PDF)	2 Drawing (angle table), 7 Python, 1 goal & reflection, 8 check card.
Lesson plans (Japanese)	Example 45-minute lessons (open them from each task in the Guide).
Videos (videos.html)	Short videos showing how to solve each task.
Lesson slides	01 Block Drawing, 05 Python Drawing (PowerPoint).

For kids and teachers

Learn Programming with Mazes

From blocks to Python — all 41 maze stages



National Institute of Technology, Maizuru College — Yasuhito Inoue

Contents

How to use this book 2

Part 1 The maze book for kids 3

Chapter 1 What is a program? 4

Chapter 2 The Arrow Maze 5

Chapter 3 The Block Maze 19

Chapter 4 The Keys & Doors Maze 37

Chapter 5 The Dodge-the-Slime Maze 47

Chapter 6 Mazes in Python 57

- Python Arrow Maze 68
- Python Keys & Doors 81
- Python Slime Maze 90

Well done! 100

Part 2 Guide for teachers and parents 101

- About this guide 102
- Overview of the stages 103
- Suggested order and lesson count 105
- Teaching points for each app 106
- Assessment (example) 108
- Using the apps and related materials 109

How to use this book

Use this book together with the “Block Maze” apps on a computer or tablet. You will think about and build programs that move the explorer to the diamond 💎.

1. Read the **problem** and look at the **maze**.
2. Pretend you are the explorer. Trace the path with your finger.
3. Build the blocks in the app and press “▶ Run”. If it doesn’t work, see where it stopped and fix it.
4. Stuck? Look at the 💡 **Hint**. When you are done, turn the page and check the 📖 **Answer** and the ✨ **Key point**.

There is more than one right answer. If you reach the goal, your program is correct, even if it is different from the book. Mistakes are hints — try again and again!

Mazes in this book

Maze	Stages	Grade
Arrow Maze	6	Grade 1+
Block Maze	8	Grade 2+
Keys & Doors Maze	4	Grade 4+
Dodge the Slime Maze	4	Grade 4+
Python mazes (4 apps)	19	Grade 5 – junior high

PART 1

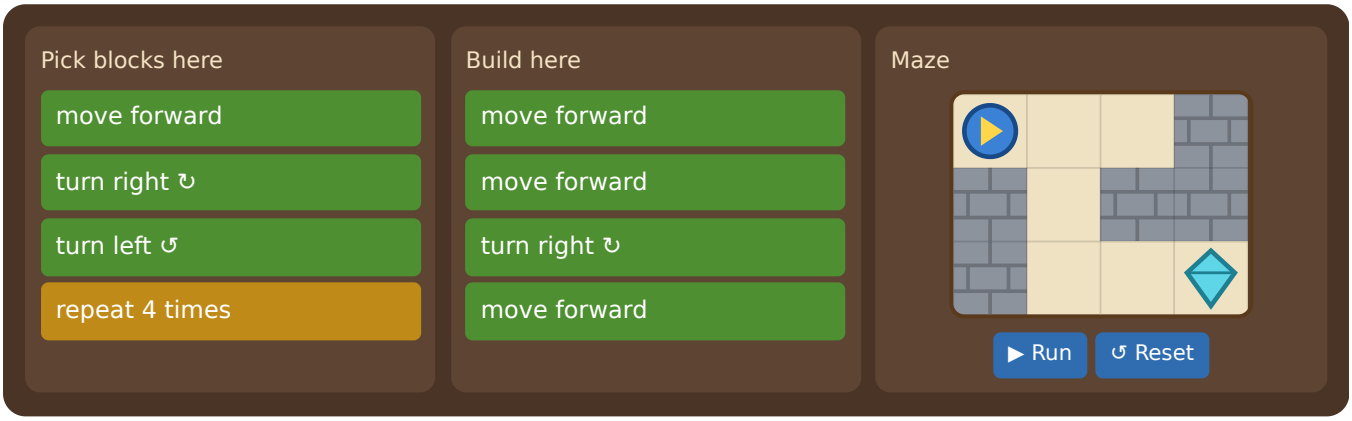
Part 1 The maze book for kids

Chapter 1

What is a program?

A **program** is a “to-do list” for a computer. A computer does exactly what is written, **in order from top to bottom**. It does not do anything that is not written.

In this book you give commands to the explorer and lead them to the goal. You make the commands by snapping “blocks” together.



What the screen looks like (sketch). Drag blocks from the left and stack them.

Which way is “forward”?

“Forward” is the way the explorer is facing. In this book, the triangle ▶ on the explorer points “forward”. “Turn right” means the explorer’s right. Turning the book, or turning your own body, helps.

When it doesn’t work

- Look where the explorer stopped, or where it bumped into a rock.
- Follow the program one block at a time with your finger.
- Finding and fixing mistakes is called **debugging**. Programmers do it every day.

Chapter 2

🧭 The Arrow Maze

Move the explorer with up ⬆, down ⬇, left ⬅ and right ➡ arrows. You don’t need to think about which way the explorer faces. One arrow = one square. This first maze is about **order**.

Commands in this chapter

up ⬆ / down ⬇ / left ⬅ / right ➡ / repeat ○ times (stage 6)

Maze symbols


Start (the explorer). The triangle shows "forward"


Rock (you cannot pass)


Key


Slime (goes back and forth)


Bushes (Arrow Maze)


The goal diamond

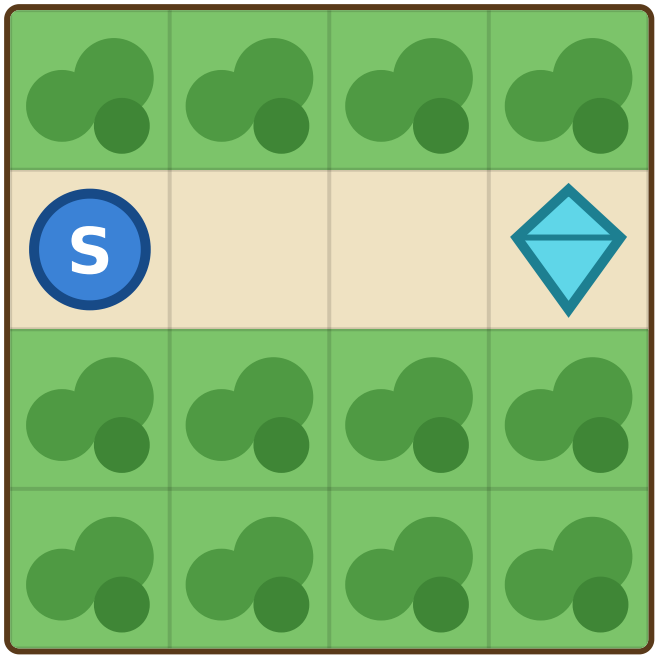

Path


Door (opens with a key)


The slime's track

Stage 1: Go right

Sequence



Problem

Stack three "right ➡" arrows and go to the diamond 💎.

🎯 **Goal** : Line up arrows to move

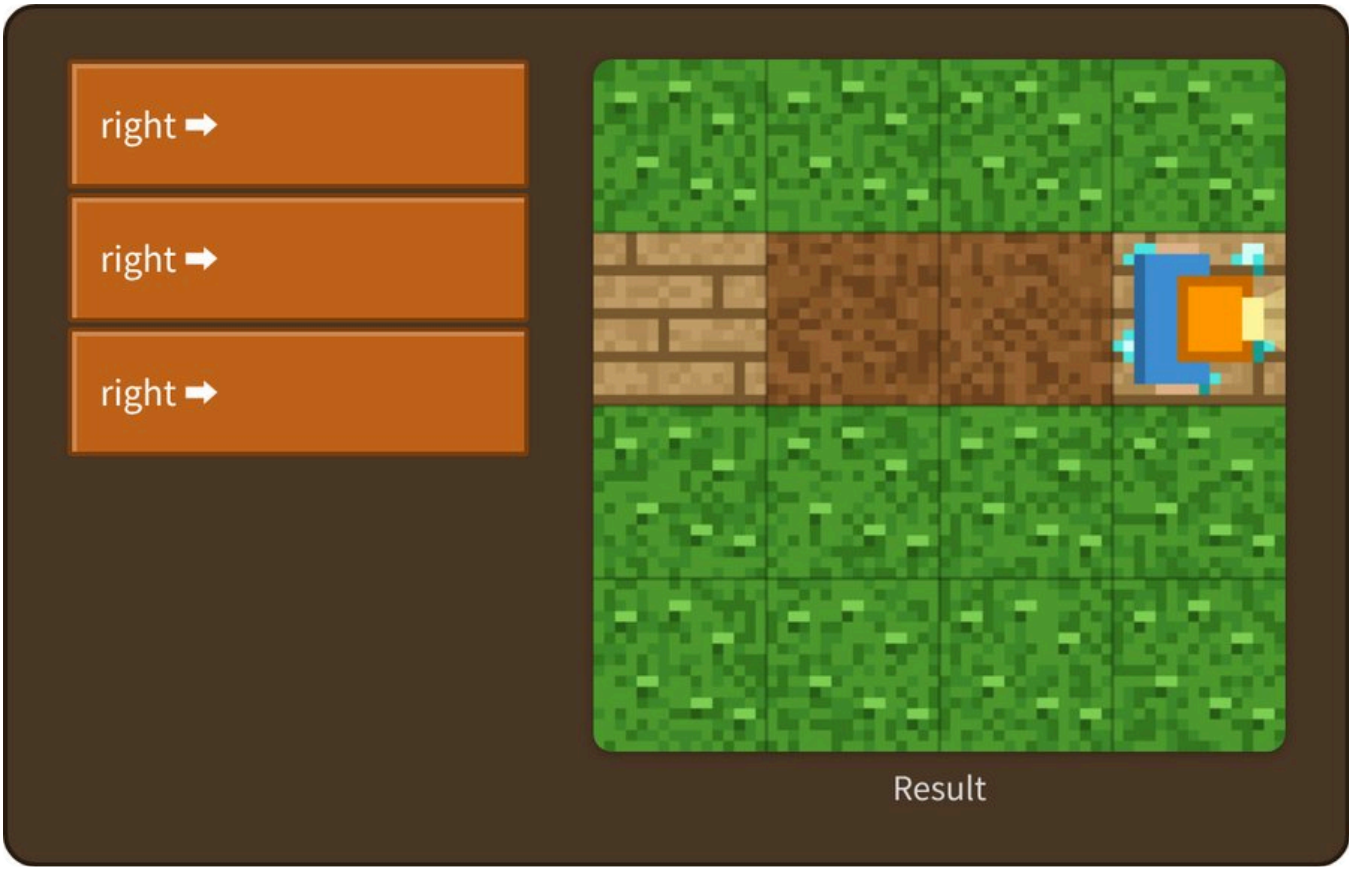
💡 **Hint (look if you get stuck)**

The diamond is 3 squares away.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go right



✨ Key point

One arrow moves one square. Count the squares to the diamond and stack that many.

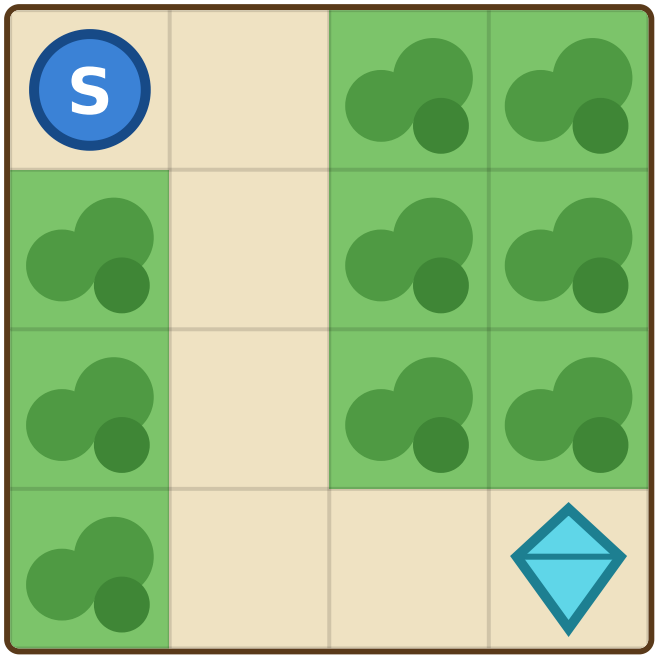
⚠ Common mistake

With 2 arrows you fall short; with 4 you bump into the bushes.

🎬 Video: `videos/en/mazejr-01.mp4`

Stage 2: Go down

Sequence Direction



Problem

Use "right ➡" and "down ⬇".

🎯 **Goal**：Use two kinds of arrows

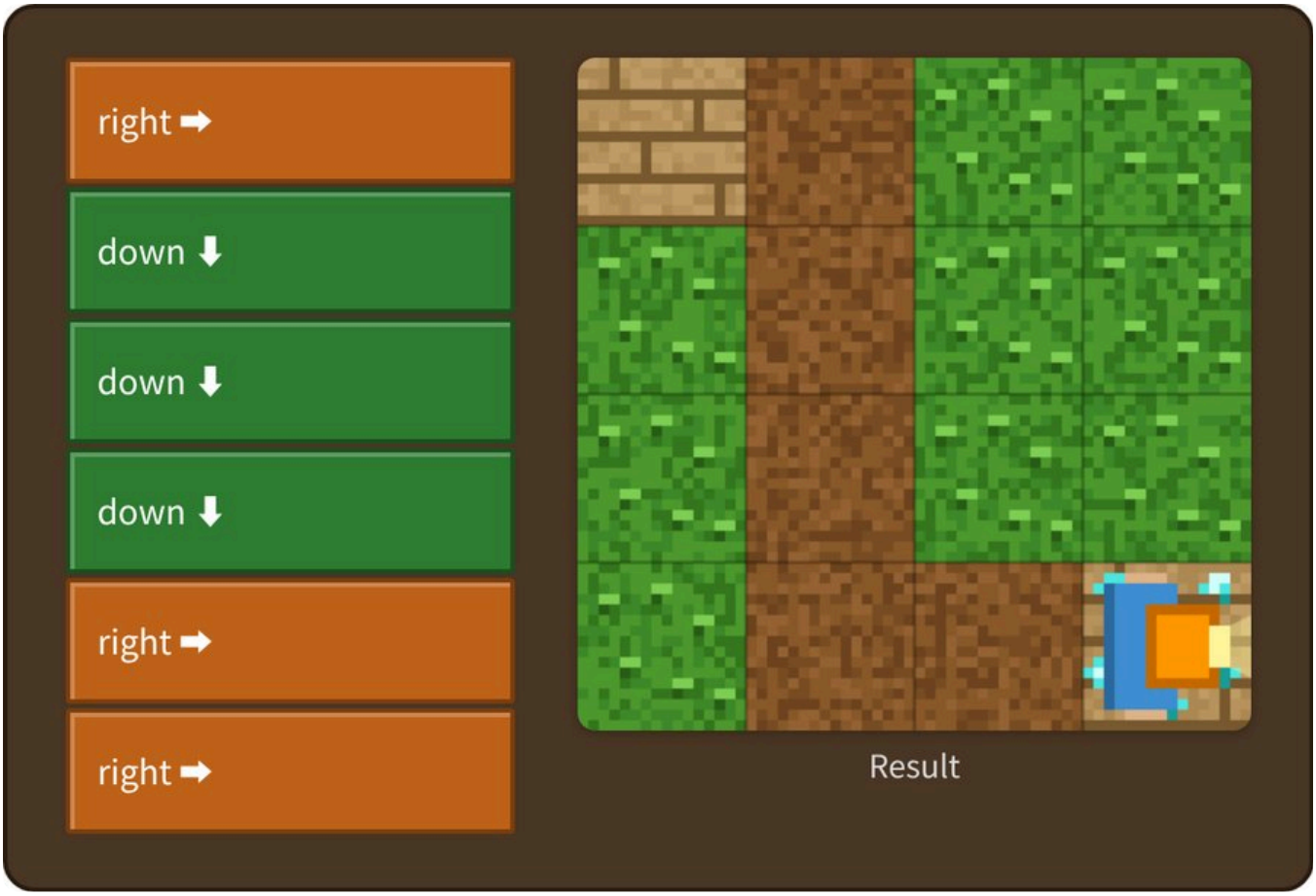
💡 **Hint (look if you get stuck)**

right, then down 3 times, then right 2 times.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Go down



✨ Key point

Right → down → right. Put the arrows in the order you go.

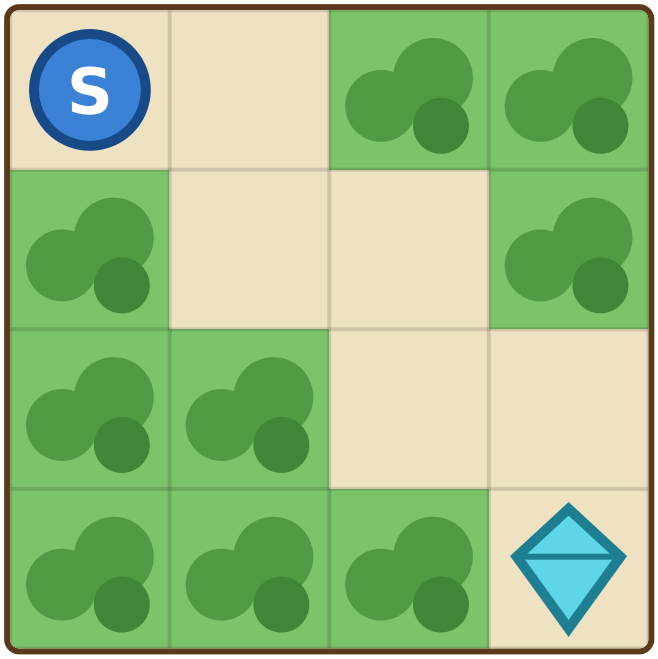
⚠ Common mistake

If you use the wrong number of “down” arrows, you bump into the bushes.

🎬 Video: `videos/en/mazejr-02.mp4`

Stage 3: Stairs

Sequence Patterns



Problem

Right and down, one after the other.

🎯 **Goal**：Find the alternating pattern

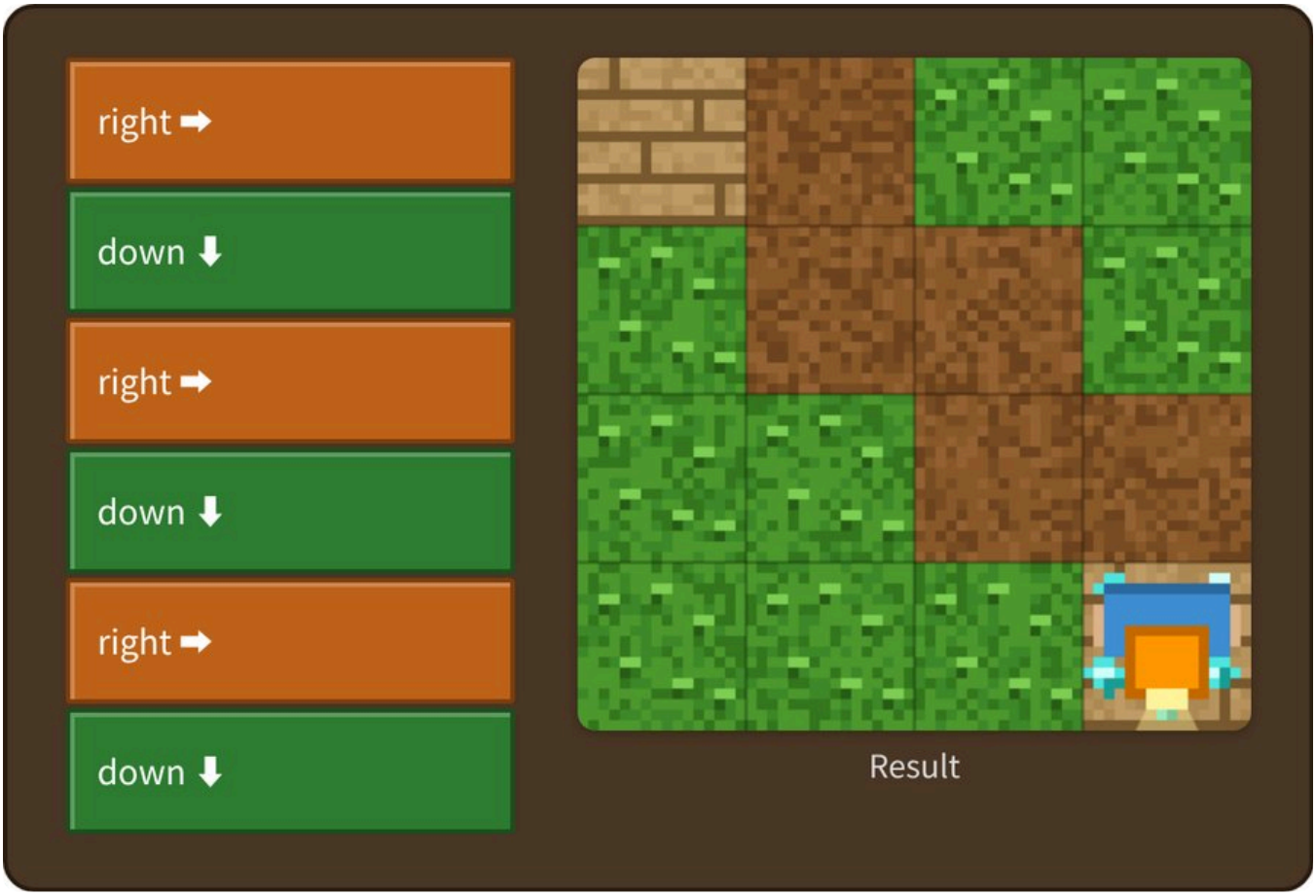
💡 **Hint (look if you get stuck)**

right, down, right, down, right, down

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Stairs



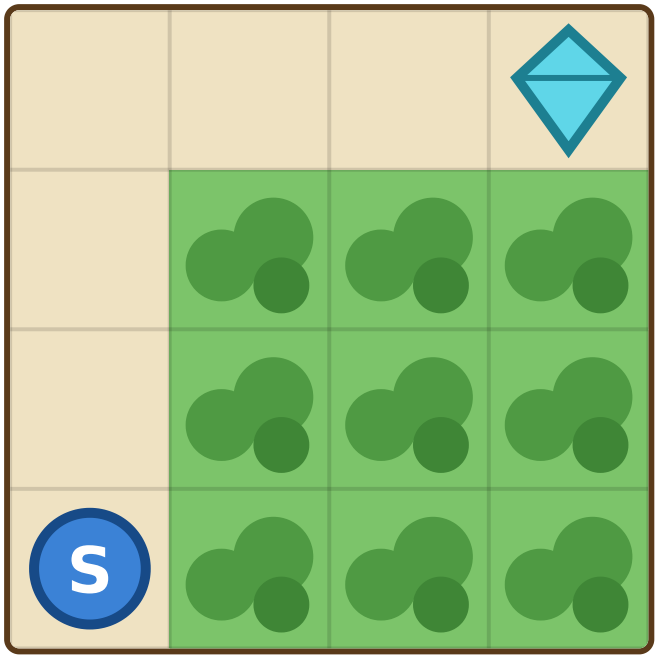
> ✨ **Key point**
>
> Right, down, right, down… the same pair keeps going.

> ⚠ **Common mistake**
>
> Two “right” arrows in a row bump into the bushes.

🎬 Video: `videos/en/mazejr-03.mp4`

Stage 4: Go up

Sequence Direction



Problem

Now use "up ⬆" too.

🎯 **Goal** : Use “up” too

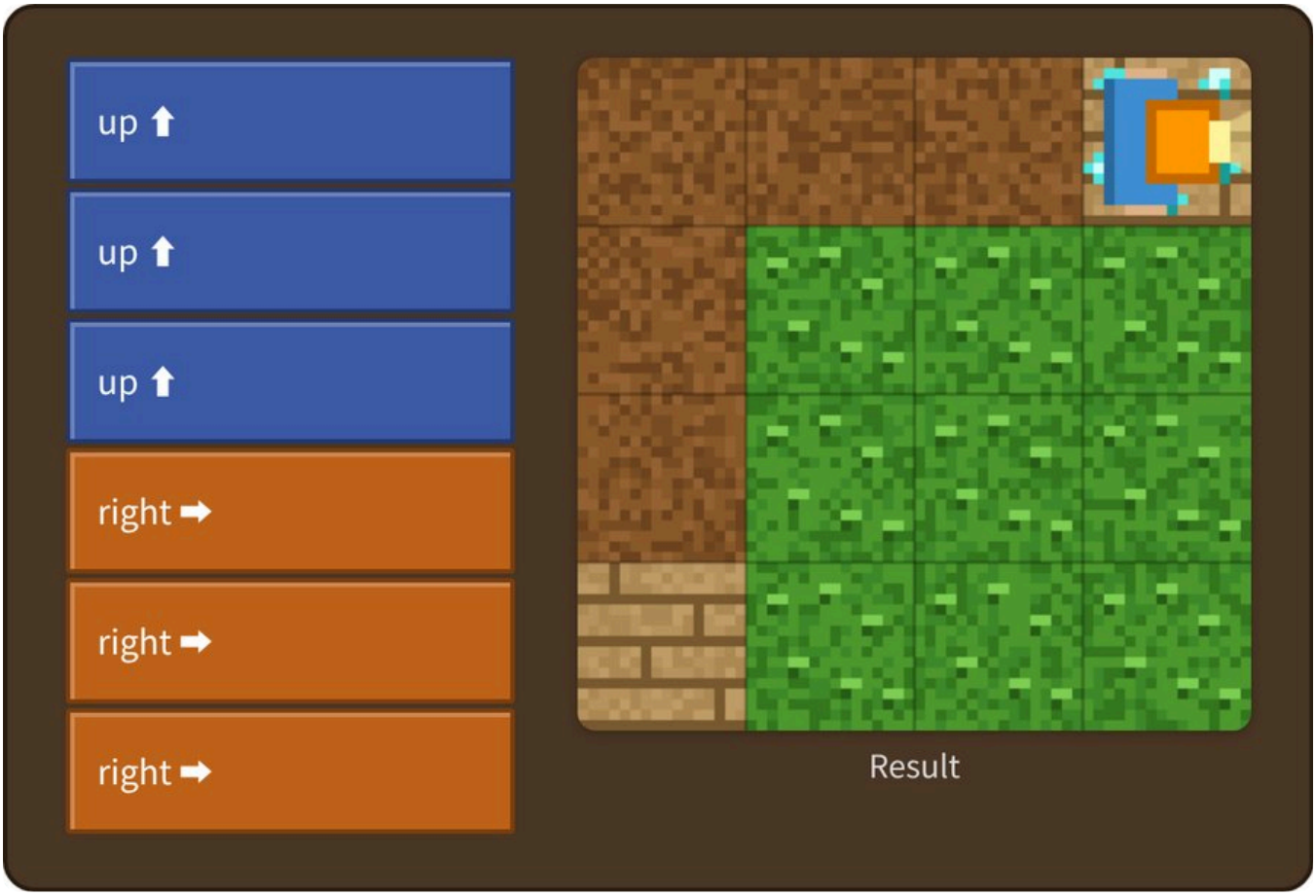
💡 **Hint (look if you get stuck)**

up 3 times, then right 3 times.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Go up



✨ **Key point**

You start at the bottom left. First up 3, then right 3.

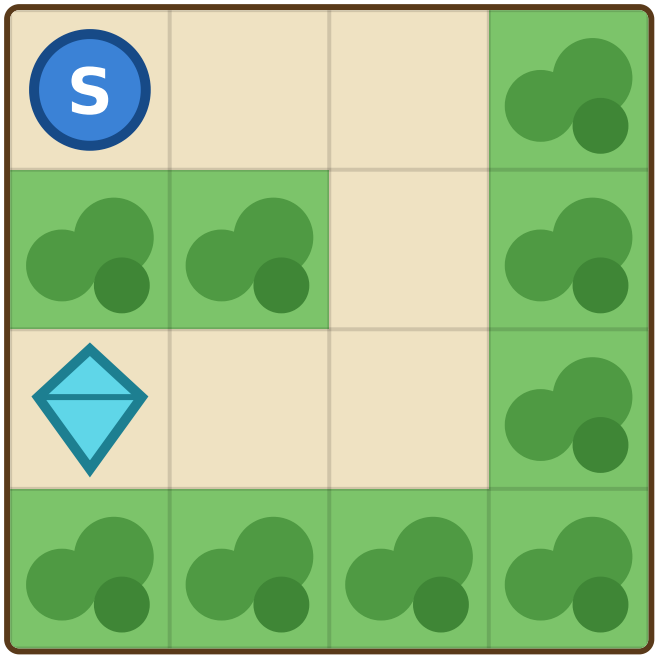
⚠ **Common mistake**

If you start with “right”, you bump into the bushes right away.

🎬 Video: `videos/en/mazejr-04.mp4`

Stage 5: Go around

Sequence Direction



Problem

Use all four arrows.

🎯 **Goal** : Use all four arrows

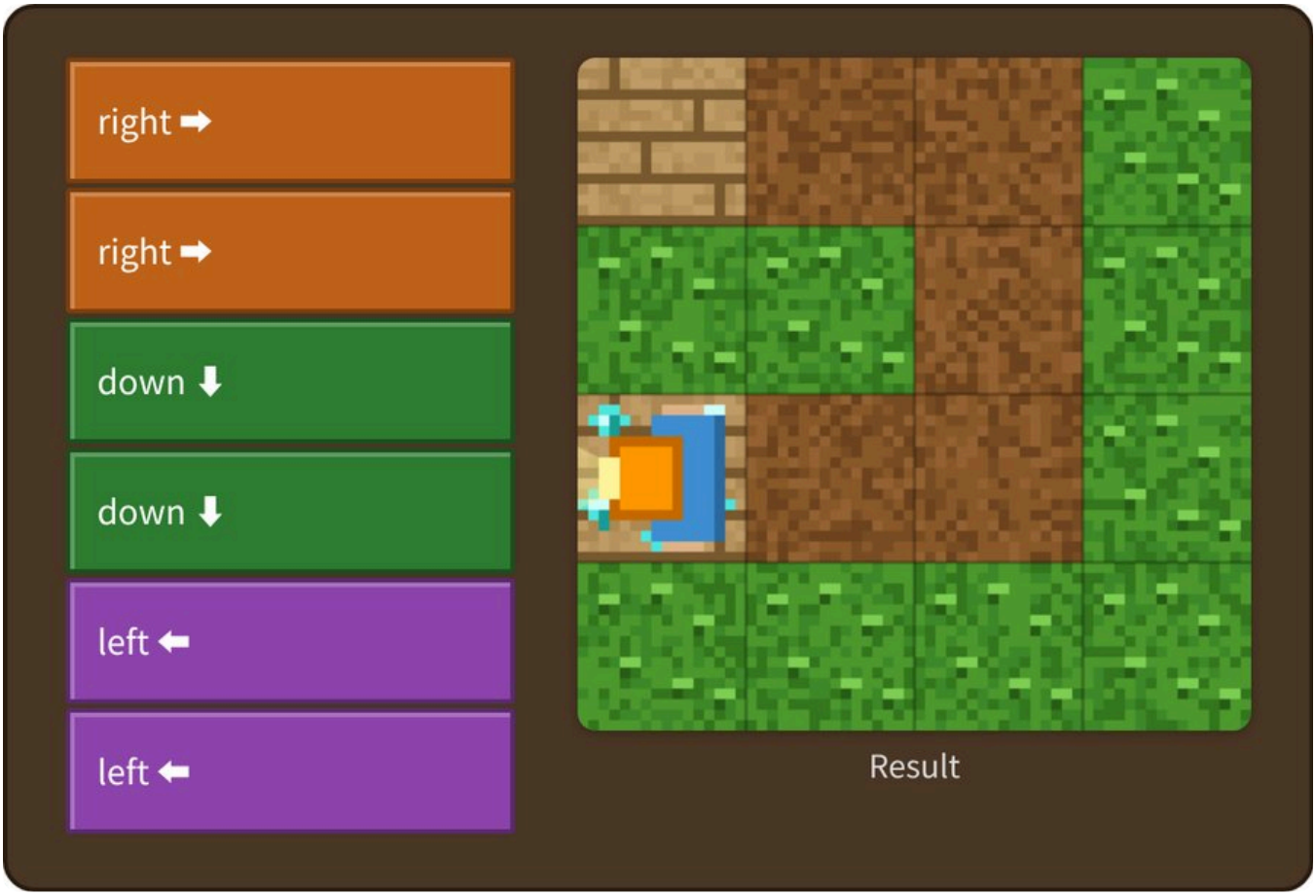
💡 **Hint (look if you get stuck)**

right 2, down 2, left 2.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Go around



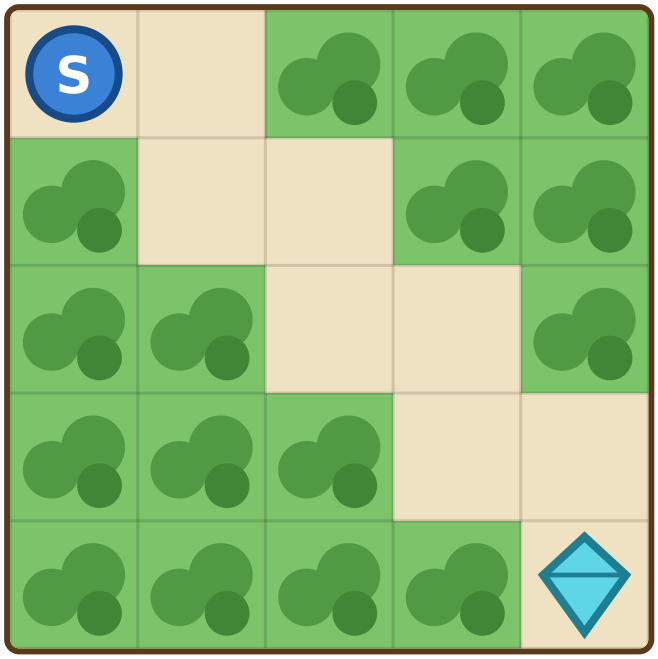
> ✨ **Key point**
>
> Right → down → left. Go around to the diamond.

> ⚠ **Common mistake**
>
> “Left” and “right” are easy to mix up. They mean left and right on the screen.

🎬 Video: `videos/en/mazejr-05.mp4`

Stage 6: Use repeat

Loops Patterns



Problem

A long staircase! Put "right ➡ down ⬇" inside "repeat".

🎯 Target: 3 blocks or fewer

🎯 **Goal**：Make it shorter with repeat

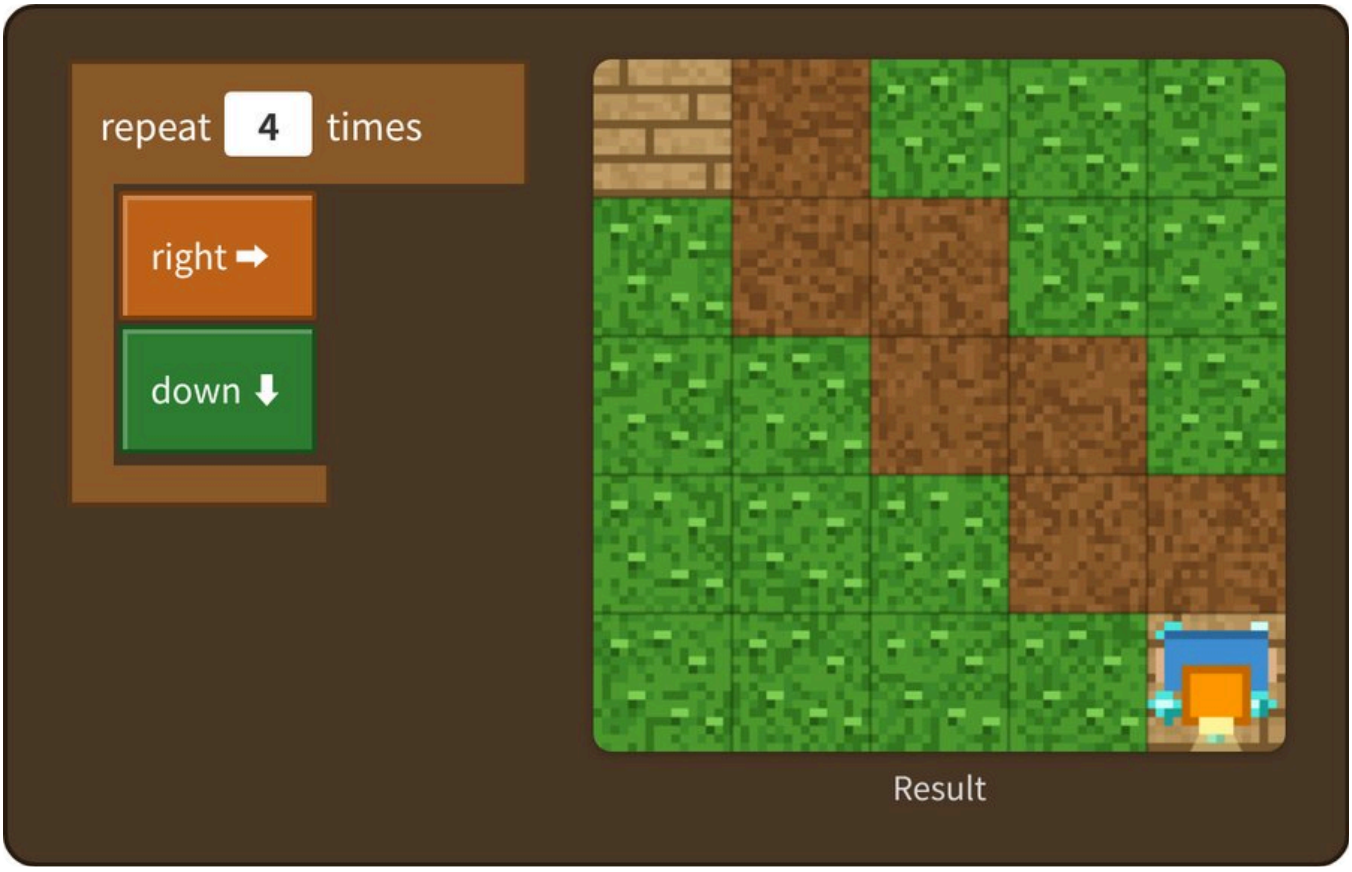
💡 **Hint (look if you get stuck)**

repeat 4 times: right, down

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Use repeat



> ✨ **Key point**
>
> Repeat the “right, down” set 4 times. Just 3 blocks!

> ⚠ **Common mistake**
>
> With 3 repeats you stop just before the diamond.

🎬 Video: `videos/en/mazejr-06.mp4`

Column

Count the squares

In the arrow maze, **counting squares** is the most important thing. Touch each square with your finger and count “1, 2, 3”, then stack that many arrows. When the same arrow repeats, you can pack them into a “repeat” block.

Chapter 3

⛏️ The Block Maze

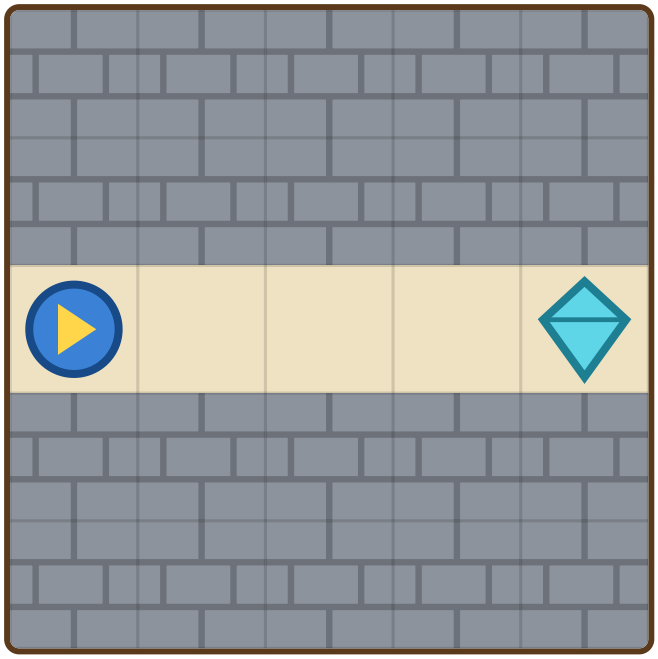
Now the explorer has a **direction**. Use “move forward”, “turn right” and “turn left”. This central chapter goes from **order → loops → if → a way to solve any maze**.

Commands in this chapter

move forward / turn right / turn left / repeat ○ times / repeat until goal / if front (right, left) is open

Stage 1: Go straight

Sequence



Problem

Move the explorer to the diamond 💎. Drag "move forward" blocks from the left and stack them.

🎯 **Goal**：Put the steps in order

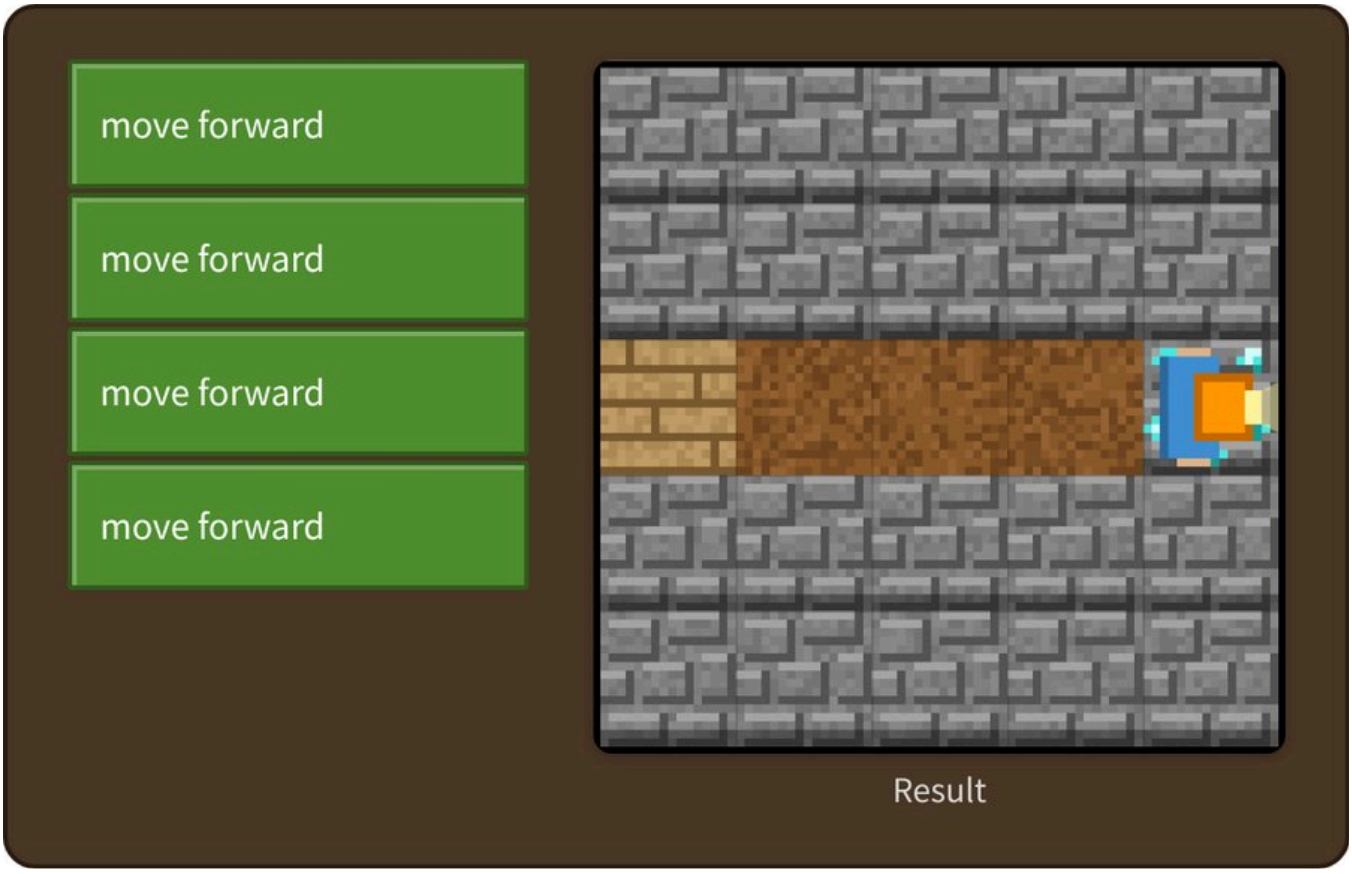
💡 **Hint (look if you get stuck)**

How many squares to the goal? Stack that many "move forward" blocks.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go straight



✨ Key point

Commands run in order from top to bottom. You stacked four “move forward” blocks.

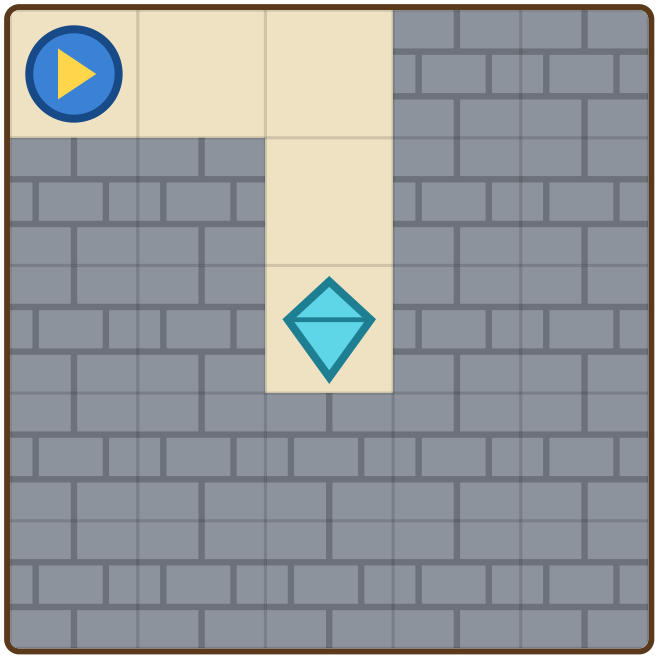
⚠ Common mistake

Blocks that are not connected do not run.

🎬 Video: `videos/en/maze-01.mp4`

Stage 2: Let's turn

Sequence Direction



Problem

You turn on the way. "Forward" is where the explorer's light (yellow beam) is shining.

🎯 **Goal**：Think about direction

💡 **Hint (look if you get stuck)**

After 2 squares, "turn right". Turning does not move you to another square.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Let's turn



✨ Key point

“Turn” only changes the direction. The explorer stays on the same square.

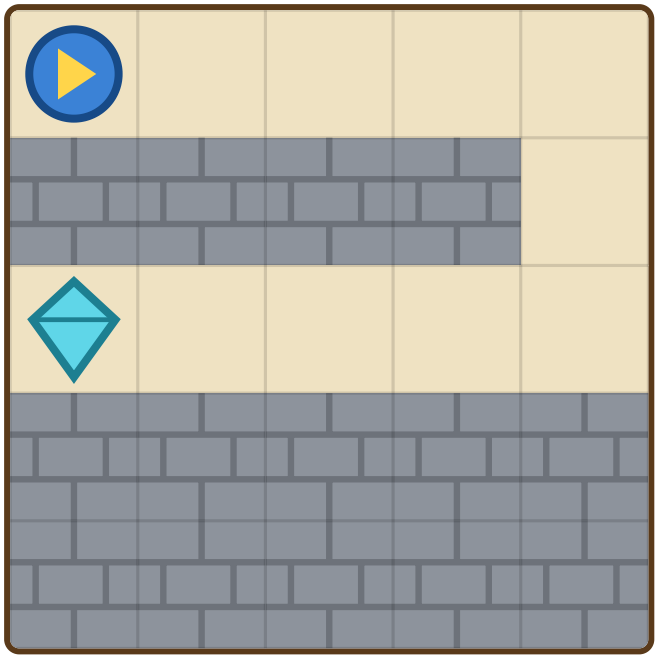
⚠ Common mistake

Turning alone does not move you. Add “move forward” after turning.

🎬 Video: `videos/en/maze-02.mp4`

Stage 3: A long path

Sequence Direction



Problem

Walk the long path. Pretend you are the explorer and think about which way to turn.

🎯 **Goal**：Build a long set of steps

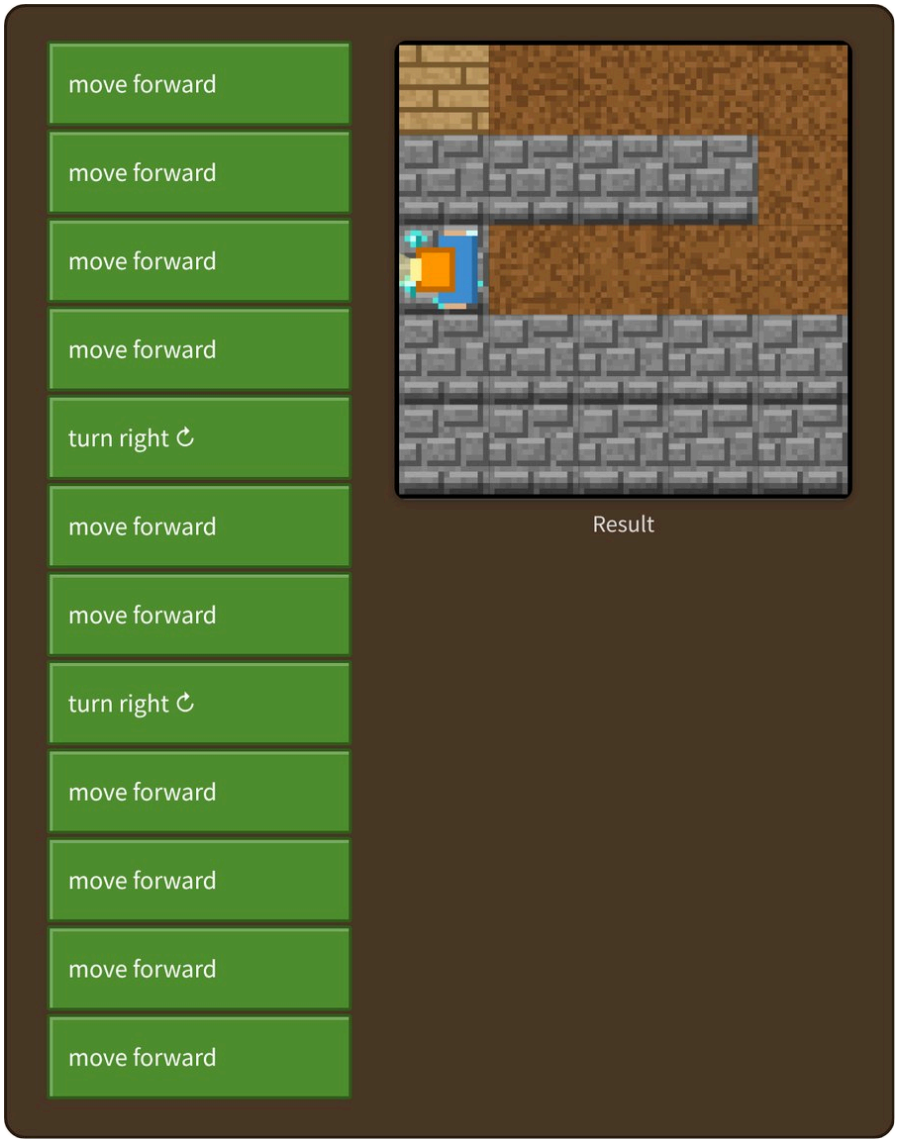
💡 **Hint (look if you get stuck)**

Forward 4 → turn right → forward 2 → turn right → forward 4

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: A long path



✨ Key point

That took 12 blocks! Next, you will learn a way to make it shorter.

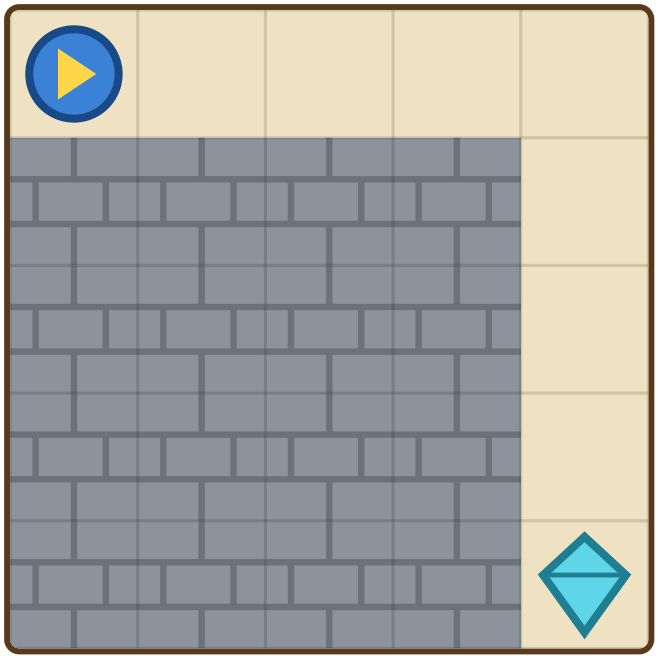
⚠ Common mistake

It is easy to mix up left and right. Imagine you are the explorer.

🎬 Video: `videos/en/maze-03.mp4`

Stage 4: Use a loop

Loops



Problem

Stacking the same block again and again is hard work. Put blocks inside "repeat" to repeat them.

🎯 Target: 5 blocks or fewer

🎯 **Goal**：Make it shorter with a loop

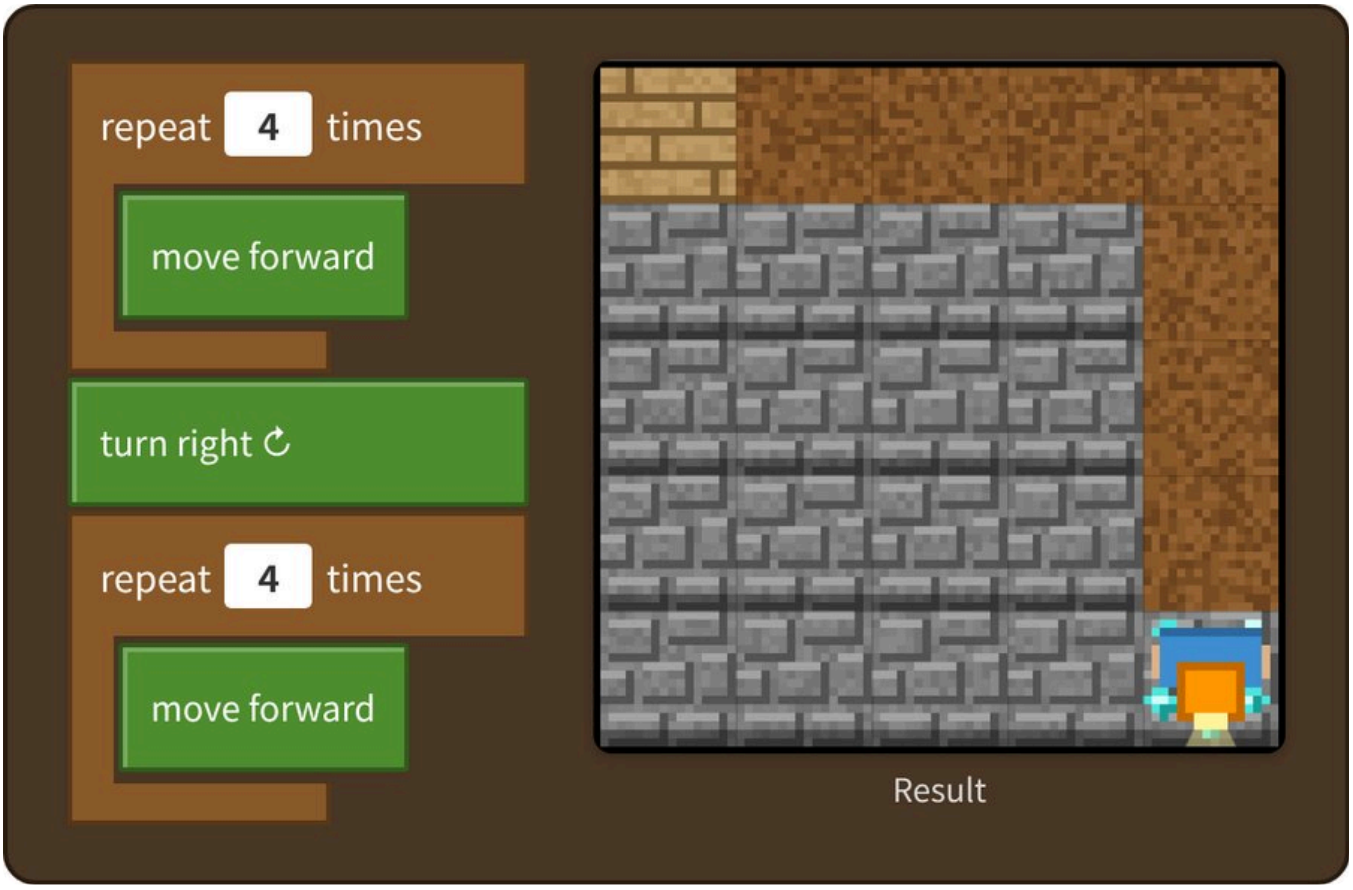
💡 **Hint (look if you get stuck)**

"repeat 4 (move forward)" → "turn right" → "repeat 4 (move forward)"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Use a loop



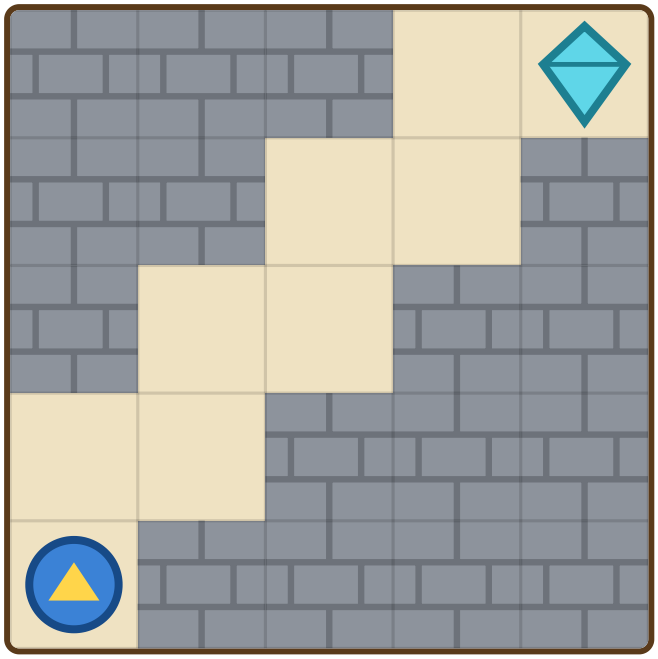
> ✨ **Key point**
>
> Repeated blocks can be grouped with “repeat __ times”.

> ⚠ **Common mistake**
>
> Putting “turn right” inside the loop changes the movement.

🎬 Video: `videos/en/maze-04.mp4`

Stage 5: Climb the stairs

Loops Patterns



Problem

Look closely at the stairs. The same set of moves repeats.

🎯 Target: 5 blocks or fewer

🎯 **Goal**：Find the pattern that repeats

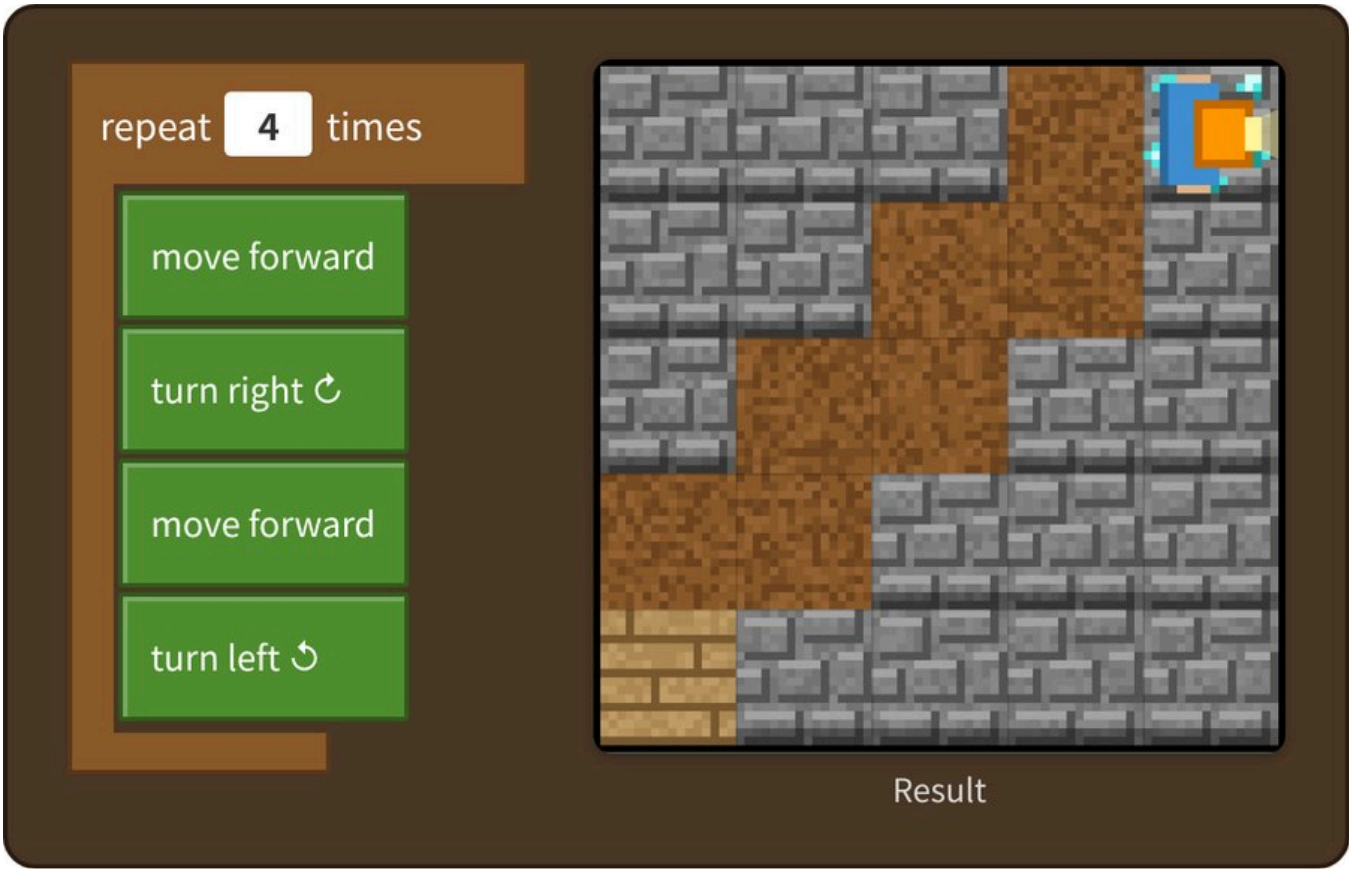
💡 **Hint (look if you get stuck)**

One set is "forward → turn right → forward → turn left". How many times should it repeat?

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Climb the stairs



✨ Key point

One set is forward, right, forward, left. The program stops when it reaches the goal.

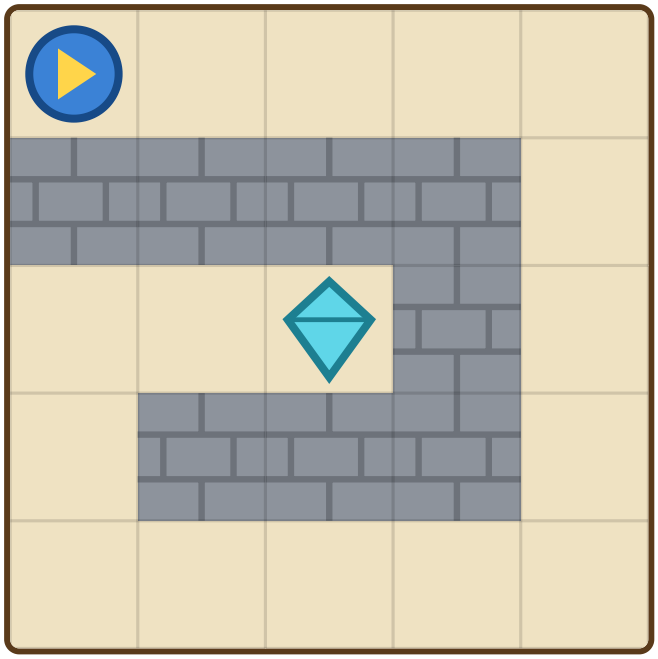
⚠ Common mistake

It is easy to get the order of the set wrong (forward → right → forward → left).

🎬 Video: `videos/en/maze-05.mp4`

Stage 6: If…

Conditions | Repeat until



Problem

Try "repeat until goal" and "if front is open". The explorer looks ahead and decides to move or turn by itself.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Let the explorer decide with “if”

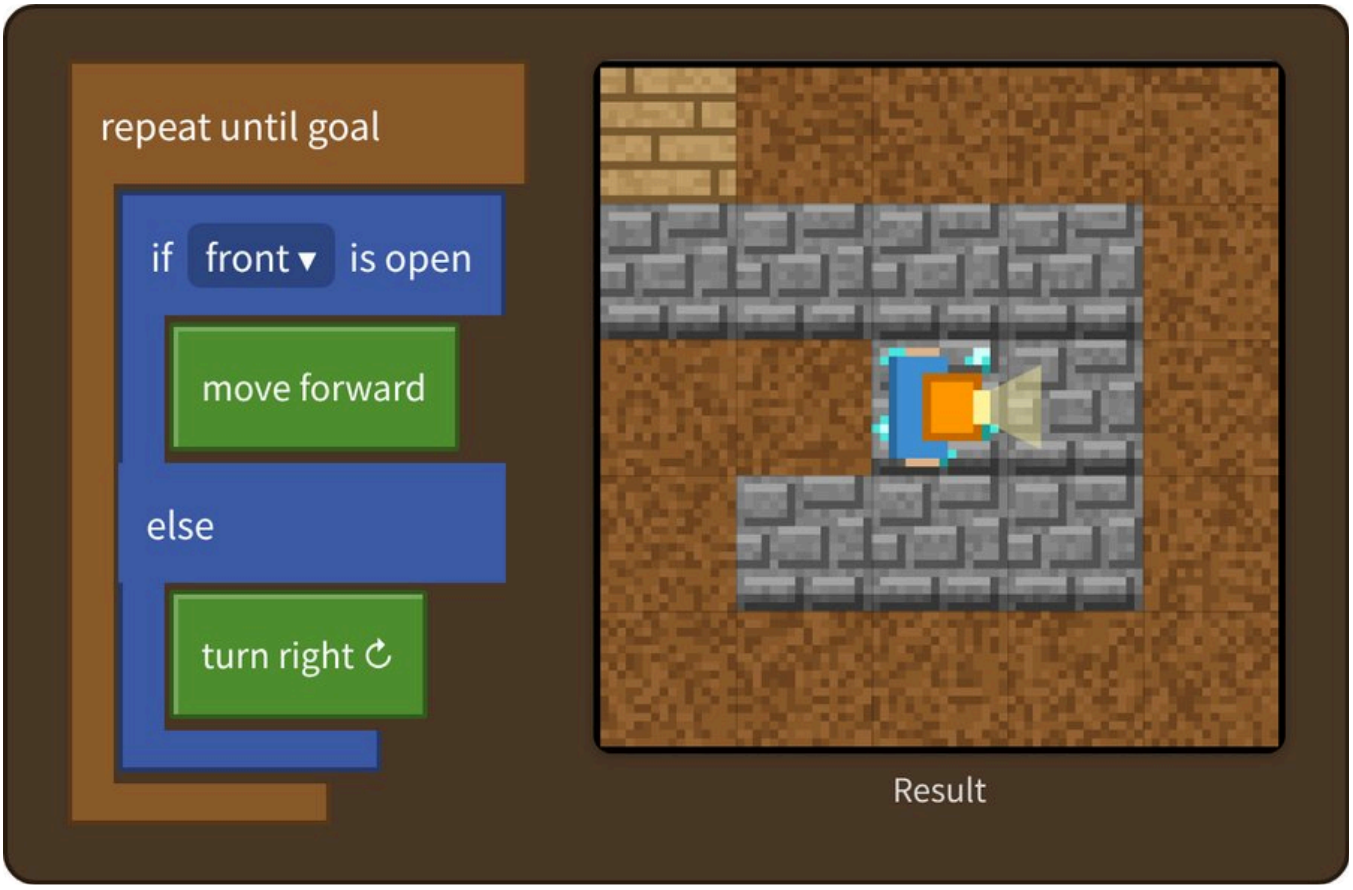
💡 **Hint (look if you get stuck)**

Repeat until goal: if front is open "move forward", else "turn right".

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: If…



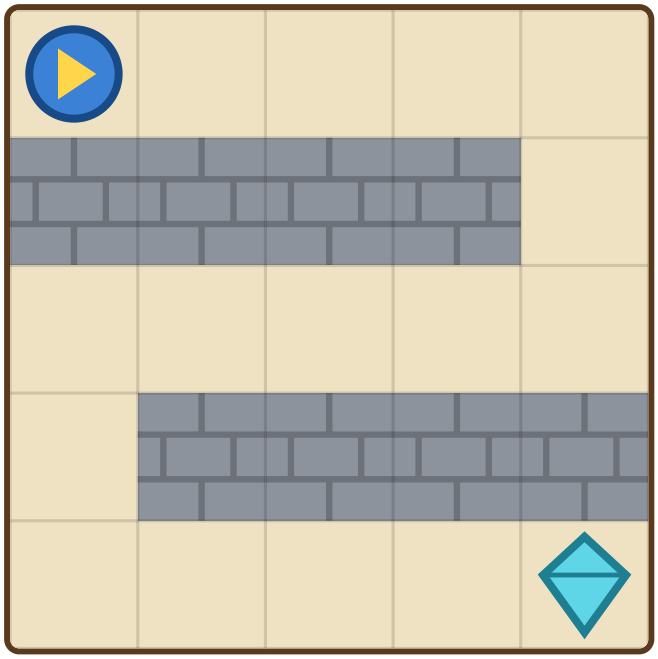
> ✨ **Key point**
>
> If the front is open, move. If not, turn right. The explorer decided by itself!

> ⚠ **Common mistake**
>
> If the “if” is outside the loop, it only checks once.

🎬 Video: `videos/en/maze-06.mp4`

Stage 7: Right? Left?

Conditions Nesting



Problem

Now you turn both right and left. If the front is blocked, check the right. If that is blocked too, turn left.

🎯 Target: 6 blocks or fewer

🎯 **Goal**：Put an “if” inside an “if”

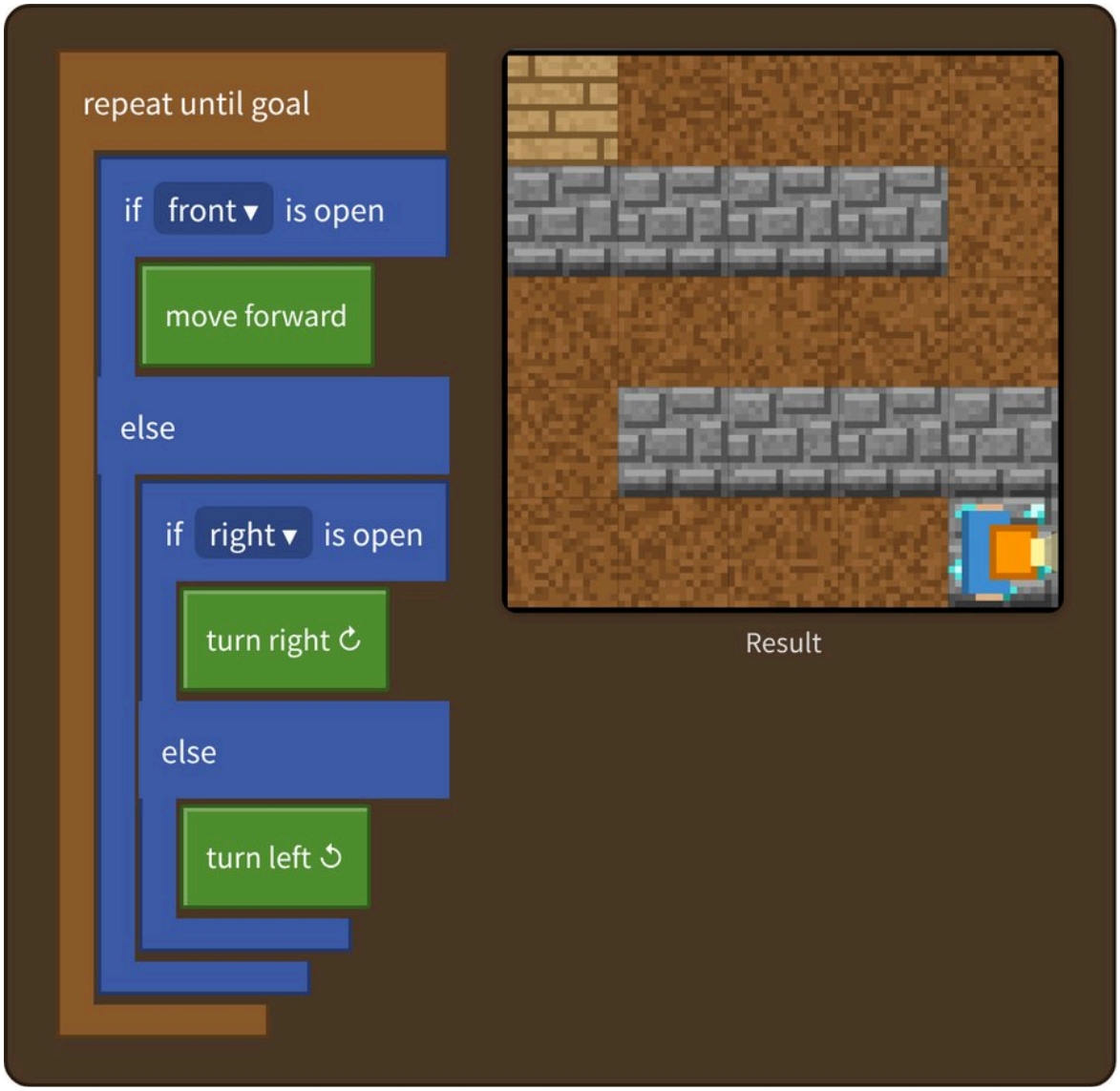
💡 **Hint (look if you get stuck)**

Put another "if right is open" inside "else".

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 7: Right? Left?



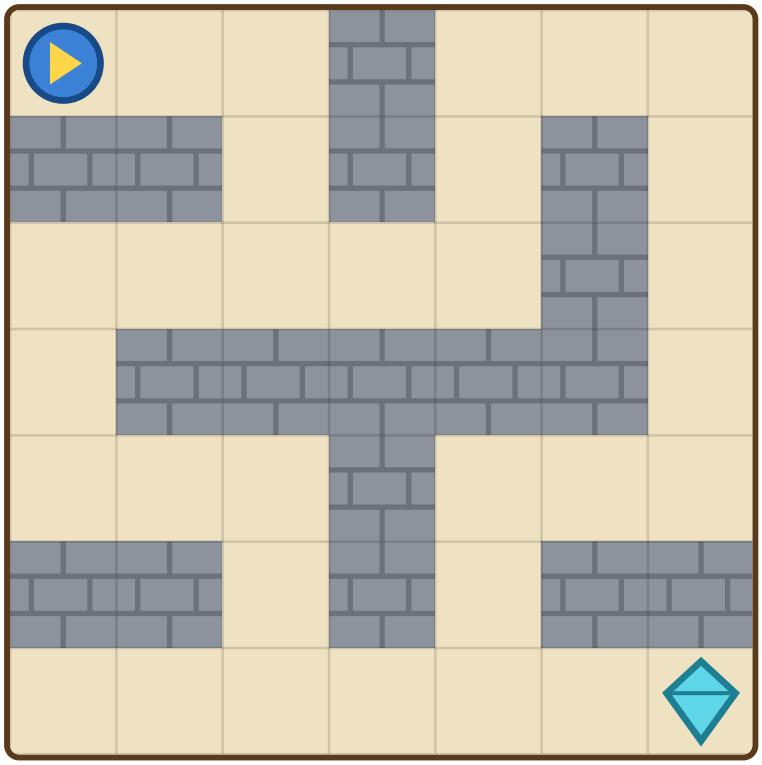
> ✨ **Key point**
>
> Another “if” inside “else” lets it choose right or left.

> ⚠ **Common mistake**
>
> With the Stage 6 program, you hit a dead end after turning.

🎬 Video: `videos/en/maze-07.mp4`

Stage 8: Any maze!

Generalization | Conditions | Repeat until



Example: the maze changes every time you run

Problem

Final challenge! Your program is tested on 30 random mazes. Reach the goal in all of them to clear!

🎯 **Goal**：A way to solve any maze

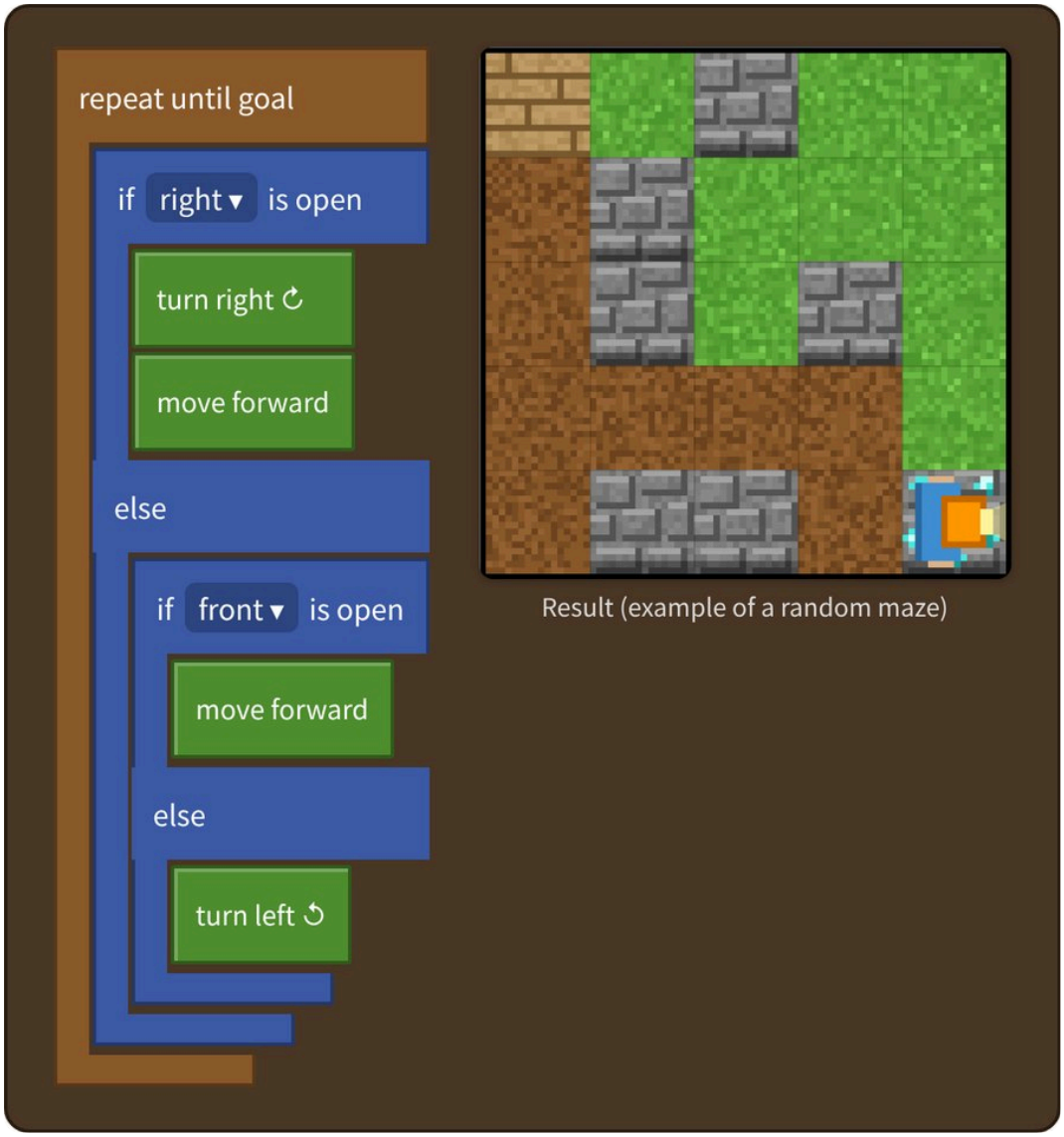
💡 **Hint (look if you get stuck)**

The "right-hand rule": keep your right hand on the wall and you always reach the goal. If the right is open, turn right first, then move.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 8: Any maze!



Result (example of a random maze)

> ✨ **Key point**
>
> The “right-hand rule”: keep your right hand on the wall, and you can solve any maze!

> ⚠ **Common mistake**
>
> The Stage 7 program can keep going in circles in some mazes.

🎬 Video: `videos/en/maze-08.mp4`

Column

The right-hand rule

Even in a dark maze, if you walk **keeping your right hand on the wall**, you will always reach the exit (when all the walls are connected). This is the “right-hand rule”.

As a program: **repeat until goal** — if the right is open, turn right and move; else if the front is open, move; otherwise turn left. Stage 8 can be solved this way. A method that always works is called an **algorithm**.

Chapter 4

🗝️ The Keys & Doors Maze

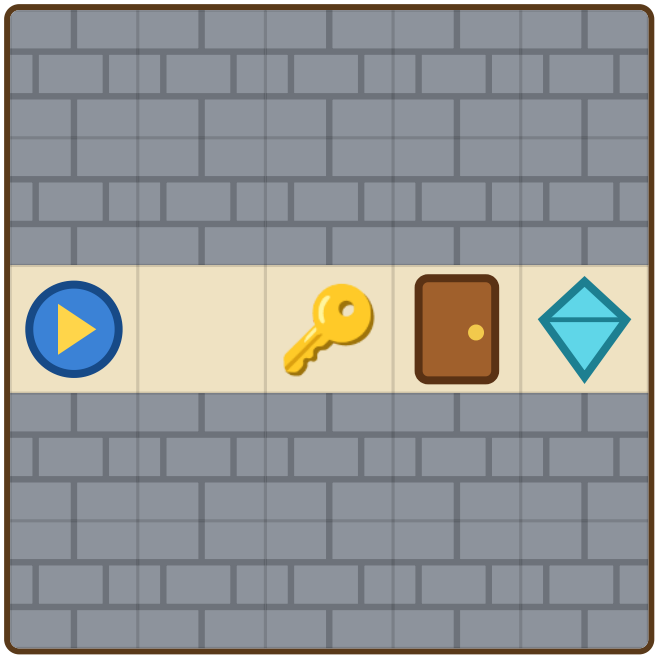
Pick up keys 🔑 and open doors 🚪 as you go. You must remember “how many keys am I carrying now?” This is called **state**.

Commands in this chapter

move forward / turn right, left / pick up key / open door / repeat / if key here / if door ahead

Stage 1: Pick up the key, open the door

Sequence State



Problem

Move onto the key 🔑 and use "pick up key". In front of the door 🚪, use "open door" and you can go through.

🎯 **Goal**：Pick up the key and open the door

💡 **Hint (look if you get stuck)**

forward → forward → pick up key → open door → forward → forward

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Pick up the key, open the door



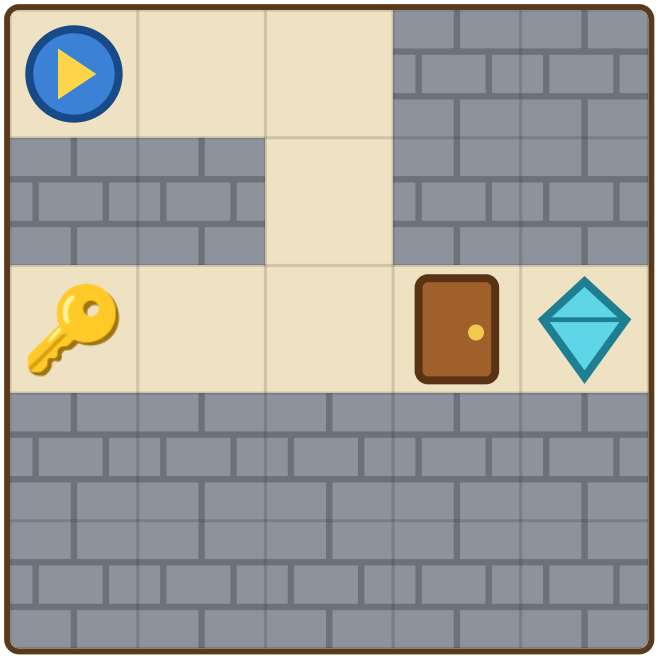
> ✨ **Key point**
>
> “Pick up key” makes your key count 1, and “open door” brings it back to 0.

> ⚠ **Common mistake**
>
> Picking up when you are not on the key gives “There is no key here.” Check how far you move.

🎬 Video: `videos/en/mazekey-01.mp4`

Stage 2: Where is the key?

Sequence State Decomposition



Problem

The diamond is behind the door. Go and get the key first. Without a key, the door will not open.

🎯 **Goal**：Get the key first

💡 **Hint (look if you get stuck)**

Walk to the key and pick it up. Then turn right twice and go back.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Where is the key?



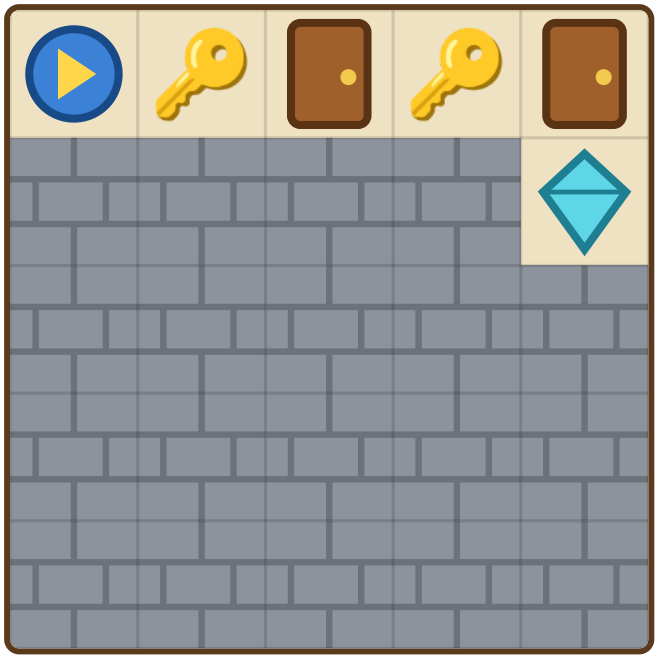
> ✨ **Key point**
>
> Take a side trip before the goal. Think of it as “get the key” and “go back to the door”.

> ⚠ **Common mistake**
>
> Going to the door without a key gives “The door is locked. Where is the key?”

🎬 Video: `videos/en/mazekey-02.mp4`

Stage 3: Two doors

Loops Patterns State



Problem

Keys and doors are in a row. Find the set of moves that repeats and use "repeat".

🎯 Target: 7 blocks or fewer

🎯 **Goal**：Repeat the same set

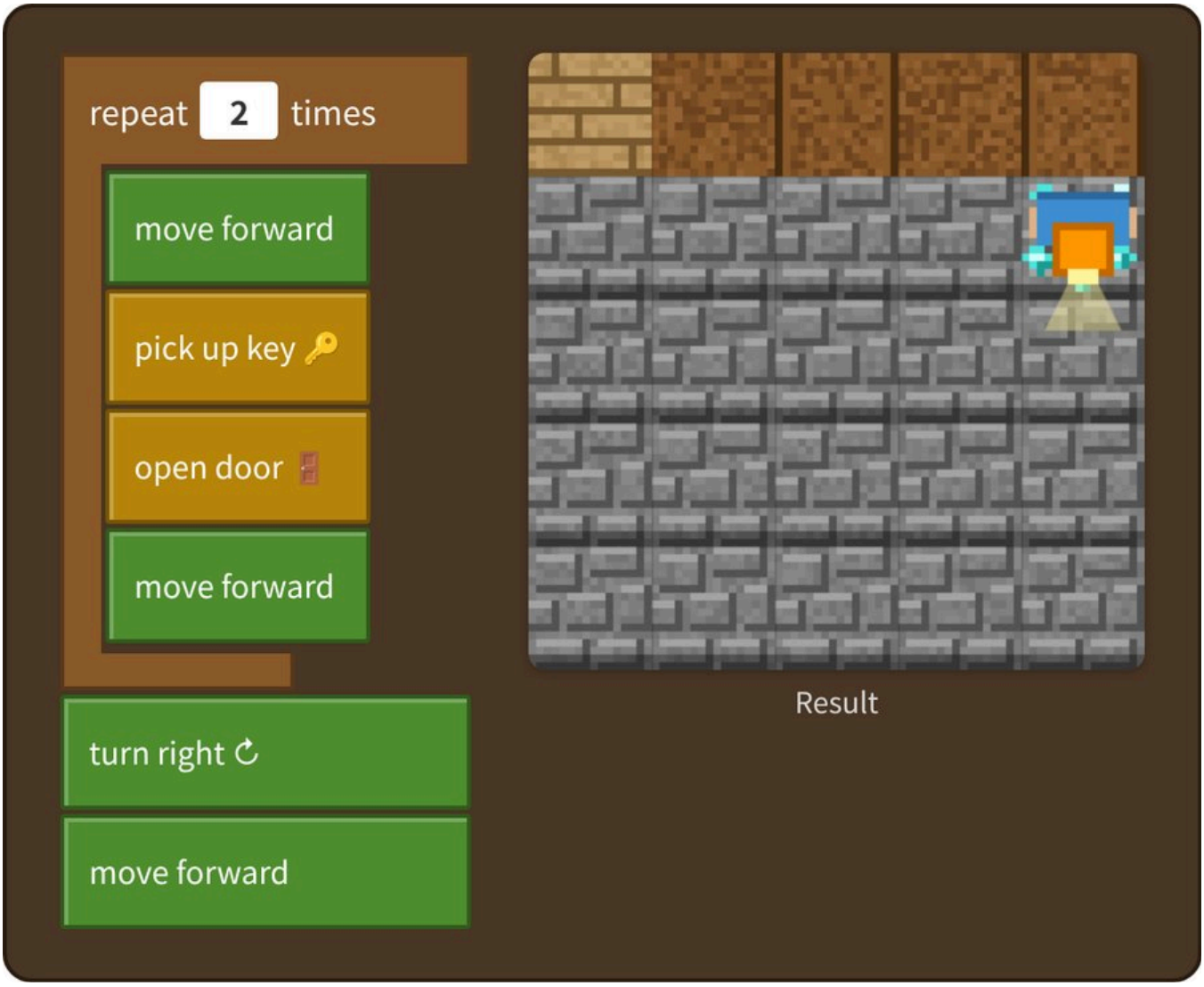
💡 **Hint (look if you get stuck)**

Repeat 2 times: forward → pick up key → open door → forward. Then turn right and move forward.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 3: Two doors



✨ Key point

Key → door appears twice. Repeat the 4 blocks 2 times.

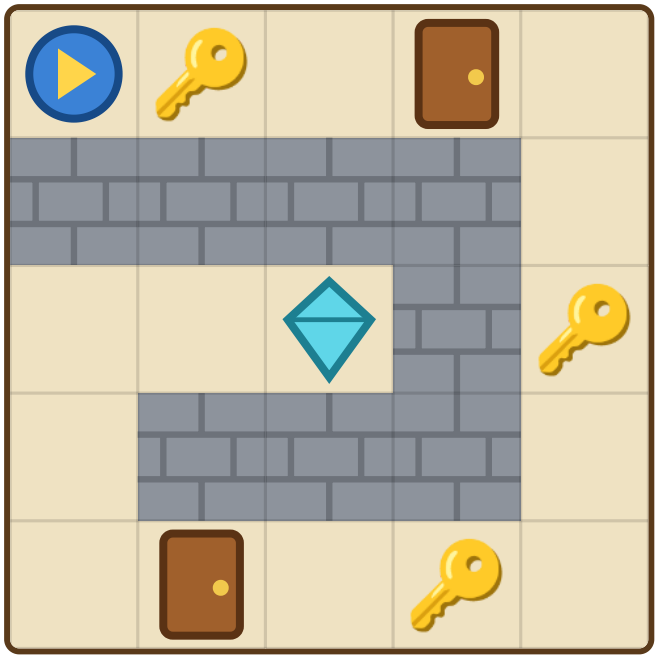
⚠ Common mistake

If you forget “move forward” after “open door”, you never reach the next key.

🎬 Video: `videos/en/mazekey-03.mp4`

Stage 4: A thinking explorer

Conditions | Repeat until | State | Generalization



Problem

Use "repeat until goal" and "if". Make a program that picks up a key when it finds one and opens a door when there is one.

🎯 Target: 8 blocks or fewer

🎯 **Goal**：Check and decide by yourself

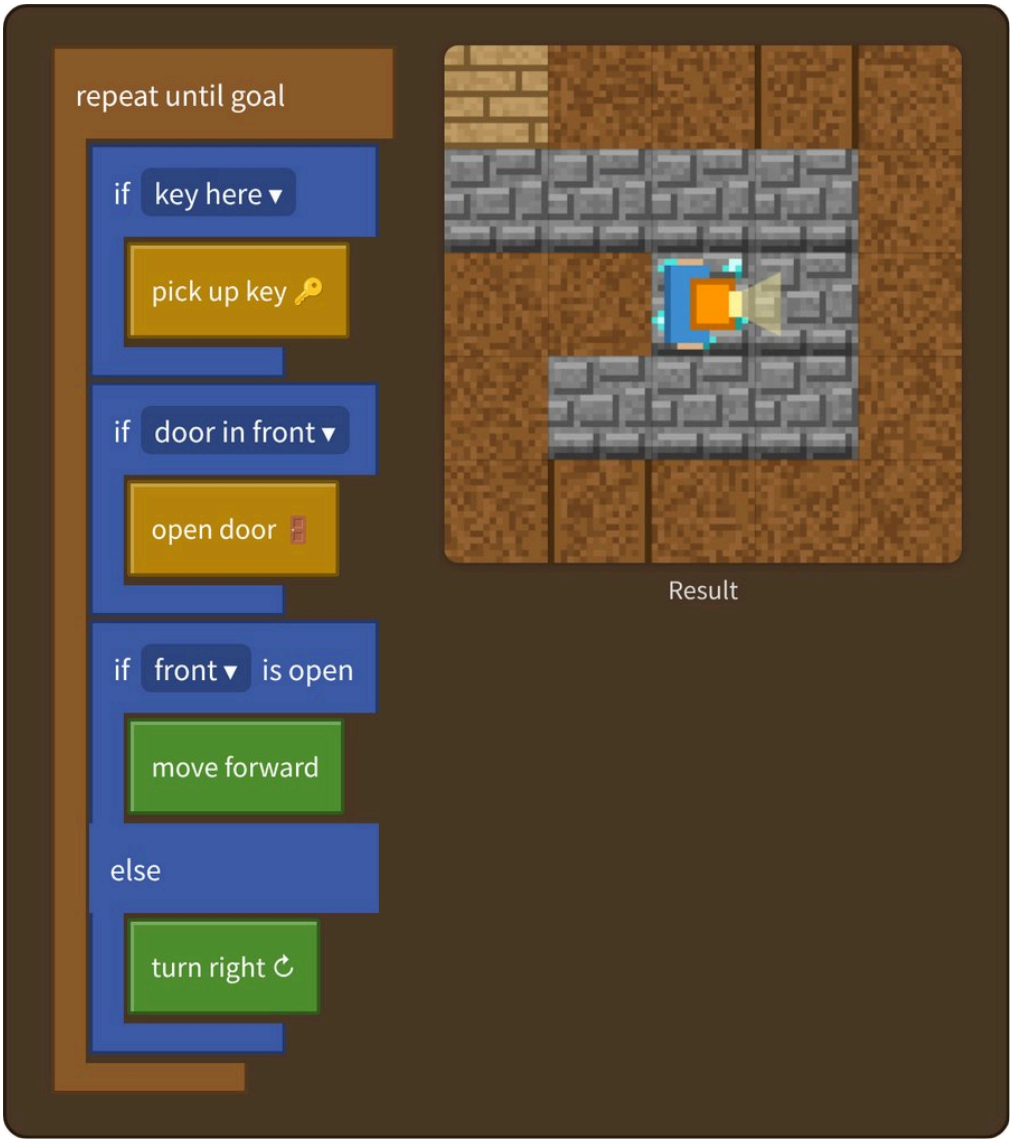
💡 **Hint (look if you get stuck)**

Inside the loop: if key here → pick up key; if door in front → open door; if front is open → forward, else → turn right.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: A thinking explorer



> ✨ **Key point**
>
> Pick up a key when you find one, open a door when there is one, and then go forward or turn right. Check all three every time.

> ⚠ **Common mistake**
>
> If “if front is open” comes before “if door in front”, you turn around at the door.

🎬 Video: `videos/en/mazekey-04.mp4`

Column

Write down what you carry

Picking up a key adds 1 to what you carry; opening a door takes 1 away. It is easy to forget if you only think in your head, so write it on the worksheet: “forward → 1 key → open → 0 keys”. Computers also remember this kind of “state” in **variables**.

Chapter 5

👾 The Dodge-the-Slime Maze

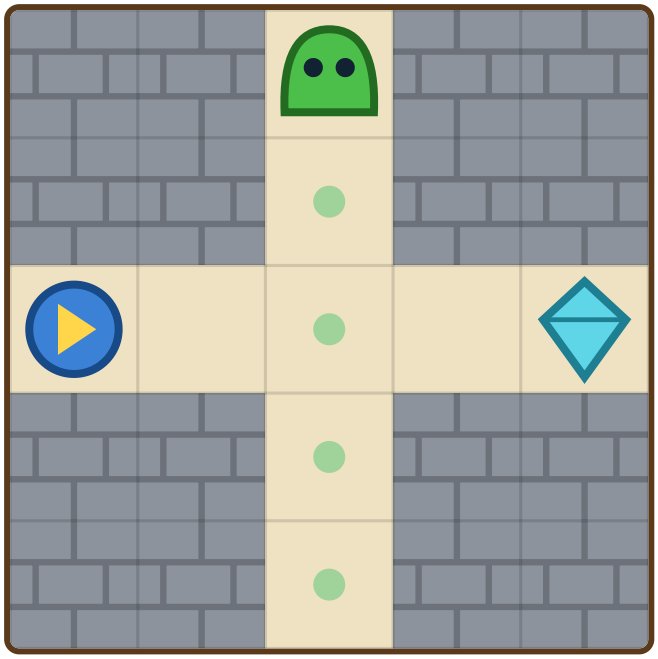
Slimes go back and forth on fixed tracks. Every time the explorer acts (move, turn or wait), each slime moves one square. Use “wait” to get the **timing** right.

Commands in this chapter

move forward / turn right, left / wait / repeat until goal / if danger ahead / if front is open

Stage 1: Wait and dodge

Sequence Timing



Problem

The slime goes up and down. Use "wait" to time your move so you do not bump into it.

🎯 **Goal**：Wait to time your move

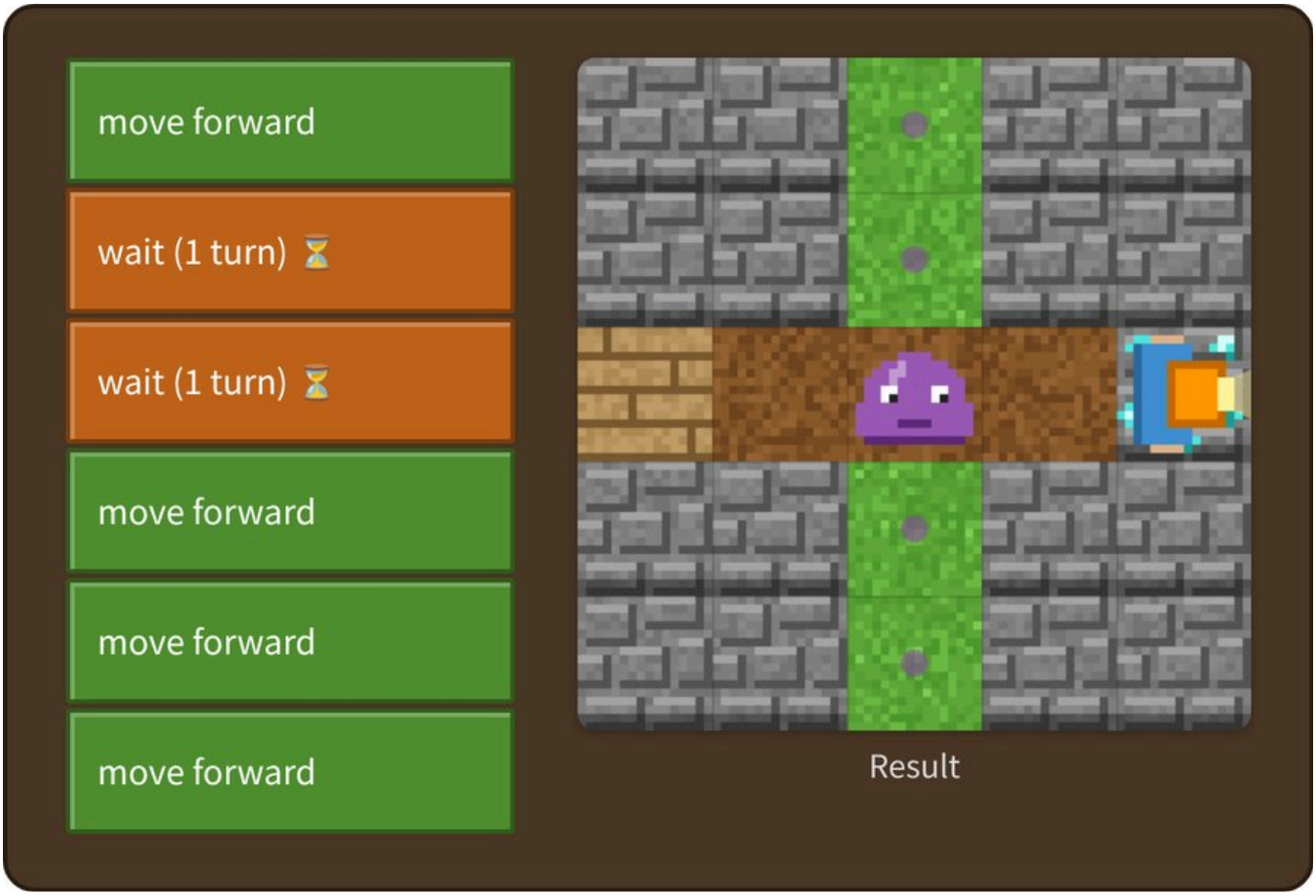
💡 **Hint (look if you get stuck)**

Move 1 square, then wait 2 times until the slime passes. Then move 3 squares.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Wait and dodge



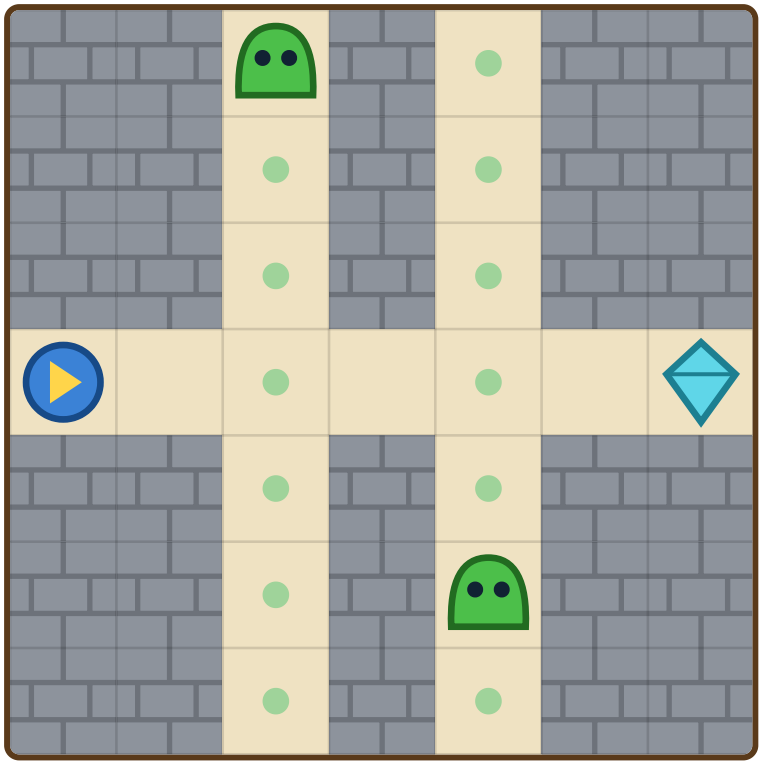
> ✨ **Key point**
>
> The slime moves one square each turn. Wait 2 times until it passes.

> ⚠ **Common mistake**
>
> With only one “wait”, you walk right into the slime.

🎬 Video: `videos/en/mazeenemy-01.mp4`

Stage 2: Wait if it is dangerous

Conditions | Repeat until | Timing



Problem

Use "if danger ahead". It is dangerous when a slime is in front of you, or will move in front of you next.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Wait when it is dangerous

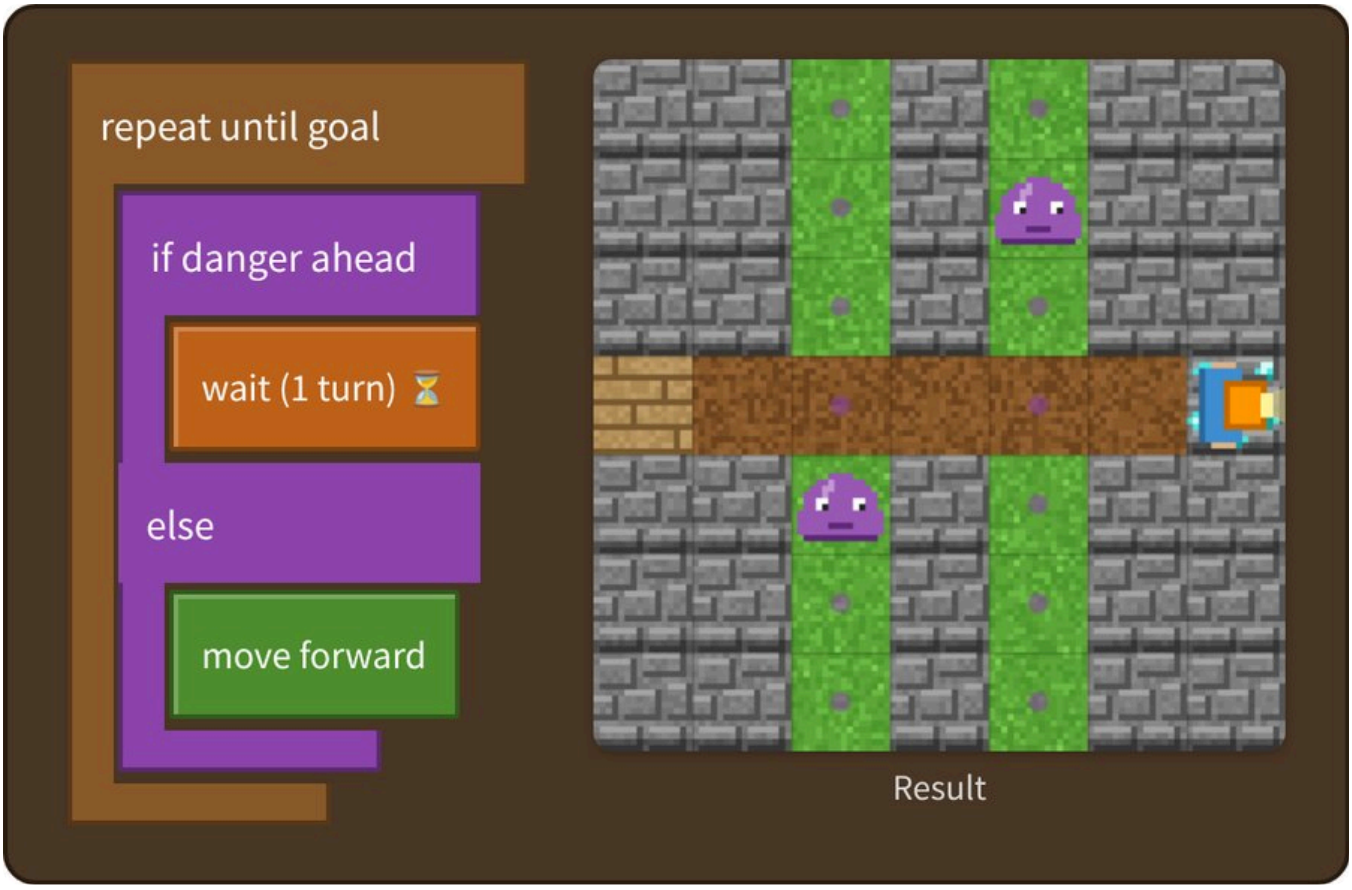
💡 **Hint (look if you get stuck)**

Repeat until goal: if danger ahead → wait, else → move forward.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Wait if it is dangerous



✨ **Key point**

“Danger” means a slime is in front or will come there next. If so, wait; if not, move.

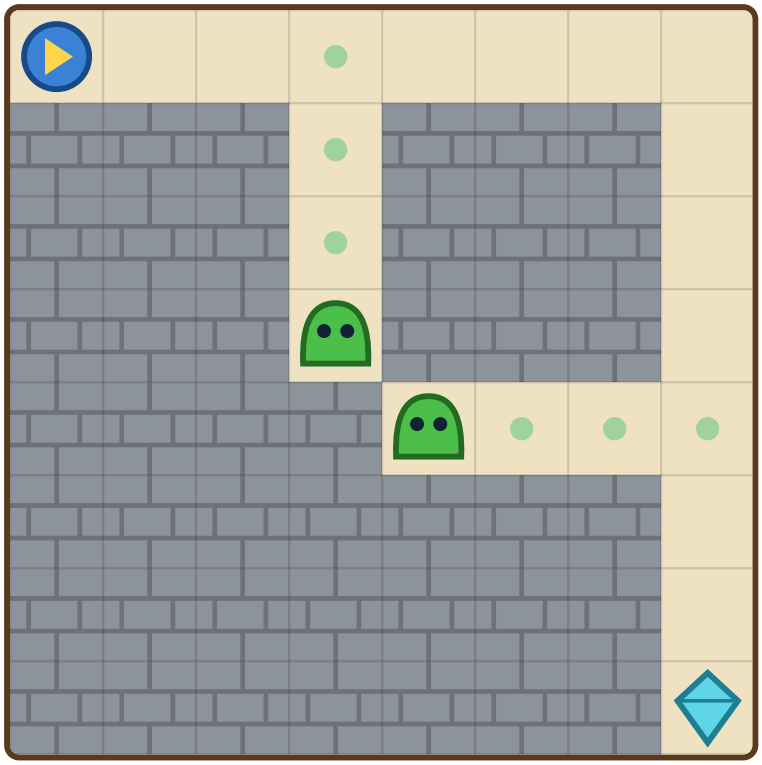
⚠ **Common mistake**

If the “if” is outside the loop, it checks only once.

🎬 Video: `videos/en/mazeenemy-02.mp4`

Stage 3: A bigger maze

Conditions | Nesting | Repeat until | Timing



Problem

The maze is bigger. If it is dangerous, wait. If not: move forward when the front is open, turn right when it is blocked.

🎯 Target: 6 blocks or fewer

🎯 **Goal**：An if inside an if

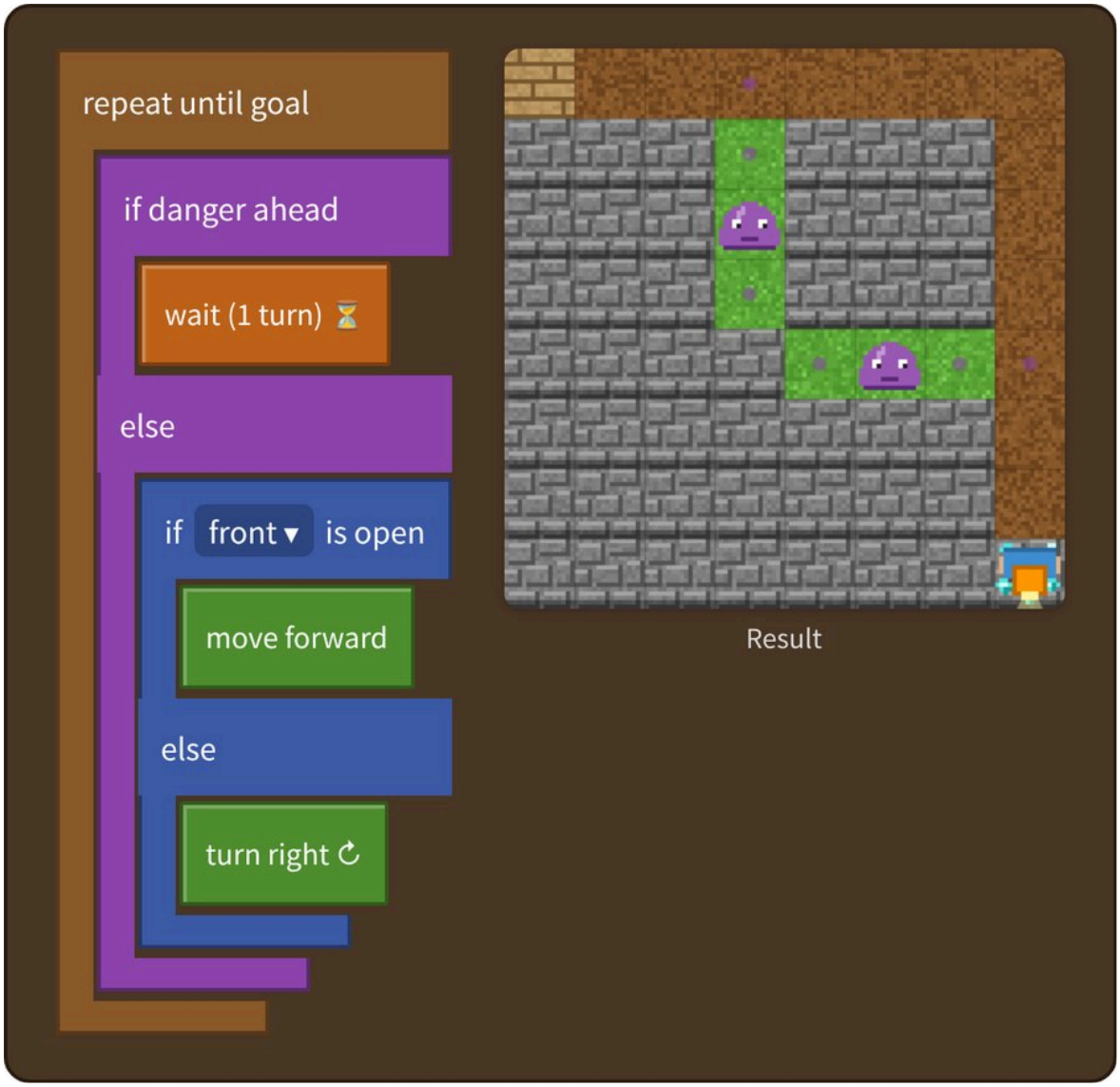
💡 **Hint (look if you get stuck)**

Repeat until goal: if danger ahead → wait, else → (if front is open → forward, else → turn right).

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: A bigger maze



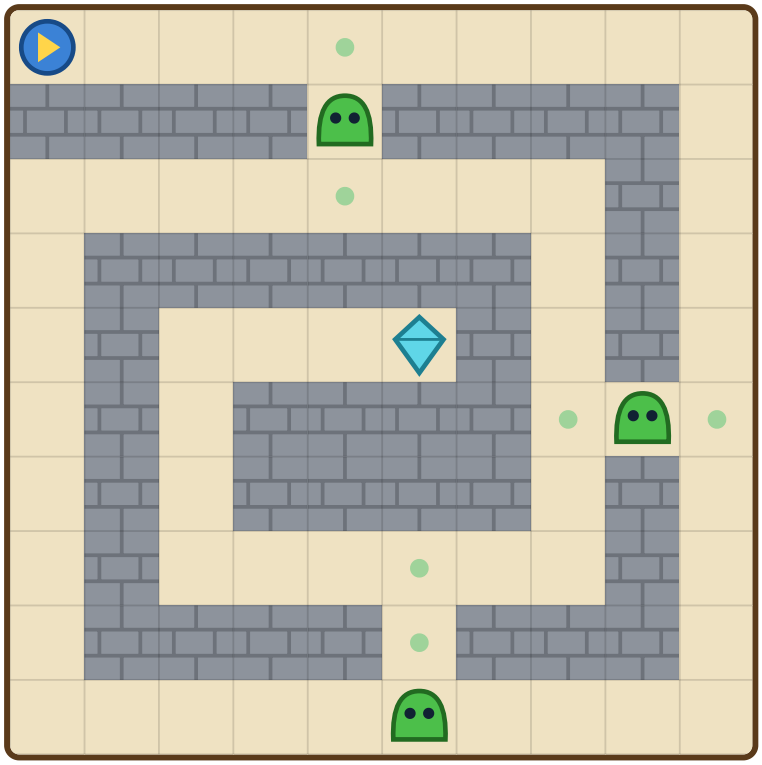
> ✨ **Key point**
>
> First check for danger. If it is safe, check if the front is open. The order matters.

> ⚠ **Common mistake**
>
> If you move without checking for danger, a slime catches you on the way.

🎬 Video: `videos/en/mazeenemy-03.mp4`

Stage 4: The spiral maze

Generalization | Repeat until | Timing



Problem

Last one: a 10×10 spiral with three slimes. Can the program from Stage 3 reach the goal?

🎯 Target: 6 blocks or fewer

🎯 **Goal**：The same program, a bigger maze

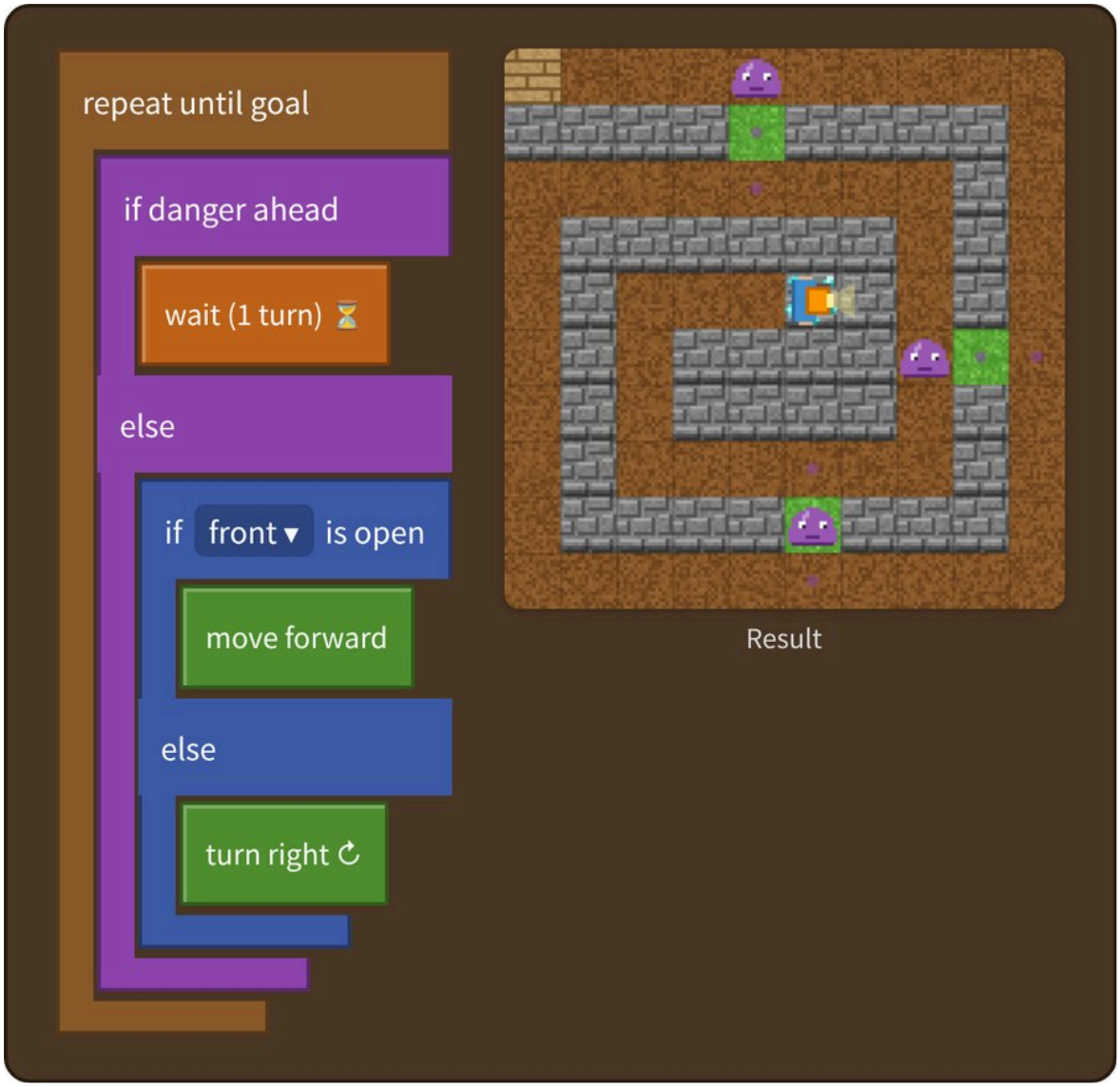
💡 **Hint (look if you get stuck)**

Use the same program as Stage 3. The slimes cross the path, so check for danger every time.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: The spiral maze



✨ Key point

The Stage 3 program works as it is. That is a method that works anywhere (generalization).

⚠ Common mistake

Without “wait”, one of the three slimes catches you.

🎬 Video: `videos/en/mazeenemy-04.mp4`

Column

Think about time, too

In the earlier mazes you only thought about “where to go”. With slimes, “**when** to go” matters too. Write the slime’s square for each turn in the worksheet table, and you will see the turn where you would bump into it.

Chapter 6

🐍 Mazes in Python

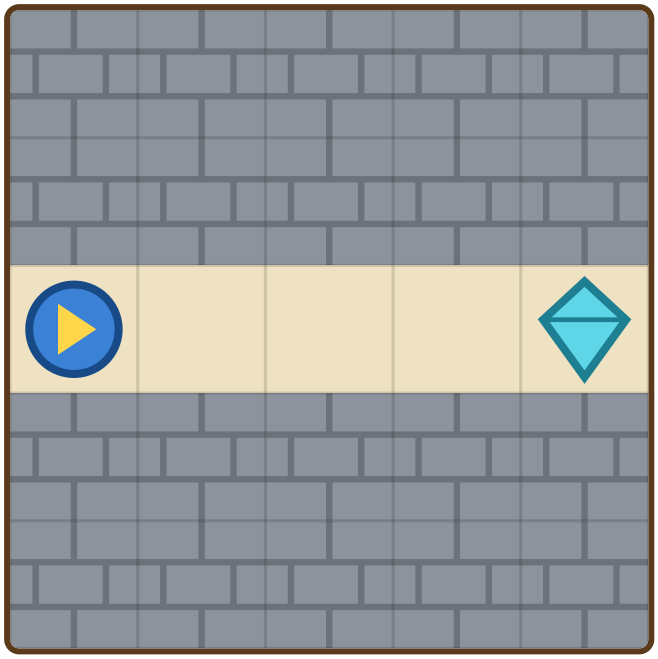
Programs you build with blocks can also be written in **Python**. “move forward” is `forward()`, “repeat 4 times” is `for i in range(4):`. Python is a real language used by programmers all over the world.

Commands in this chapter

forward() / turn_right() / turn_left() / for i in range(4): / while not at_goal(): / if is_open('front'): … else:

Stage 1: Go straight

Sequence | Python syntax (colon & indent)



Same maze as Block Maze stage 1

Problem

Type forward() to move the explorer to the diamond 💎. One line moves one square.

🎯 **Goal**：Write Python commands one line at a time

💡 **Hint (look if you get stuck)**

Write forward() 4 times, one per line.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go straight

```
forward()
forward()
forward()
forward()
```

✨ Key point

Python runs from the top line down. The “move forward” block = forward()

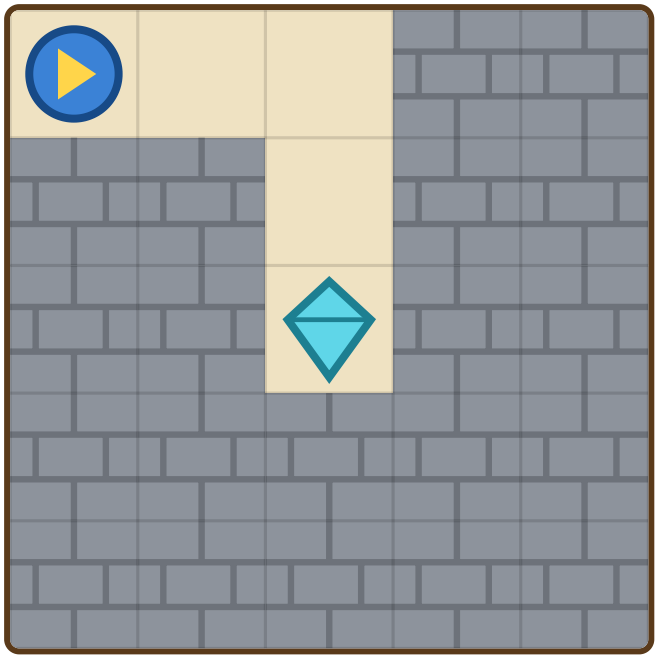
⚠ Common mistake

Writing forward without “()” causes an error.

🎬 Video: `videos/en/pymaze-01.mp4`

Stage 2: Let's turn

Sequence | Direction | Python syntax (colon & indent)



Same maze as Block Maze stage 2

Problem

turn_right() turns right, turn_left() turns left. Turning does not move you.

🎯 **Goal**：Use commands that change direction

💡 **Hint (look if you get stuck)**

forward() ×2 → turn_right() → forward() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Let's turn

```
forward()
forward()
turn_right()
forward()
forward()
```

✨ Key point

Turning does not move you. Don’t forget the “_” (underscore).

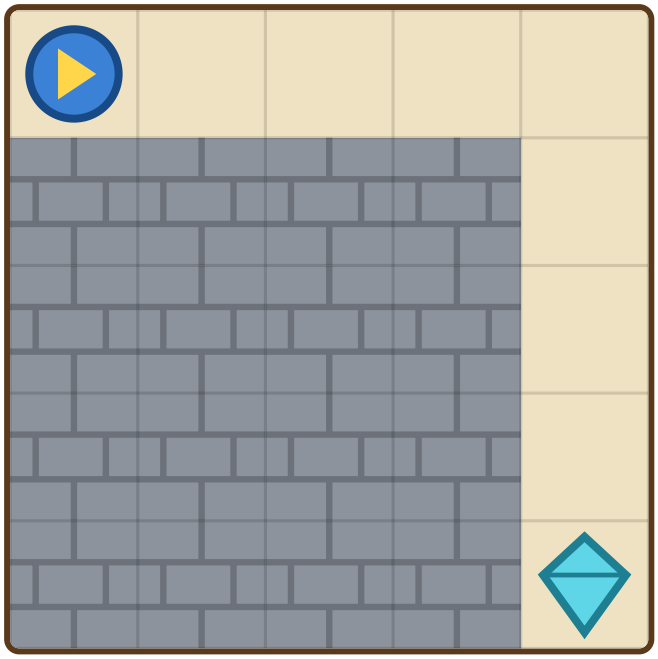
⚠ Common mistake

“turn right” (space) or “Turn_right” (capital) are different names.

🎬 Video: `videos/en/pymaze-02.mp4`

Stage 3: Repeat with for

Loops | Python syntax (colon & indent)



Same maze as Block Maze stage 4

Problem

“for i in range(4):” repeats the lines under it 4 times. Indent those lines with 4 spaces!

🎯 **Goal**：Write a loop with for and indentation

💡 **Hint (look if you get stuck)**

for i in range(4): → forward() (indented) → turn_right() → the same loop again

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Repeat with for

```
for i in range(4):
    forward()
turn_right()
for i in range(4):
    forward()
```

✨ Key point

Lines indented after “:” are inside the loop. Un-indent to leave the loop.

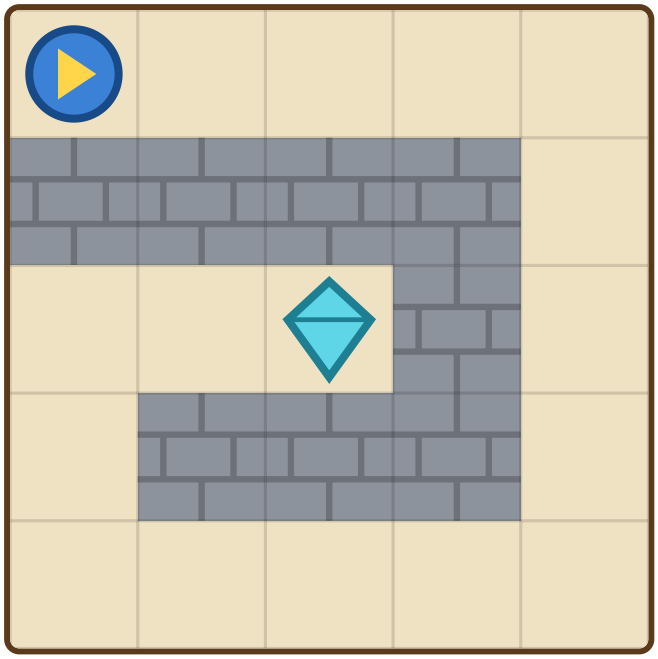
⚠ Common mistake

Forgetting the final “:”. Indenting turn_right() too makes it turn 4 times.

🎬 Video: `videos/en/pymaze-03.mp4`

Stage 4: while and if

Conditions | Repeat until | Python syntax (colon & indent)



Same maze as Block Maze stage 6

Problem

Repeat with “while not at_goal():” and decide with “if is_open('front'):”. else runs when the front is blocked.

🎯 **Goal**：Combine while with if / else

💡 **Hint (look if you get stuck)**

while not at_goal(): → if is_open('front'): forward() → else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: while and if

```
while not at_goal():
    if is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

not means “is not”. while not at_goal() = “repeat while you are not at the goal”.

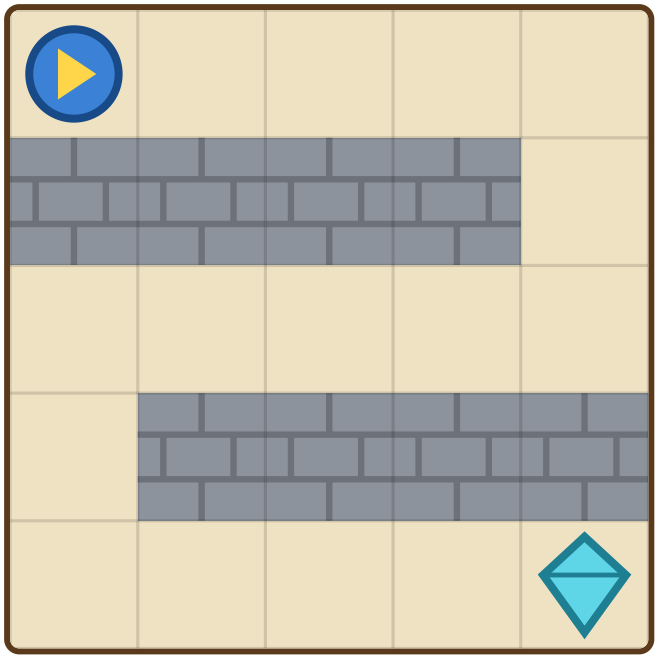
⚠ Common mistake

Misaligned if/else causes an error. Write else: at the same depth as if.

🎬 Video: `videos/en/pymaze-04.mp4`

Stage 5: Three choices with elif

Conditions | Repeat until | Nesting | Generalization | Python syntax (colon & indent)



Same maze as Block Maze stage 7

Problem

Python has elif (“else if”). Check the front, then the right, and otherwise turn left.

🎯 **Goal**：Split into three cases with elif

💡 **Hint (look if you get stuck)**

if is_open('front'): ... elif is_open('right'): ... else: ...

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Three choices with elif

```
while not at_goal():
    if is_open('front'):
        forward()
    elif is_open('right'):
        turn_right()
    else:
        turn_left()
```

✨ Key point

elif means “else if”. Putting an “if” block inside “else” becomes elif in Python.

⚠ Common mistake

Writing “else if” is an error in Python. Changing the order takes a different path.

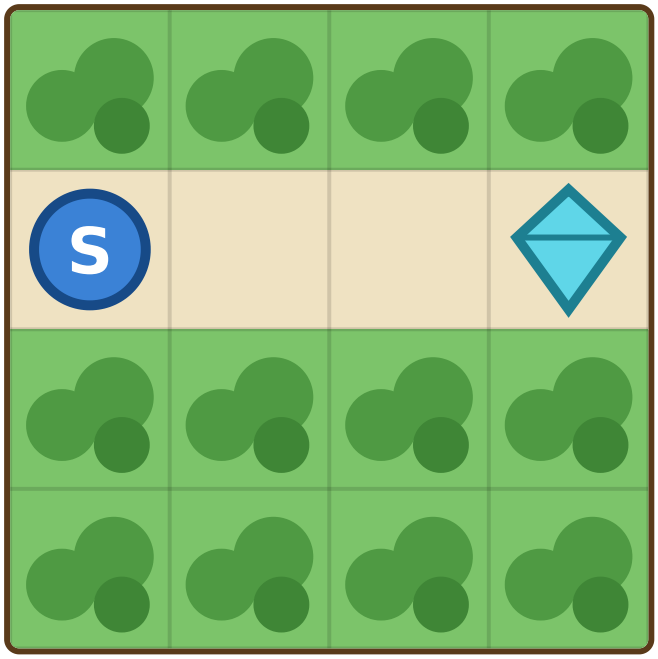
🎬 Video: `videos/en/pymaze-05.mp4`

🐍 Python Arrow Maze

The Arrow Maze in Python. Use the four commands up(), down(), left(), right() and for.

Stage 1: Go with right()

Sequence | Python syntax (colon & indent)



Same maze as Arrow Maze stage 1

Problem

right() moves one square to the right. Write it 3 times, one per line, to reach the diamond 💎.

🎯 **Goal**：Write right()

💡 **Hint (look if you get stuck)**

right() on 3 lines.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Go with right()

```
right()
right()
right()
```

✨ Key point

Python has one command per line. Three lines of right() move 3 squares.

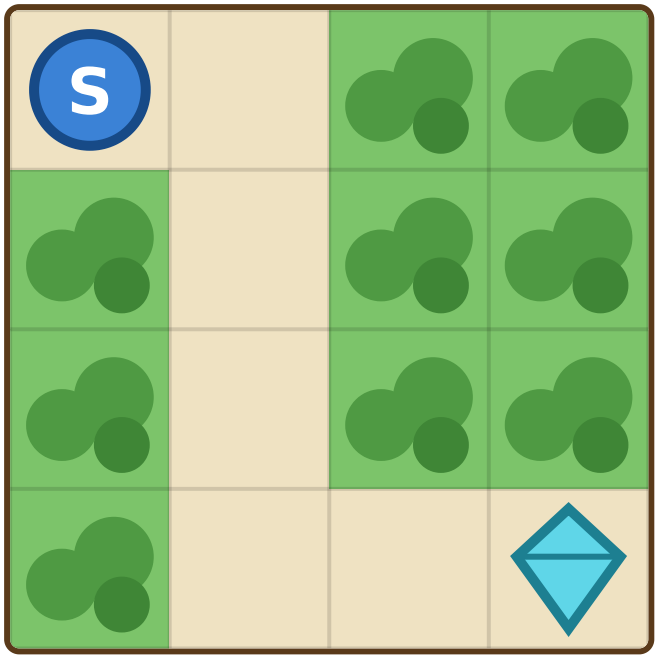
⚠ Common mistake

Writing just right without “()” causes a SyntaxError.

🎬 Video: `videos/en/pymazejr-01.mp4`

Stage 2: Go down with down()

Sequence | Direction | Python syntax (colon & indent)



Same maze as Arrow Maze stage 2

Problem

down() moves one square down.

🎯 **Goal**：Use down() too

💡 **Hint (look if you get stuck)**

right() → down() ×3 → right() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Go down with down()

```
right()
down()
down()
down()
right()
right()
```

✨ Key point

Write right() and down() in the order you go.

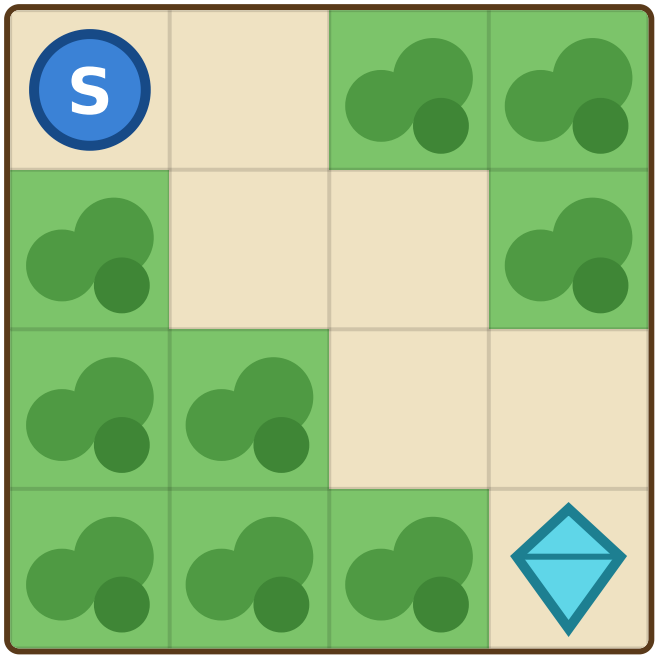
⚠ Common mistake

Down() with a capital letter causes a NameError. Python cares about capital letters.

🎬 Video: `videos/en/pymazejr-02.mp4`

Stage 3: Stairs

Sequence | Patterns | Python syntax (colon & indent)



Same maze as Arrow Maze stage 3

Problem

right() and down(), one after the other.

🎯 **Goal**：Write them alternately

💡 **Hint (look if you get stuck)**

right() down() right() down() right() down()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Stairs

```
right()
down()
right()
down()
right()
down()
```

✨ Key point

right() and down(), 3 times each, one after the other.

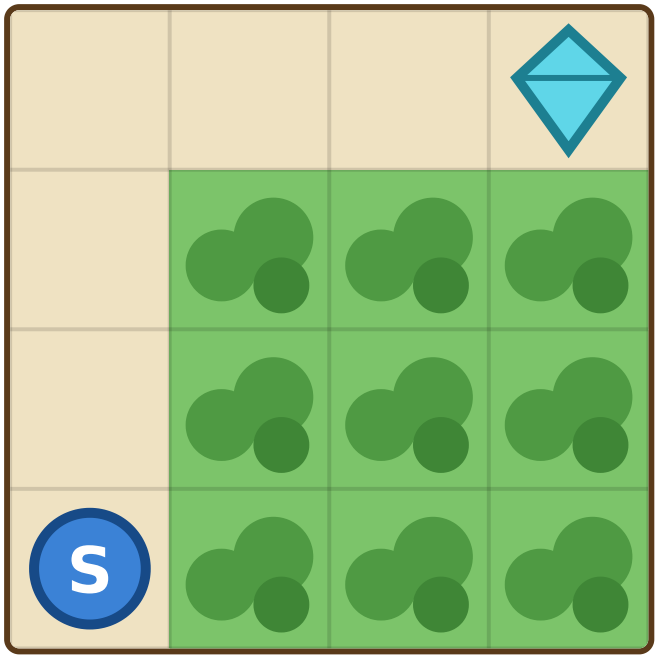
⚠ Common mistake

If the order is swapped, you bump into the bushes.

🎬 Video: `videos/en/pymazejr-03.mp4`

Stage 4: Go up with up()

Sequence | Direction | Python syntax (colon & indent)



Same maze as Arrow Maze stage 4

Problem

up() moves one square up.

🎯 **Goal**：Go up with up()

💡 **Hint (look if you get stuck)**

up() ×3 → right() ×3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Go up with up()

```
up()
up()
up()
right()
right()
right()
```

✨ **Key point**

Three lines of up(), then three lines of right().

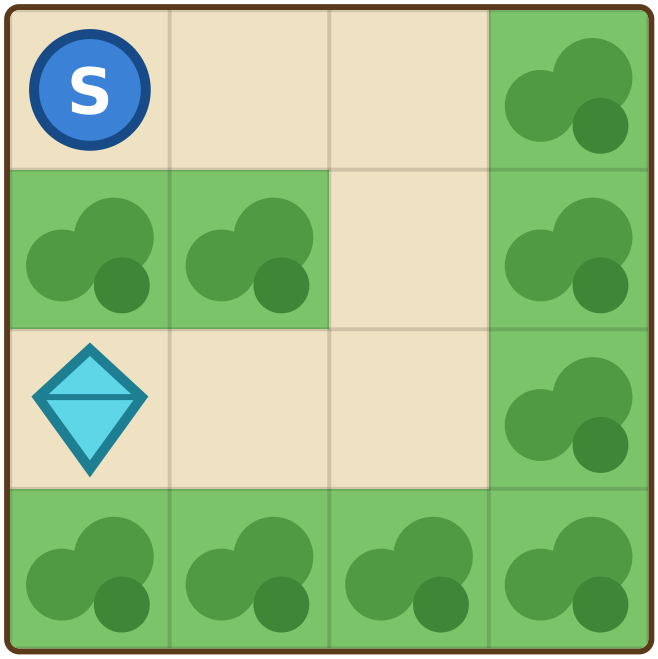
⚠ **Common mistake**

A space at the start of a line causes an IndentationError. Only lines inside for are indented.

🎬 Video: `videos/en/pymazejr-04.mp4`

Stage 5: Use left() too

Sequence | Direction | Python syntax (colon & indent)



Same maze as Arrow Maze stage 5

Problem

left() moves one square to the left. Use all four.

🎯 **Goal**：Use left() too

💡 **Hint (look if you get stuck)**

right() ×2 → down() ×2 → left() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: Use left() too

```
right()
right()
down()
down()
left()
left()
```

✨ Key point

right() → down() → left(). Now you have used all four commands.

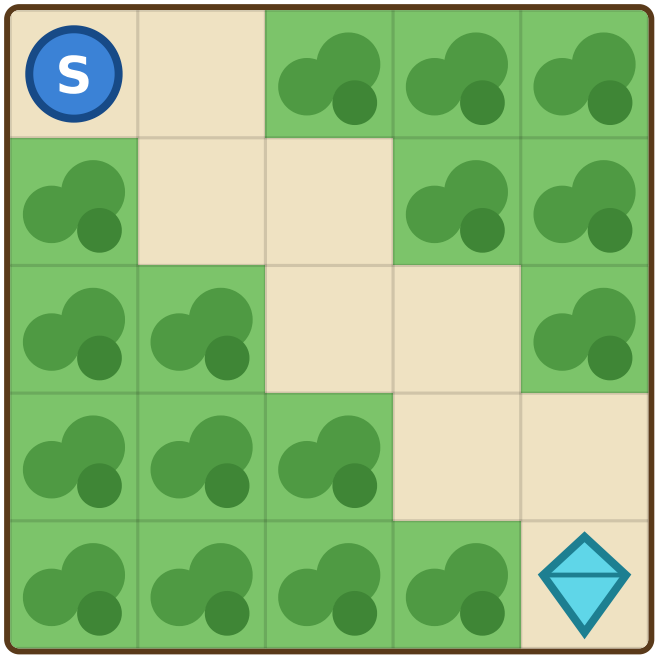
⚠ Common mistake

left and right are easy to mix up. Think of the left and right of the screen.

🎬 Video: `videos/en/pymazejr-05.mp4`

Stage 6: Repeat with for

Loops | Python syntax (colon & indent)



Same maze as Arrow Maze stage 6

Problem

“for i in range(4):” repeats the lines under it 4 times. Indent those lines with 4 spaces.

🎯 **Goal**：Repeat with for

💡 **Hint (look if you get stuck)**

for i in range(4): → (indented) right() and down()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Repeat with for

```
for i in range(4):
    right()
    down()
```

✨ Key point

Indent the 2 lines under for i in range(4): by 4 spaces. Both lines repeat.

⚠ Common mistake

If down() is not indented, only right() repeats.

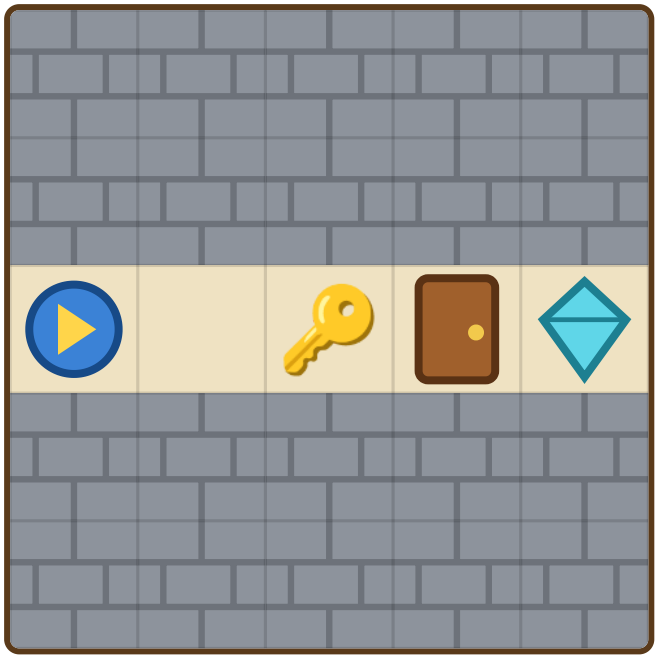
🎬 Video: `videos/en/pymazejr-06.mp4`

🐍 Python Keys & Doors

Keys & Doors in Python. Use pick_up(), open_door() and if with key_here() and door_ahead().

Stage 1: pick_up() and open_door()

Sequence | State | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 1

Problem

On the key 🔑, pick_up(). In front of the door 🚪, open_door(). Then you can go through.

🎯 **Goal**：pick_up() and open_door()

💡 **Hint (look if you get stuck)**

forward() ×2 → pick_up() → open_door() → forward() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: pick_up() and open_door()

```
forward()
forward()
pick_up()
open_door()
forward()
forward()
```

✨ Key point

pick_up() makes 1 key and open_door() makes 0. Your item count changes.

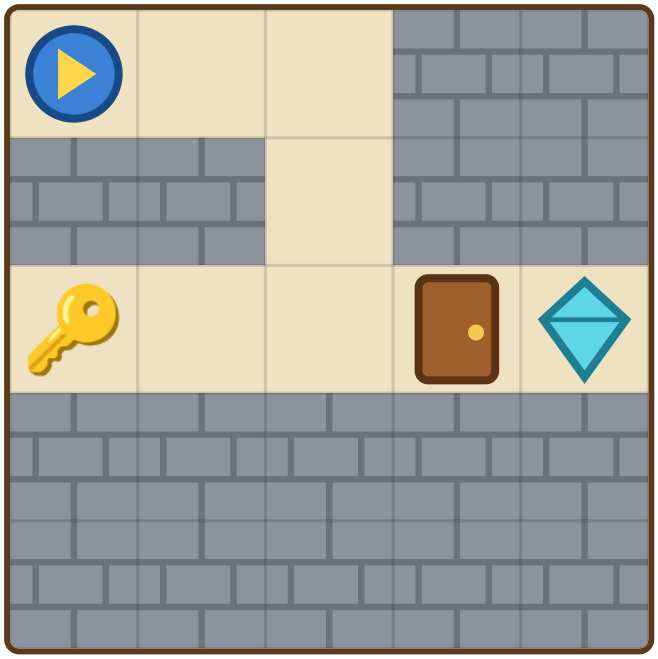
⚠ Common mistake

pickup() or open door() causes an error. Don’t forget the “_”.

🎬 Video: `videos/en/pymazekey-01.mp4`

Stage 2: Where is the key?

Sequence | State | Decomposition | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 2

Problem

Get the key first. Without a key, open_door() does not work.

🎯 **Goal**：Write the side trip in order

💡 **Hint (look if you get stuck)**

Walk to the key, pick_up(), turn_right() twice and go back.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 2: Where is the key?

```
forward()
forward()
turn_right()
forward()
forward()
turn_right()
forward()
forward()
pick_up()
turn_right()
turn_right()
forward()
forward()
open_door()
forward()
forward()
```

✨ Key point

Go to the key → pick_up() → turn_right() twice → to the door. Write a long plan one line at a time.

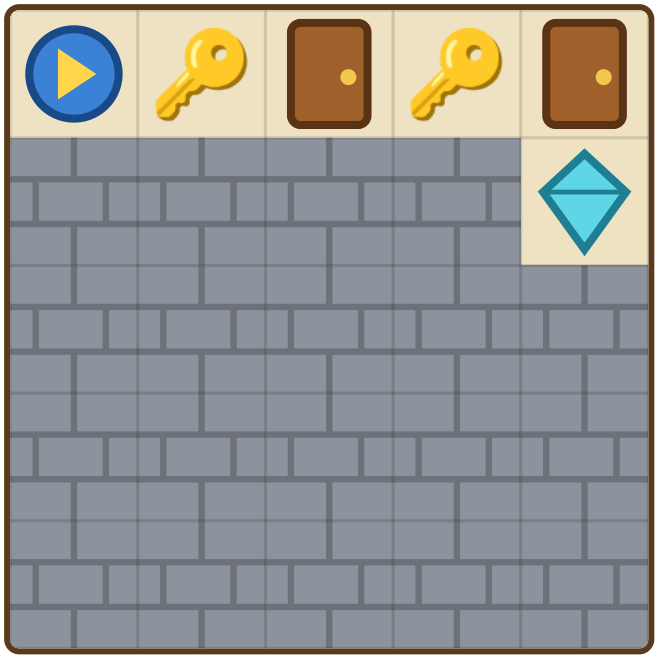
⚠ Common mistake

open_door() before pick_up() gives “You have no key.”

🎬 Video: `videos/en/pymazekey-02.mp4`

Stage 3: Two doors with for

Loops | Patterns | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 3

Problem

Repeat “forward() → pick_up() → open_door() → forward()” with for.

🎯 **Goal**：Four lines inside for

💡 **Hint (look if you get stuck)**

for i in range(2): (4 lines, indented) → turn_right() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 3: Two doors with for

```
for i in range(2):
    forward()
    pick_up()
    open_door()
    forward()
turn_right()
forward()
```

✨ **Key point**

All 4 lines inside for are indented by 4 spaces. The last 2 lines are not.

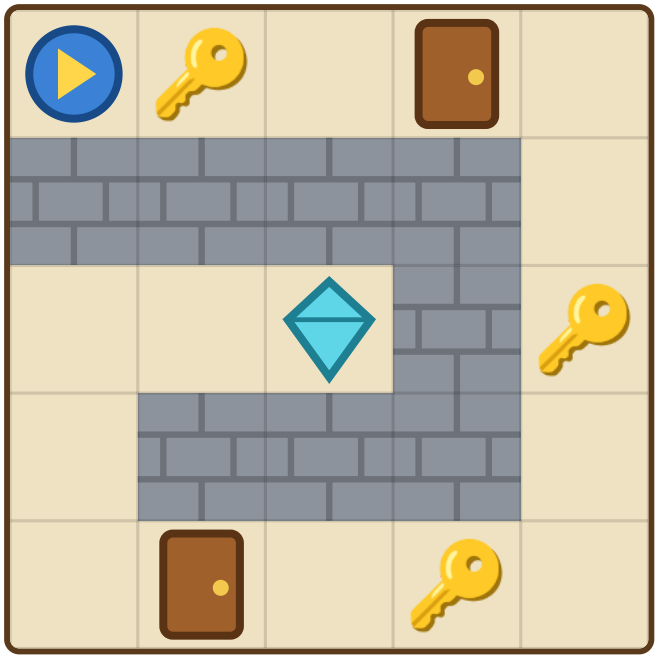
⚠ **Common mistake**

If turn_right() is indented too, you turn twice and go back.

🎬 Video: `videos/en/pymazekey-03.mp4`

Stage 4: key_here() and door_ahead()

Conditions | Repeat until | State | Generalization | Python syntax (colon & indent)



Same maze as Keys & Doors Maze stage 4

Problem

key_here() checks for a key where you stand. door_ahead() checks for a door in front. Use them inside “while not at_goal():”.

🎯 **Goal**：Check with key_here() and door_ahead()

💡 **Hint (look if you get stuck)**

while not at_goal(): if key_here(): pick_up() / if door_ahead(): open_door() / if is_open('front'): forward() else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: key_here() and door_ahead()

```
while not at_goal():
    if key_here():
        pick_up()
    if door_ahead():
        open_door()
    if is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

Two ifs in a row, then if … else. Inside while, everything is checked again each time.

⚠ Common mistake

Forgetting “()” after key_here causes a SyntaxError. Checks also need ().

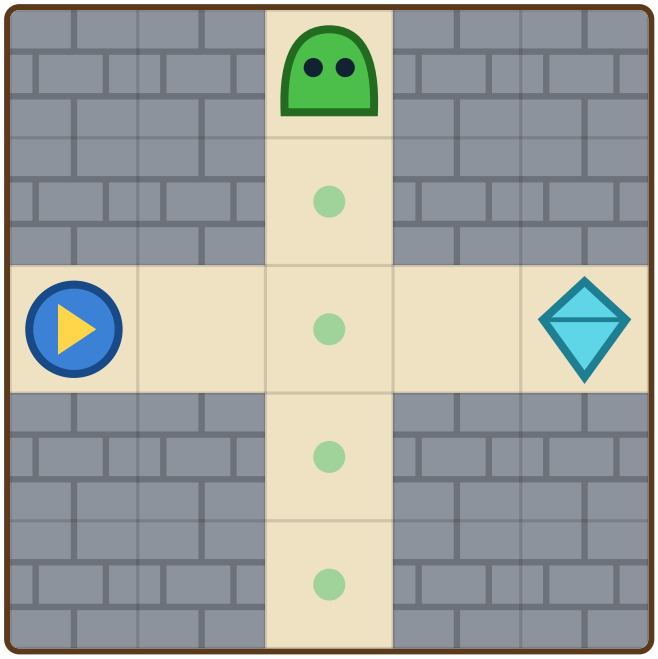
🎬 Video: `videos/en/pymazekey-04.mp4`

🐍 Python Slime Maze

Dodge the Slime Maze in Python. Use wait(), danger_ahead() and if, elif, else.

Stage 1: wait()

Sequence | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 1

Problem

wait() makes the explorer wait for 1 turn. The slime moves 1 square every turn. Time your move!

🎯 **Goal**：Wait with wait()

💡 **Hint (look if you get stuck)**

forward() → wait() ×2 → forward() ×3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: wait()

```
forward()
wait()
wait()
forward()
forward()
forward()
```

✨ Key point

wait() waits one turn. The slime moves one square meanwhile.

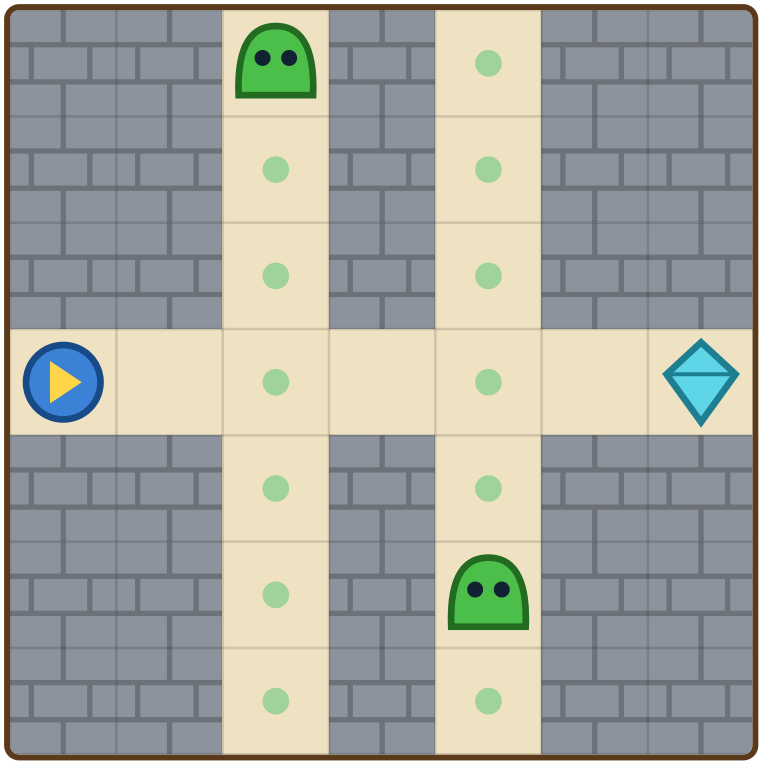
⚠ Common mistake

wait(2) causes a TypeError. To wait twice, write wait() on 2 lines.

🎬 Video: `videos/en/pymazeenemy-01.mp4`

Stage 2: if danger_ahead():

Conditions | Repeat until | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 2

Problem

danger_ahead() is True when a slime is in front of you or will come there next. If so, wait(); else, forward().

🎯 **Goal**：Check with if danger_ahead():

💡 **Hint (look if you get stuck)**

while not at_goal(): if danger_ahead(): wait() else: forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: if danger_ahead():

```
while not at_goal():
    if danger_ahead():
        wait()
    else:
        forward()
```

✨ **Key point**

danger_ahead() is True when it is dangerous. if ... else decides between waiting and moving.

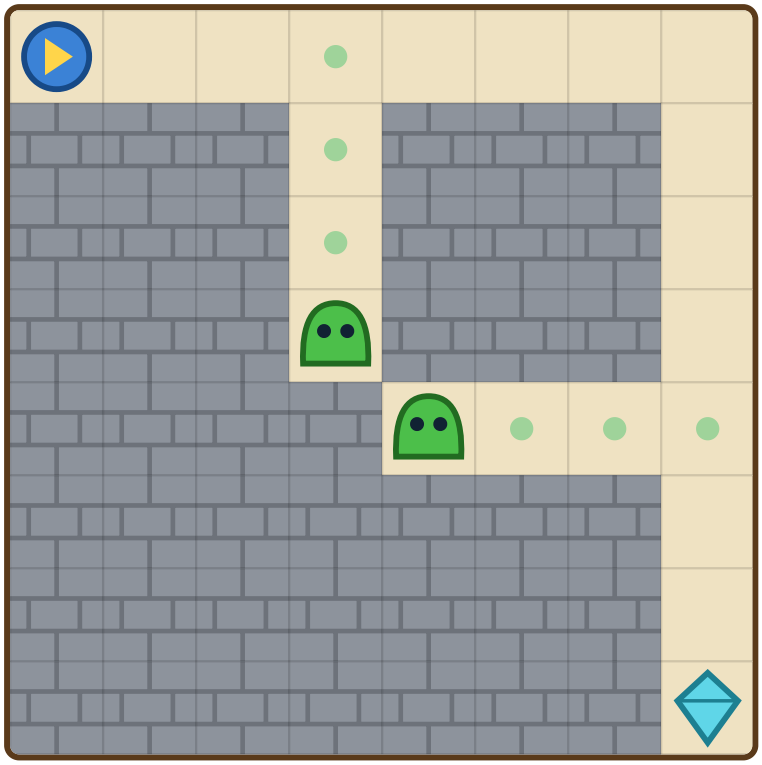
⚠ **Common mistake**

Forgetting the “:” after else causes a SyntaxError.

🎬 Video: `videos/en/pymazeenemy-02.mp4`

Stage 3: A bigger maze with elif

Conditions | Nesting | Repeat until | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 3

Problem

If danger, wait(). elif the front is open, forward(). else turn_right().

🎯 **Goal**：Split into three with elif

💡 **Hint (look if you get stuck)**

while not at_goal(): if danger_ahead(): wait() / elif is_open('front'): forward() / else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: A bigger maze with elif

```
while not at_goal():
    if danger_ahead():
        wait()
    elif is_open('front'):
        forward()
    else:
        turn_right()
```

✨ **Key point**

if (danger) → elif (front is open) → else (turn right). They are checked from the top.

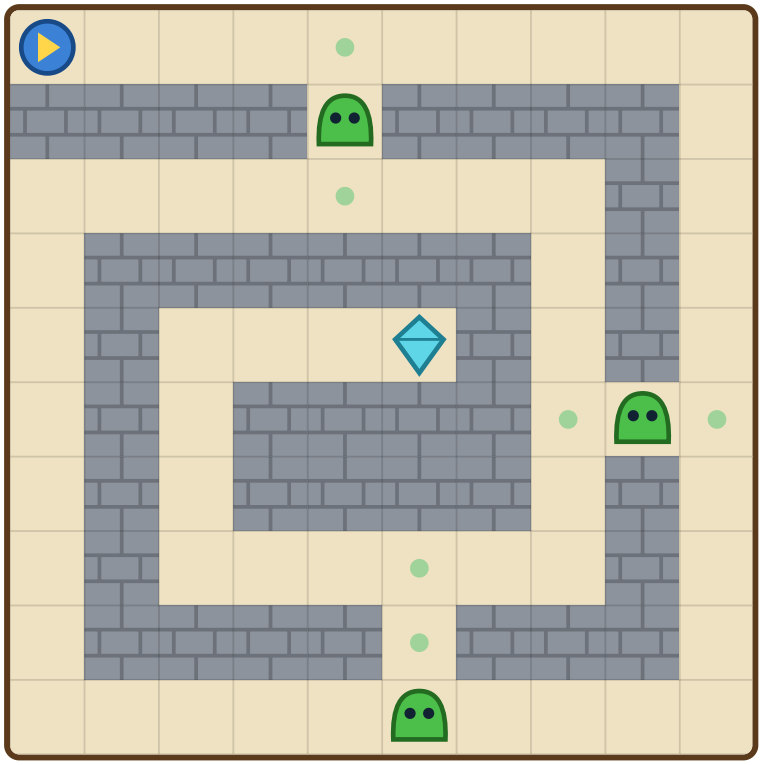
⚠ **Common mistake**

If you change elif to if, you may move right after waiting and get caught.

🎬 Video: `videos/en/pymazeenemy-03.mp4`

Stage 4: The spiral maze

Generalization | Repeat until | Timing | Python syntax (colon & indent)



Same maze as Dodge the Slime Maze stage 4

Problem

A 10×10 spiral with three slimes. Does your program from Stage 3 work here too?

🎯 **Goal**：The same code, a bigger maze

💡 **Hint (look if you get stuck)**

The Stage 3 program works as it is.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: The spiral maze

```
while not at_goal():
    if danger_ahead():
        wait()
    elif is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

Use the Stage 3 code as it is. It works in any maze.

⚠ Common mistake

Forgetting the quotes in is_open('front') causes an error.

🎬 Video: `videos/en/pymazeenemy-04.mp4`

Column

Three Python rules

① Every command ends with `()`. ② Lines starting with `for`, `while` or `if` end with `:` (a colon). ③ The lines inside are indented by **4 spaces**. If you get an error, check these three first. Capital and small letters are different, too.

Well done!

You worked hard to the end! Let’s look back at the ideas you used in this book.

Idea	Where did it appear?
Order	Commands run from top to bottom (every maze)
Loops	Pack the same moves together (Block Maze stages 4–5)
If (conditions)	Look ahead and decide (Block Maze stages 6–8, keys, slimes)
State	How many keys you carry (Keys & Doors)
Timing	Dodge with “wait” (Slime maze)
Algorithm	Steps that solve any maze (Block Maze stage 8)
Python	Write the same thing in words (code)

When you clear every stage, get your certificate from “⚙ Settings” → “📜 Certificate” in the app. Next, try “Block Drawing”, “Space Star Hunt” and “Break & Build Quest”!

PART 2

Part 2 Guide for teachers and parents

About this guide

Part 1 is written in simple language so that students can read it on their own. Part 2 is for teachers who plan and run lessons, and for parents who support learning at home.

This book collects the eight maze apps (4 block, 4 Python; 41 stages in total) of a free, browser-based block programming curriculum published on GitHub Pages. It is designed for elementary programming education focused on logical thinking (“computational thinking”), and as an introduction to programming in junior high school technology classes.

Aims

- Think of the steps to reach a goal as a combination of sequence, loops and conditions.
- Find and fix the cause of a problem from the result of running the program (trial and error, debugging).
- Express the same task in both blocks and Python (a text language) and notice how they correspond.

Overview of the stages

App	#	Stage	Ideas
🧭 Arrow Maze	1	Go right	Sequence
	2	Go down	Sequence, Direction
	3	Stairs	Sequence, Patterns
	4	Go up	Sequence, Direction
	5	Go around	Sequence, Direction
	6	Use repeat	Loops, Patterns
⛏️ Block Maze	1	Go straight	Sequence
	2	Let's turn	Sequence, Direction
	3	A long path	Sequence, Direction
	4	Use a loop	Loops
	5	Climb the stairs	Loops, Patterns
	6	If...	Conditions, Repeat until
	7	Right? Left?	Conditions, Nesting
	8	Any maze!	Generalization, Conditions, Repeat until
🗝️ Keys & Doors Maze	1	Pick up the key, open the door	Sequence, State
	2	Where is the key?	Sequence, State, Decomposition
	3	Two doors	Loops, Patterns, State
	4	A thinking explorer	Conditions, Repeat until, State, Generalization
👾 Dodge the Slime Maze	1	Wait and dodge	Sequence, Timing
	2	Wait if it is dangerous	Conditions, Repeat until, Timing
	3	A bigger maze	Conditions, Nesting, Repeat until, Timing
	4	The spiral maze	Generalization, Repeat until, Timing
🐍 Python Maze	1	Go straight	Sequence, Python syntax (colon & indent)
	2	Let's turn	Sequence, Direction, Python syntax (colon & indent)
	3	Repeat with for	Loops, Python syntax (colon & indent)
	4	while and if	Conditions, Repeat until, Python syntax (colon & indent)
	5	Three choices with elif	Conditions, Repeat until, Nesting, Generalization, Python syntax (colon & indent)
🐍 Python Arrow Maze	1	Go with right()	Sequence, Python syntax (colon & indent)
	2	Go down with down()	Sequence, Direction, Python syntax (colon & indent)
	3	Stairs	Sequence, Patterns, Python syntax (colon & indent)
	4	Go up with up()	Sequence, Direction, Python syntax (colon & indent)
	5	Use left() too	Sequence, Direction, Python syntax (colon & indent)
	6	Repeat with for	Loops, Python syntax (colon & indent)
🐍 Python Keys & Doors	1	pick_up() and open_door()	Sequence, State, Python syntax (colon & indent)
	2	Where is the key?	Sequence, State, Decomposition, Python syntax (colon & indent)
	3	Two doors with for	Loops, Patterns, Python syntax (colon & indent)

	4	key_here() and door_ahead()	Conditions, Repeat until, State, Generalization, Python syntax (colon & indent)
🐍 Python Slime Maze	1	wait()	Sequence, Timing, Python syntax (colon & indent)
	2	if danger_ahead():	Conditions, Repeat until, Timing, Python syntax (colon & indent)
	3	A bigger maze with elif	Conditions, Nesting, Repeat until, Timing, Python syntax (colon & indent)
	4	The spiral maze	Generalization, Repeat until, Timing, Python syntax (colon & indent)

Suggested order and lesson count

Combine the apps according to grade and experience. Each stage takes from a few minutes to about 20 minutes.

Students	Apps	Lessons	Key ideas
Grades 1–2	Arrow Maze 1–6	1–2	Order, counting
Grades 2–3	Block Maze 1–5	2	Order, direction, loops
Grades 4–6	Block Maze 6–8, Keys & Doors, Slime	3–4	Conditions, state, timing, generalization
Grade 5 – junior high	Python mazes (4 apps)	3–5	Blocks ↔ code, syntax, reading errors

Students progress at different speeds. For fast finishers: try the block limit, act as a “hint giver” for friends, or solve the same maze in the Python version.

Example flow of a 45-minute lesson

Time	Activity	Teacher support
Intro 5 min	Review the last lesson; share today’s goal (Worksheet 1).	You can write the book’s “Goal” on the board as it is.
Plan 5 min	Look at the maze and predict the steps (Worksheets 3–7).	Have students trace the path with a finger before touching the device.
Try 20 min	Try in the app; find and fix the cause if it fails.	Don’t give the answer; ask “Where did it stop?” Allow pair talk.
Share 10 min	Show different solutions; think about shorter programs.	Compare 2–3 programs on the big screen.
Reflect 5 min	Write a reflection (Worksheet 1).	Separate “what I learned” and “what I want to try next”.

Teaching points for each app

🧭 Arrow Maze (lower grades)

- **Aim:** focus only on “order” by lining up arrows, without thinking about direction.
- **Common difficulty:** miscounting squares (counting the start square). → “Don’t count the square the explorer is on.” Have students touch and count one square at a time.
- **Question to ask:** “If you have one arrow too few, where will it stop?”

⛏️ Block Maze

- **Aim:** move step by step from sequence → loops → conditions → generalization (algorithm).
- **Common difficulties:** ① thinking “turn right” also moves forward (it only turns); ② confusing left and right (screen right vs. the explorer’s right); ③ in stage 8, making steps that work only for one maze.
- **Support:** for ②, rotate the book or tablet, or have students turn their bodies. For ③, have students say a rule that works for any maze before building it (the right-hand rule).

🗝️ Keys & Doors

- **Aim:** think while keeping track of state (“how many am I carrying now?”) — a basis for variables.
- **Common difficulties:** picking up when not on a key, walking to a door without a key, losing count of keys.
- **Support:** have students write the number of keys after each move in the Worksheet 6 table, and watch the item display on screen.

👾 Dodge the Slime

- **Aim:** think about “when” as well as “where”, then generalize with “if danger ahead”.
- **Common difficulties:** too few or too many “wait”s; missing that slimes also move when you only turn.
- **Support:** show the run one turn at a time and have students record the slime’s position in a table.

🐍 Python mazes

- **Aim:** notice how blocks and code correspond, and that the same thinking works in text.

- **Common difficulties:** missing `()` or `:` (SyntaxError), wrong indentation (IndentationError), capital letters or spelling (NameError).
- **Support:** present errors as hints, not failures, and have students log them on Worksheet 7. Building in block mode and comparing with the Python shown below also helps.

Assessment (example)

Area	Meets expectations (B)	Exceeds (A), e.g.
Knowledge & skills	Arranges blocks (code) correctly and clears the maze.	Uses loops and conditions to clear it with a short program.
Thinking, judgment & expression	Finds where the error is from the result and fixes it.	Explains a method that works for any maze and turns it into a program.
Attitude toward learning	Keeps trying even when it doesn’t work.	Shares ideas with friends and looks for better ways on their own.

Use the worksheets (1 reflection; 3–7 steps and records) and the app progress (8 check card, certificate) as evidence.

Using the apps and related materials

Requirements and tips

- Works in the browser on computers, tablets and smartphones. No installation needed (it loads the block library from the internet, so use it online).
- On narrow screens it switches to a phone layout automatically.
- Cleared stages are saved in that device’s browser. If devices are shared, check progress at the start of each lesson.
- **Teacher mode:** add `?sensei` to the end of the address to show an “Answer” button that loads the answer blocks.
- From “⚙ Settings” you can switch Japanese/English, use night mode, choose a maze (“Choose a maze…”) and show the certificate.

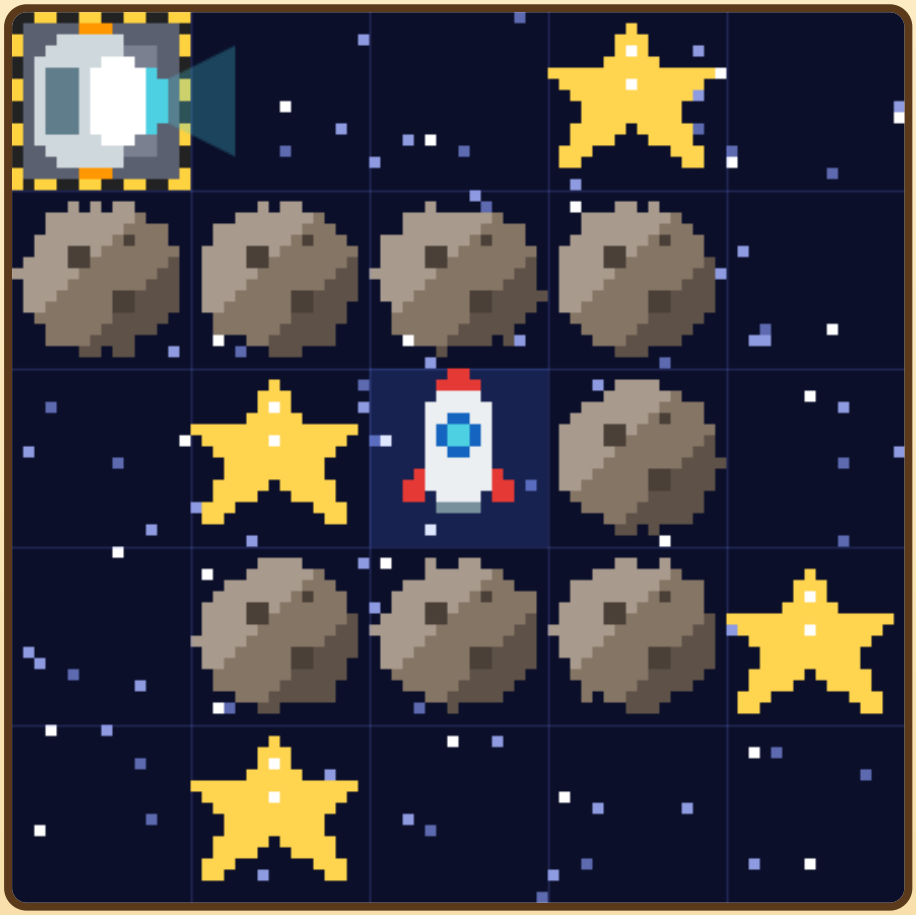
Related materials

Material	Contents
Guide (guide.html)	Goals, common mistakes, hints and answers for every task. This book is built from the same data.
Worksheets (PDF)	1 goal & reflection, 3 maze, 4 arrows, 6 keys & slime, 7 Python, 8 check card, and more.
Videos (videos.html)	Short videos showing how to solve each stage.
Lesson slides	02 Block Maze, 06 Python Maze, 09–14 maze variations (PowerPoint).
Answer list (ANSWERS.md)	Answer blocks and code for every stage.

For kids and teachers

Learn Programming in Space

From blocks to Python — 13 star-hunting stages



National Institute of Technology, Maizuru College — Yasuhito Inoue

Contents

How to use this book 2

Part 1 The space book for kids 3

Chapter 1 Collect stars and go to the rocket 4

Chapter 2 Space Star Hunt 5

Chapter 3 Space in Python 22

Well done! 36

Part 2 Guide for teachers and parents 37

About this guide 38

Overview of the stages 39

Suggested order and teaching points 40

Assessment (example) 41

Using the apps and related materials 42

How to use this book

Use this book together with the “Space Star Hunt” app on a computer or tablet. You will learn how to think like a programmer by building programs with blocks and with Python.

1. Read the **problem** and look at the picture.
2. Think about the order: trace it with your finger or write it in your notebook.
3. Build the program in the app and press “▶ Run”. If it doesn’t work, find what was different and fix it.
4. Stuck? Look at the 💡 **Hint**. When you are done, turn the page and check the 📖 **Answer** and the ✨ **Key point**.

There is more than one right answer. If it works, your program is correct, even if it is different from the book. Mistakes are hints — try again and again!

Apps in this book

App	Tasks	Grade
Space Star Hunt	7	Grade 3+
Python Space	6	Grade 5 – junior high

PART 1

Part 1 The space book for kids

Chapter 1

Collect stars and go to the rocket

Move the astronaut, pick up every ⭐ star, then board the 🚀 rocket. You bump into rocks, so plan your path.

Command	What it does
move forward	Moves one square the way it is facing
turn right / turn left	Changes direction on the spot (does not move)
⭐ pick up star	Picks up the star on this square (fails if there is none)
if there is a ⭐ star here	Runs the commands inside only when there is a star
🚀 repeat until mission complete	Repeats until all stars are collected and you reach the rocket

The big idea in this app

Check first, then act. Even if you don’t know where the stars are, you can collect them by checking every time. Automatic doors and sensor lights also check before they act.

When it doesn’t work

- Look where it stopped, and whether any stars are left.
- On the worksheet, write the number of stars collected after each move.

Chapter 2

🚀 Space Star Hunt

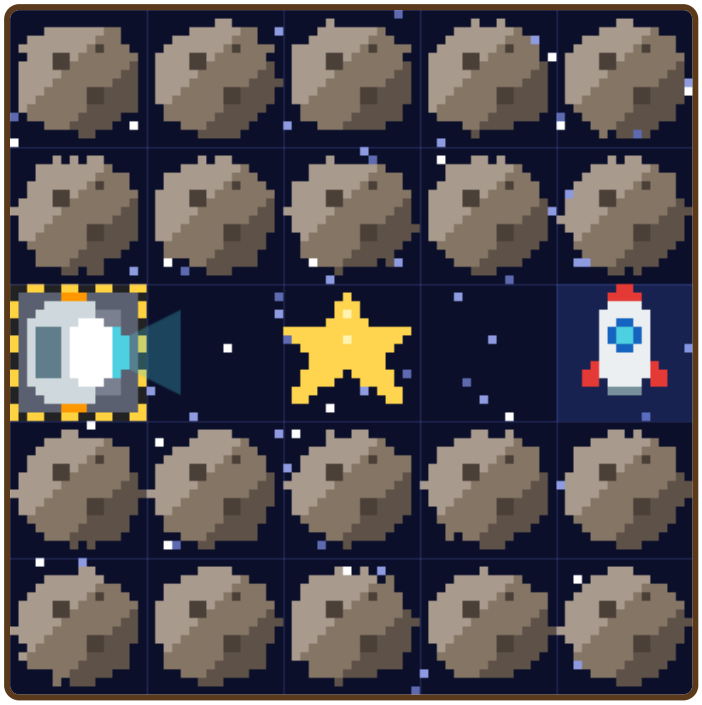
Collect stars and go to the rocket. You will go from **order → loops → patterns → if → any space**. The last stage is tested on 30 random worlds.

Commands in this chapter

move forward / turn right, left / ⭐ pick up star / repeat ○ times / if there is a ⭐ star here / if ○ is open / 🚀 repeat until mission complete

Stage 1: Pick up a star

Sequence



The start

Problem

Pick up every star ⭐, then board the rocket 🚀! Use "pick up star" on a star square.

🎯 **Goal**：Use “pick up”

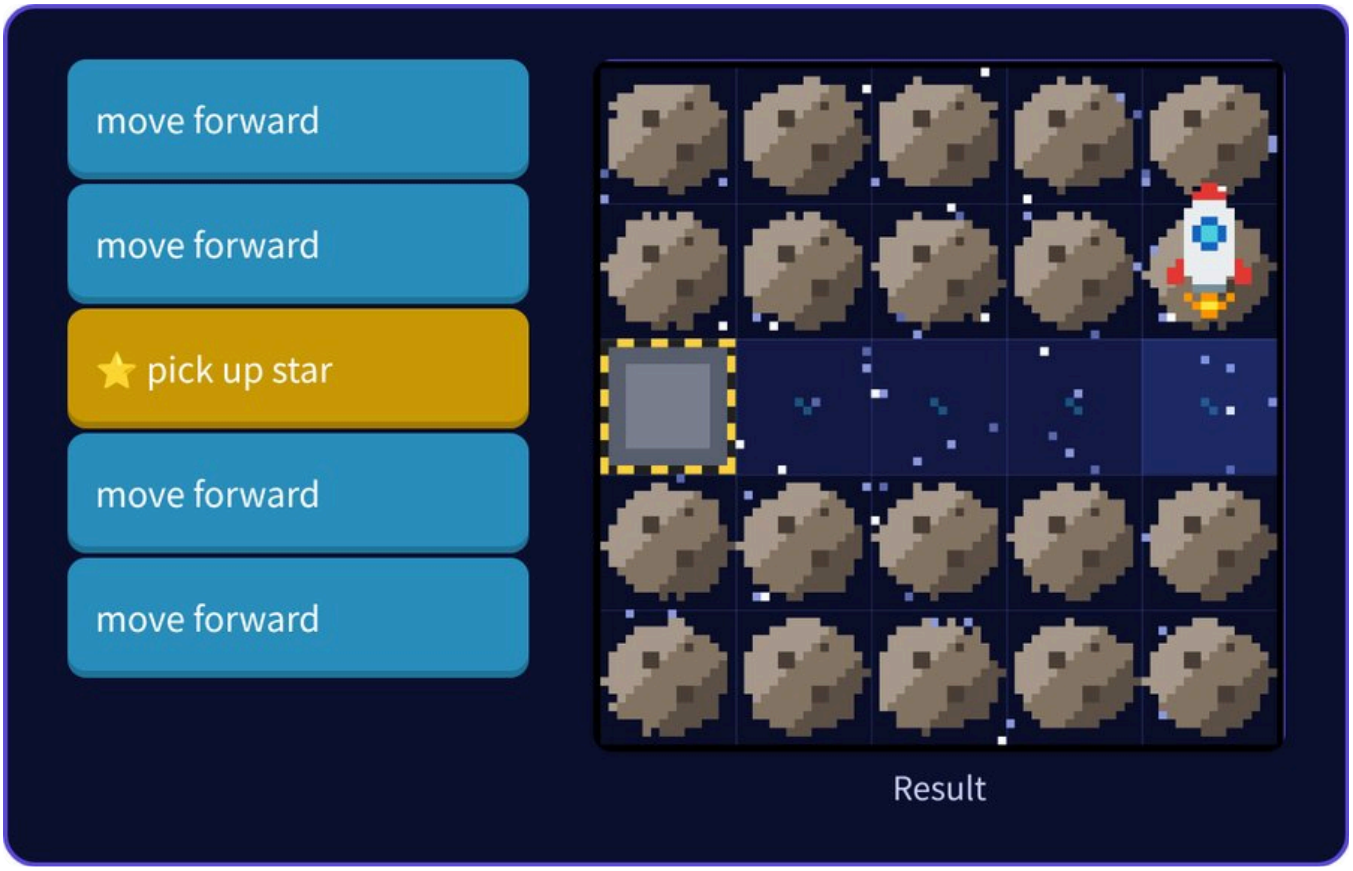
💡 **Hint (look if you get stuck)**

Forward 2 → pick up star → forward 2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Pick up a star



✨ Key point

Use “pick up” on the star square. Collect them all, then go to the rocket!

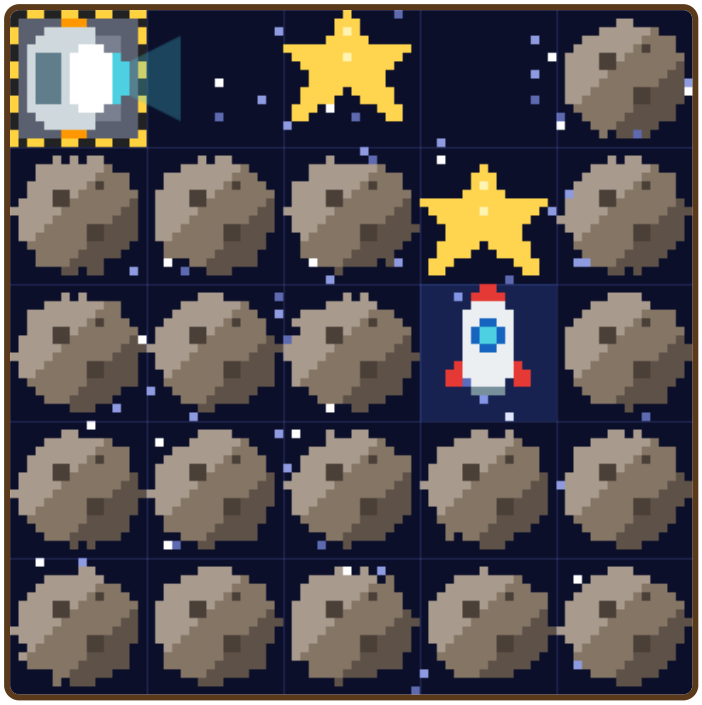
⚠ Common mistake

Picking up on a square with no star fails. Count where to pick up.

🎬 Video: `videos/en/space-01.mp4`

Stage 2: Turn and collect

Sequence Direction



The start

Problem

There are 2 stars. Be careful where you turn.

🎯 **Goal**：Turn and collect in order

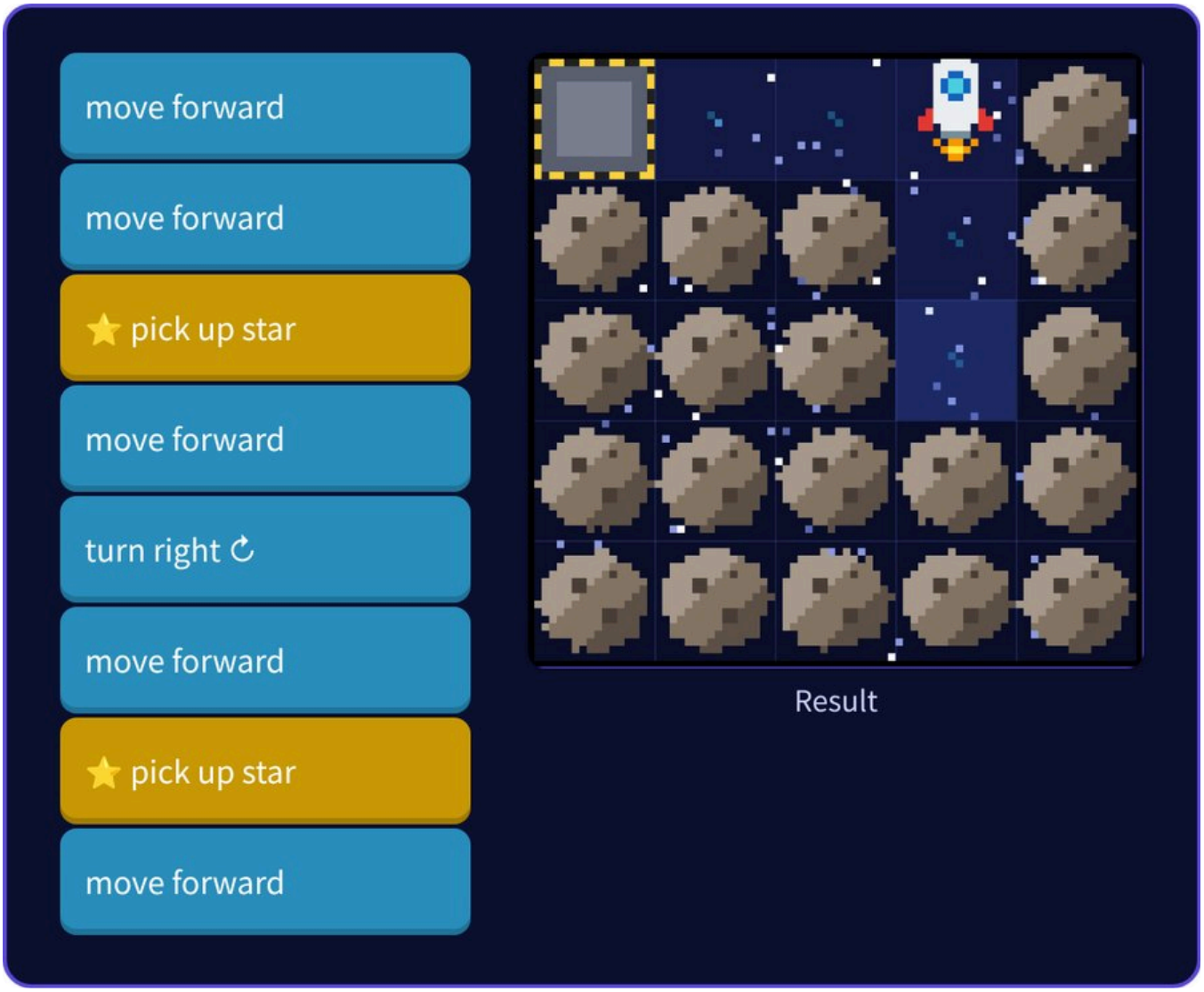
💡 **Hint (look if you get stuck)**

Forward 2 and pick up → forward 1 → turn right → forward 1 and pick up → forward 1

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Turn and collect



✨ **Key point**

The order of turning and picking up is important.

⚠ **Common mistake**

Watch the order: pick up before or after turning?

🎬 Video: `videos/en/space-02.mp4`

Stage 3: Collect with a loop

Loops



The start

Problem

The stars are in a row. Put "move → pick up" into a loop.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Collect with a loop

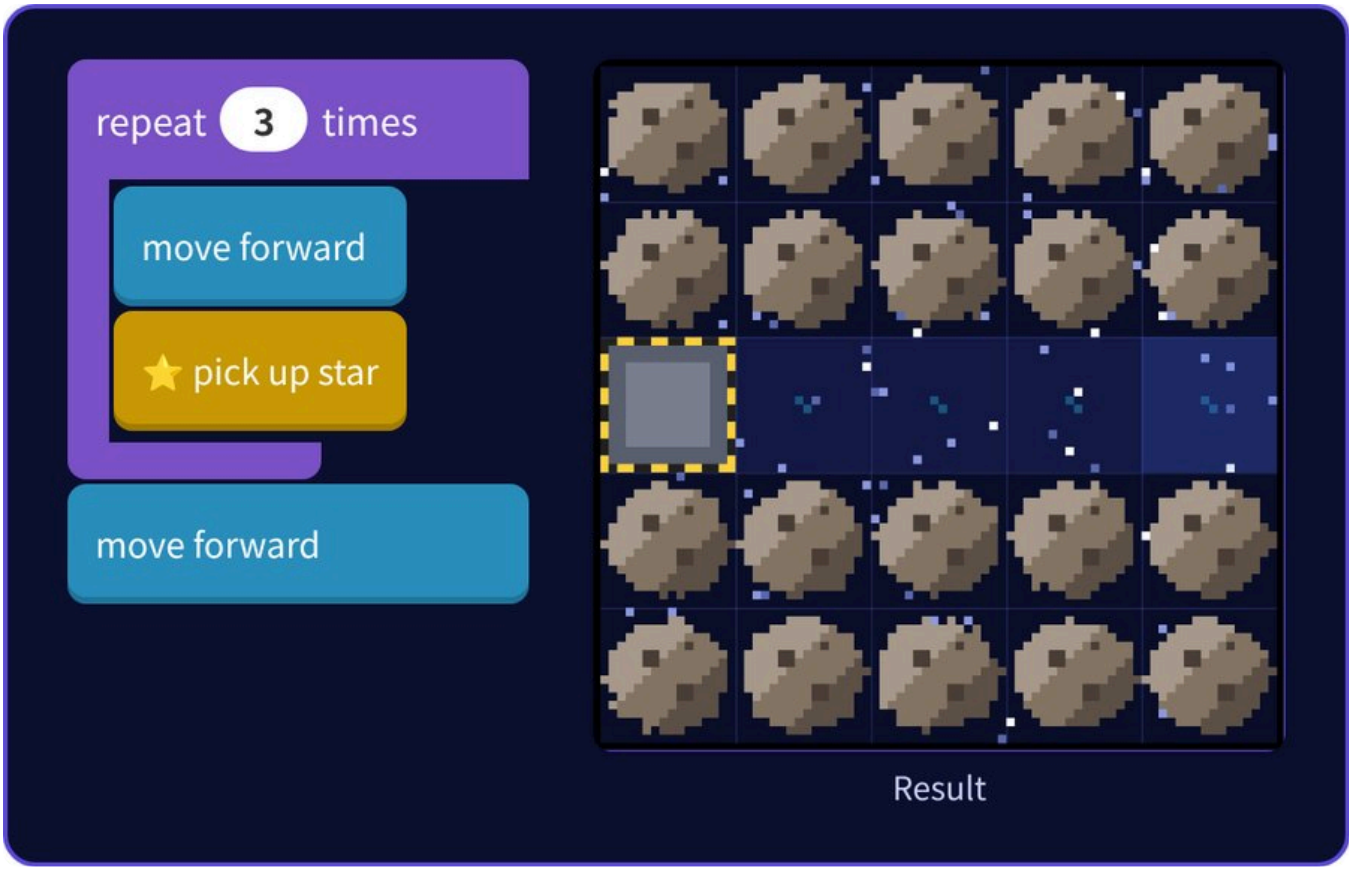
💡 **Hint (look if you get stuck)**

"repeat 3 (move forward → pick up star)" → "move forward"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Collect with a loop



✨ Key point

Repeat “move → pick up” 3 times, then move one more square.

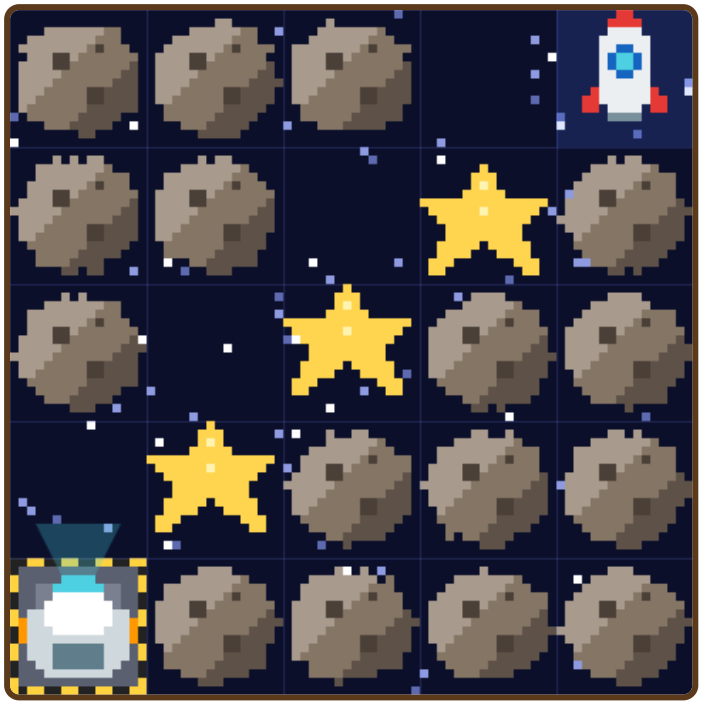
⚠ Common mistake

Forget the last “move” and you will not reach the rocket.

🎬 Video: `videos/en/space-03.mp4`

Stage 4: Star stairs

Loops Patterns



The start

Problem

There are stars on the stairs. Find the set of moves that repeats.

🎯 Target: 6 blocks or fewer

🎯 **Goal**：Find the pattern and repeat it

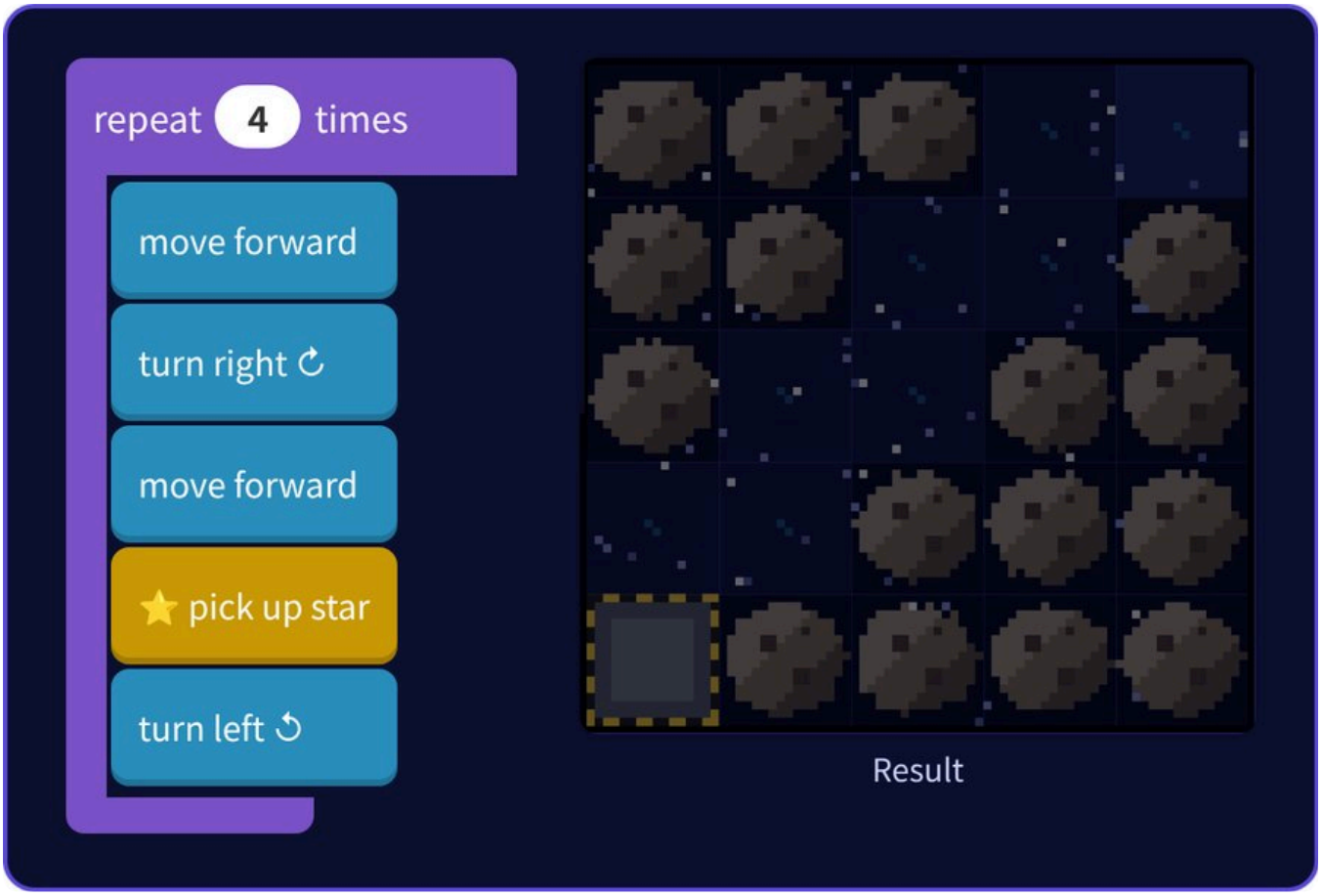
💡 **Hint (look if you get stuck)**

Repeat "forward → turn right → forward → pick up → turn left". You finish when you reach the rocket.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Star stairs



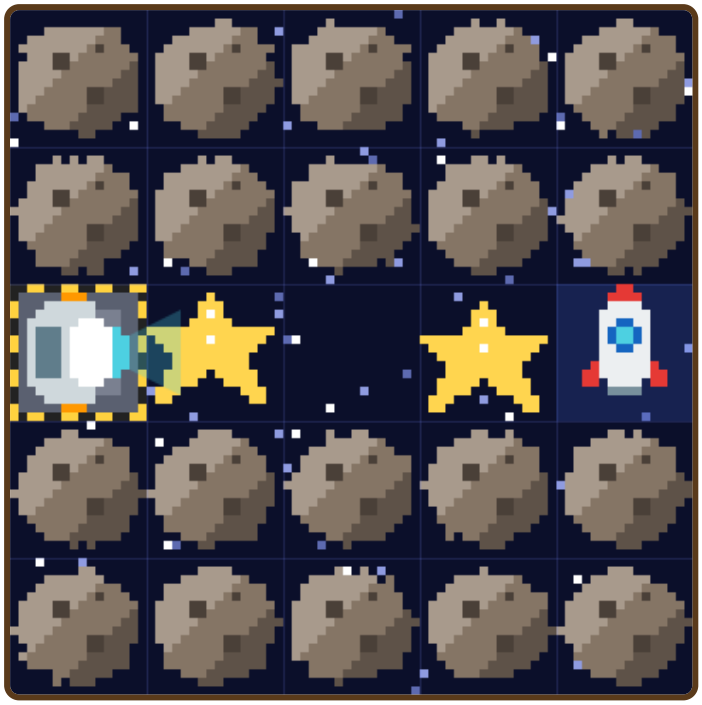
> ✨ **Key point**
>
> Each step of the stairs is one set. You reach the rocket during the 4th repeat.

> ⚠ **Common mistake**
>
> If “pick up” is in the wrong place, you pick up where there is no star.

🎬 Video: `videos/en/space-04.mp4`

Stage 5: If there is a star

Conditions Loops



The start

Problem

Some squares have stars and some don't. Picking up where there is no star is a mistake! Let the explorer check first.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Check first, then act (if)

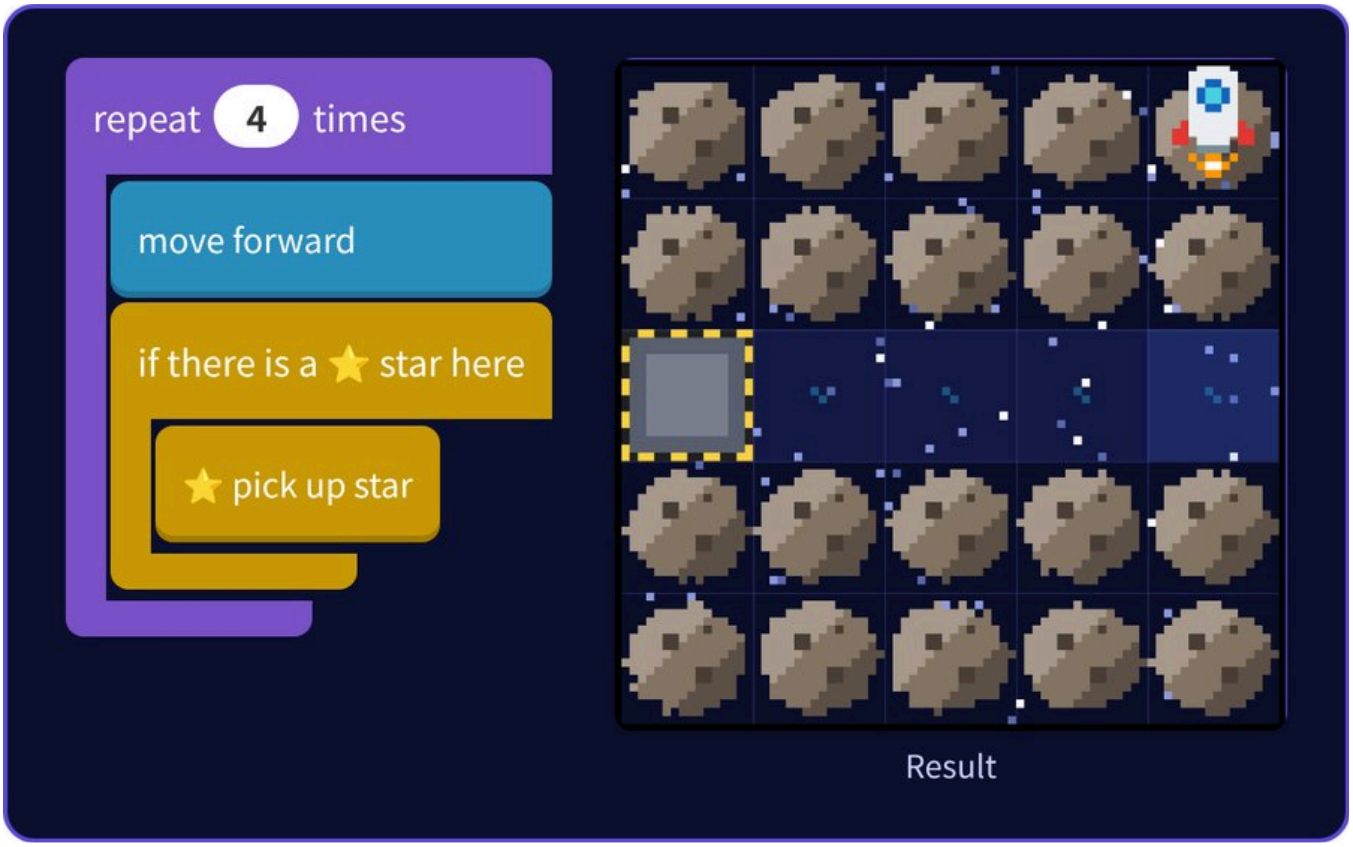
💡 **Hit (look if you get stuck)**

"repeat 4: move forward → if there is a star here, pick it up"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: If there is a star



✨ **Key point**

Pick up only when there is a star. Check with “if” before you act.

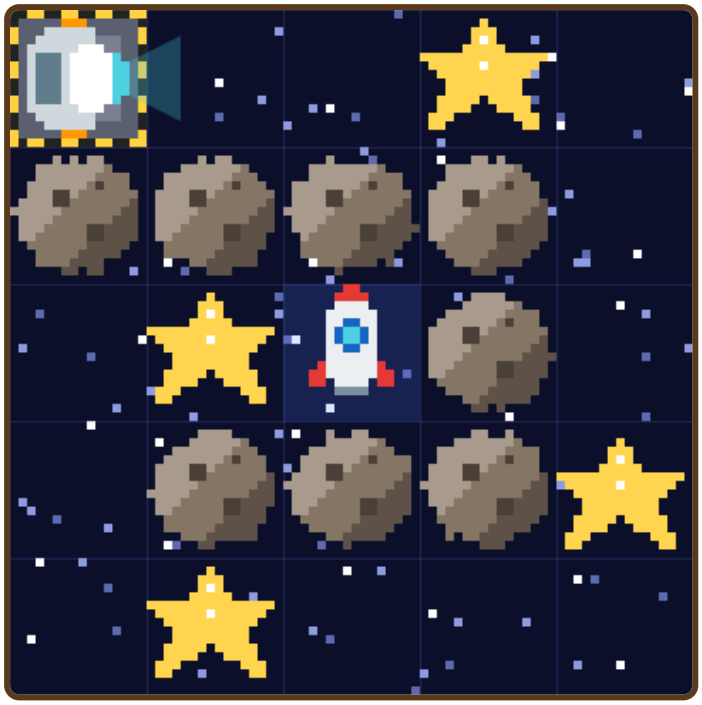
⚠ **Common mistake**

Picking up every time without “if” fails on empty squares.

🎬 Video: `videos/en/space-05.mp4`

Stage 6: Until mission complete

Conditions | Repeat until



The start

Problem

Stars are scattered around a spiral. Use "repeat until mission complete" and check as you go.

🎯 Target: 6 blocks or fewer

🎯 **Goal**：Repeat until the mission is complete

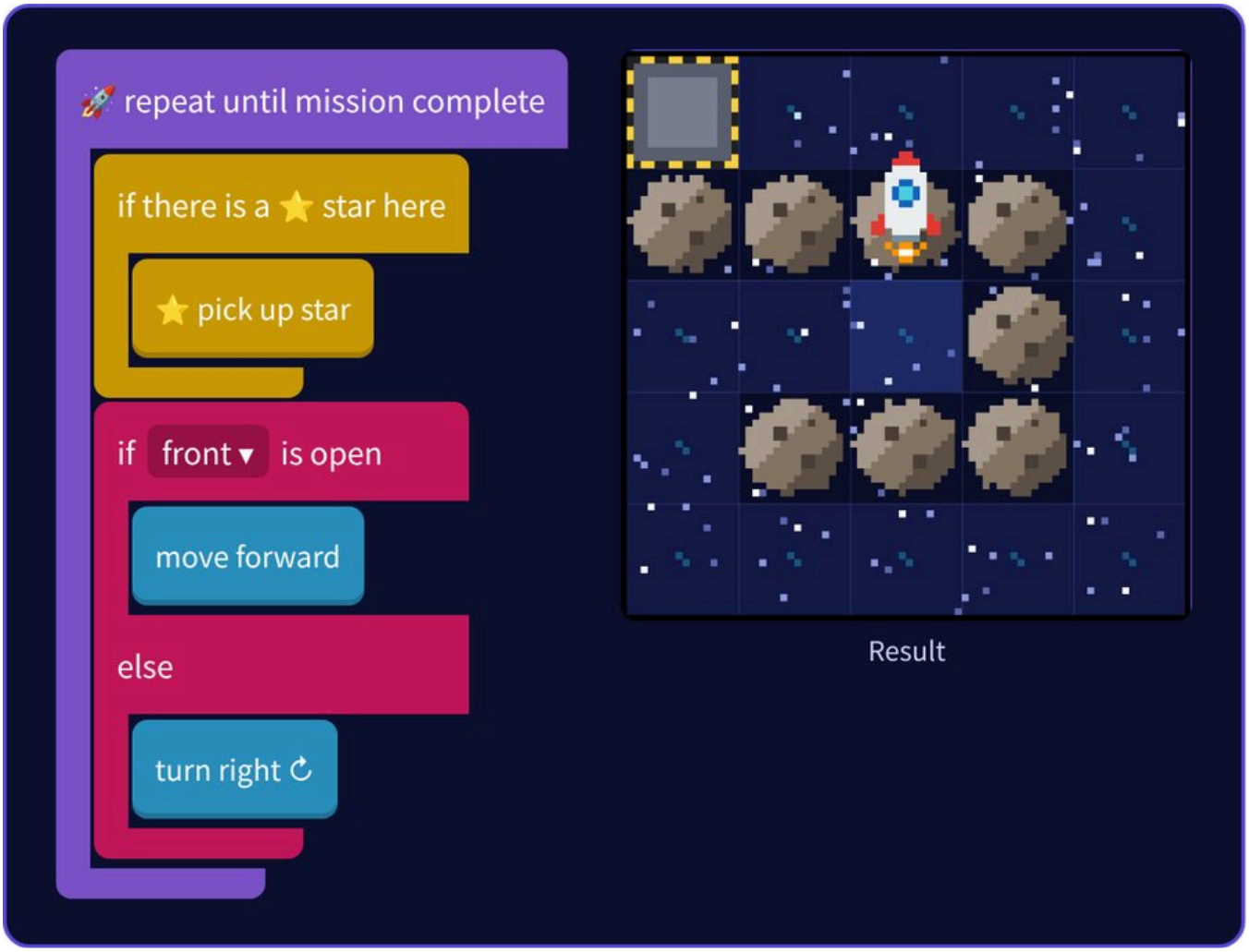
💡 **Hint (look if you get stuck)**

Inside the loop: if there is a star, pick it up → if front is open, move forward, else turn right

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Until mission complete



✨ **Key point**

Each time, first check under your feet, then move.

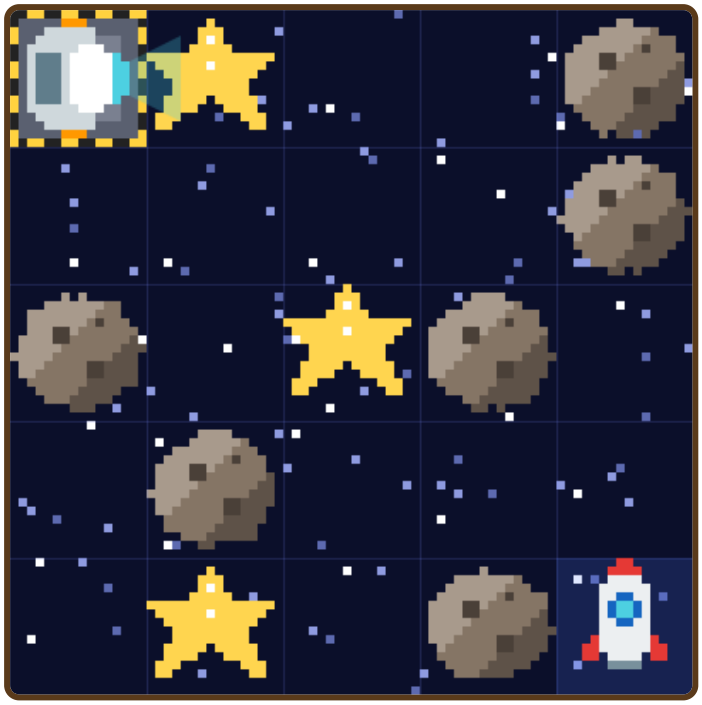
⚠ **Common mistake**

Forgetting to check at the corners leaves stars behind.

🎬 Video: `videos/en/space-06.mp4`

Stage 7: Any space!

Generalization | Conditions | Repeat until



The start

Example: the world changes every time you run

Problem

Final challenge! Your program is tested in 30 random spaces. Collect all the stars and board the rocket to clear!

🎯 **Goal**：Clear any space

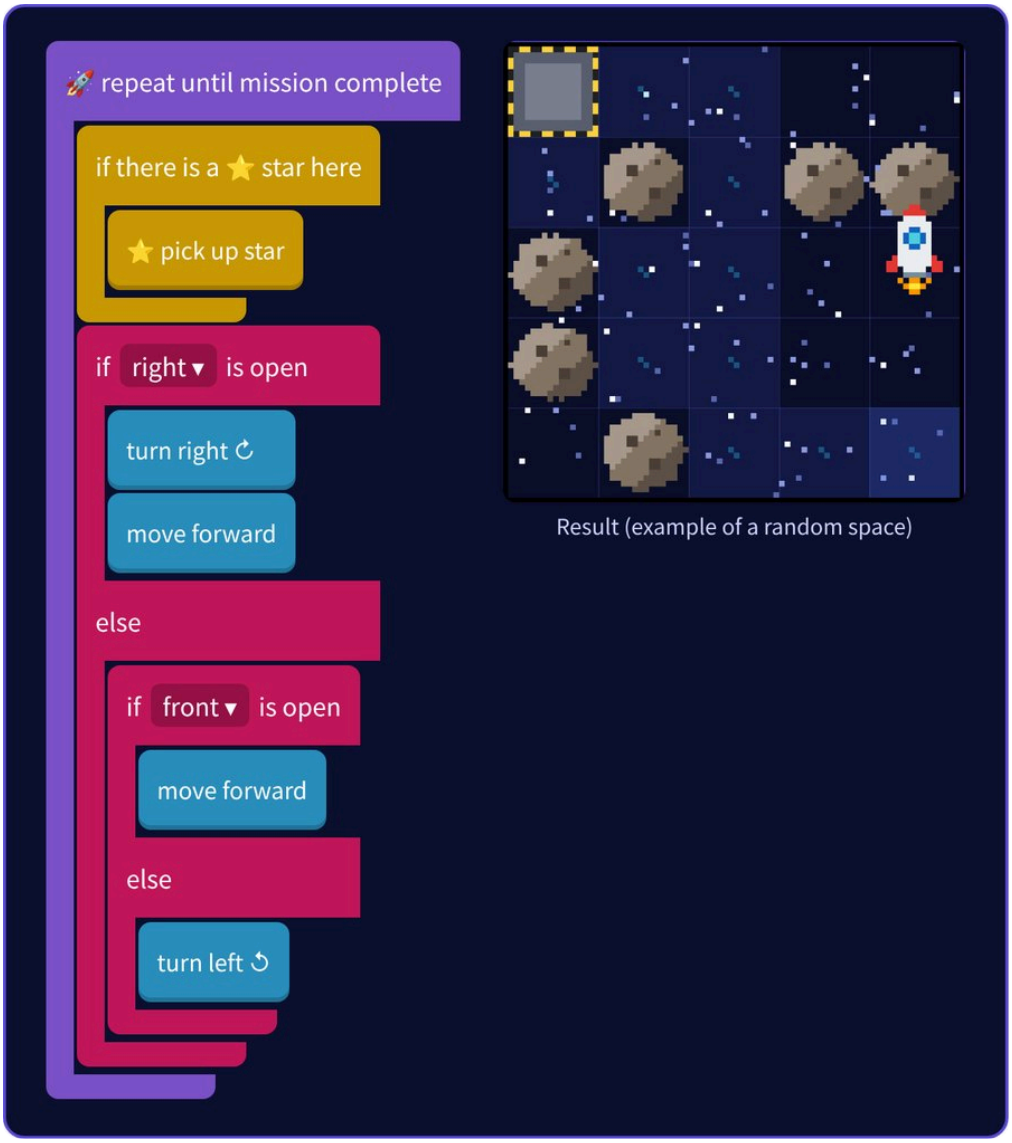
💡 **Hint (look if you get stuck)**

Use the "right-hand rule" to follow the walls, and pick up stars when you find them. It's OK to pass the rocket — keep walking until you have every star.

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 7: Any space!



Result (example of a random space)

> ✨ **Key point**
>
> Right-hand rule + pick up stars. It is OK to pass the rocket!

> ⚠ **Common mistake**
>
> You may think it stops at the rocket — it keeps going until every star is collected.

🎬 Video: `videos/en/space-07.mp4`

Column

Check first, then act

In stage 5, some squares have stars and some don’t. If you memorize “pick up on squares 2 and 4”, it breaks when the stars move. If you write “if there is a star, pick it up”, it works for any layout. That is the power of conditions.

Chapter 3

🐍 Space in Python

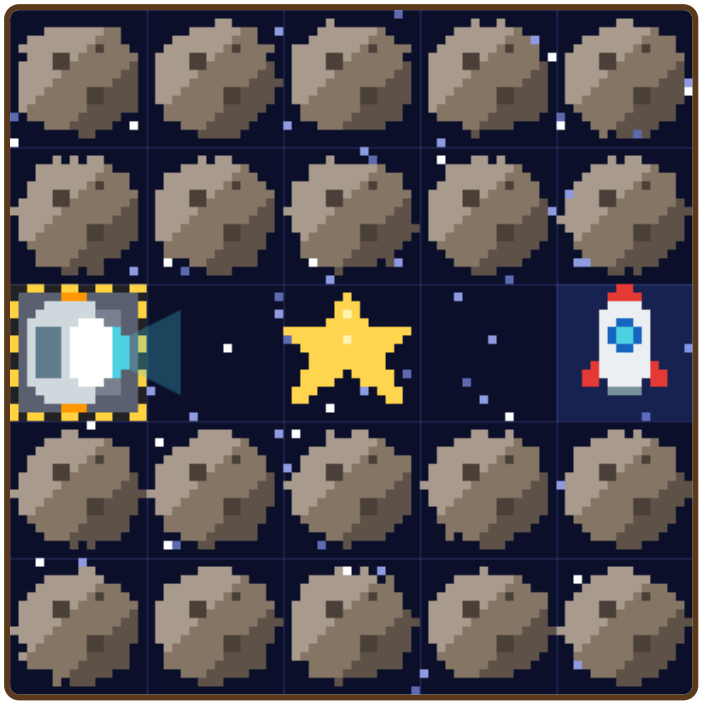
Write star hunting in **Python**: `forward()`, `turn_right()`, `pick_up()`; check with `if has_star():`, and repeat to the end with `while not mission_done():`.

Commands in this chapter

forward() / turn_right() / turn_left() / pick_up() / for i in range(4): / if has_star(): / while not mission_done():

Stage 1: Pick up a star

Sequence | Python syntax (colon & indent)



The start

Problem

Pick up the star ⭐ with pick_up(), then go to the rocket 🚀. pick_up() works only on a star square.

🎯 **Goal**：Use pick_up()

💡 **Hint (look if you get stuck)**

forward() ×2 → pick_up() → forward() ×2

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Pick up a star

```
forward()
forward()
pick_up()
forward()
forward()
```

✨ Key point

pick_up() picks up the star on your square. Collect them all, then go to the rocket!

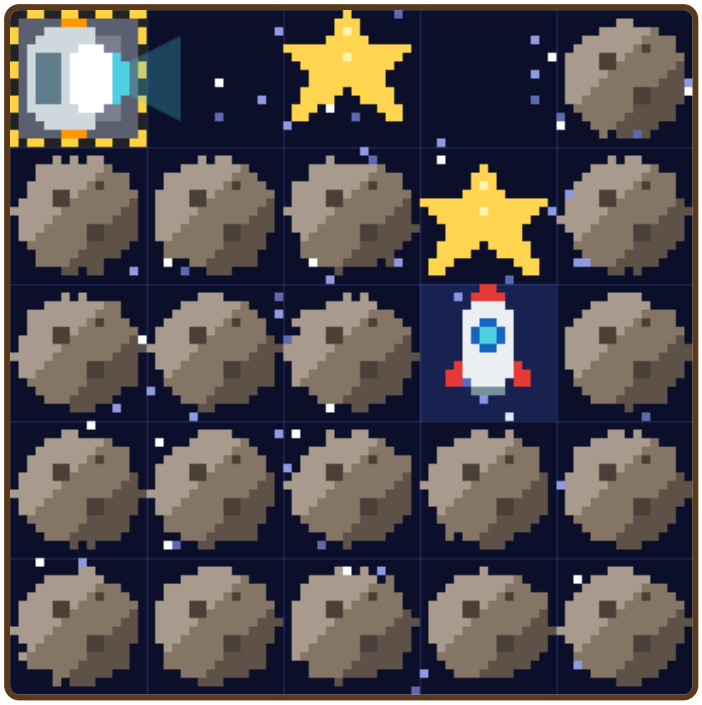
⚠ Common mistake

Writing pickup() or pick up() causes a NameError. Don’t forget the “_”.

🎬 Video: `videos/en/pyspace-01.mp4`

Stage 2: Turn and collect

Sequence | Direction | Python syntax (colon & indent)



The start

Problem

There are 2 stars. Use turn_right() where the path turns.

🎯 **Goal**：Turn and collect in order

💡 **Hint (look if you get stuck)**

forward() ×2, pick_up() → forward() → turn_right() → forward(), pick_up() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Turn and collect

```
forward()
forward()
pick_up()
forward()
turn_right()
forward()
pick_up()
forward()
```

✨ Key point

turn_right() only changes direction. The order of turning and picking up matters.

⚠ Common mistake

pick_up() on an empty square fails. Check which square has the star.

🎬 Video: `videos/en/pyspace-02.mp4`

Stage 3: Collect with for

Loops | Python syntax (colon & indent)



The start

Problem

The stars are in a row. Repeat “forward() → pick_up()” with for. Indent the lines inside the loop by 4 spaces.

🎯 **Goal**：Repeat with for

💡 **Hint (look if you get stuck)**

for i in range(3): → (indented) forward() and pick_up() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Collect with for

```
for i in range(3):
    forward()
    pick_up()
forward()
```

✨ Key point

To put 2 lines inside for, indent both by 4 spaces. The last forward() is not indented.

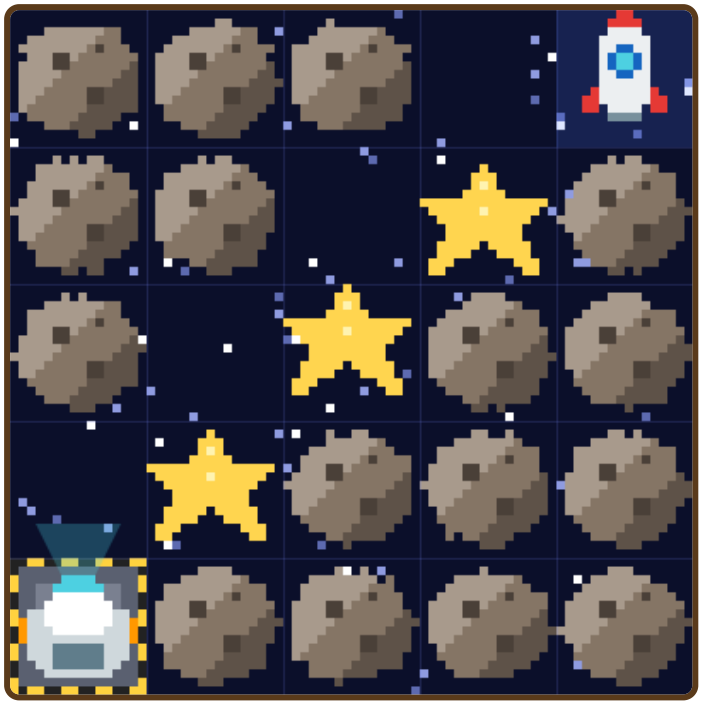
⚠ Common mistake

If you forget to indent pick_up(), it runs only once.

🎬 Video: `videos/en/pyspace-03.mp4`

Stage 4: Star stairs

Loops | Patterns | Python syntax (colon & indent)



The start

Problem

Find the set of moves that repeats on the stairs and put all of it inside the for loop.

🎯 **Goal**：Find the set that repeats

💡 **Hint (look if you get stuck)**

for i in range(4): forward(), turn_right(), forward(), pick_up(), turn_left()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: Star stairs

```
for i in range(4):
    forward()
    turn_right()
    forward()
    pick_up()
    turn_left()
```

✨ Key point

Put the whole 5-line set inside for. You reach the rocket during the 4th repeat, and it ends there.

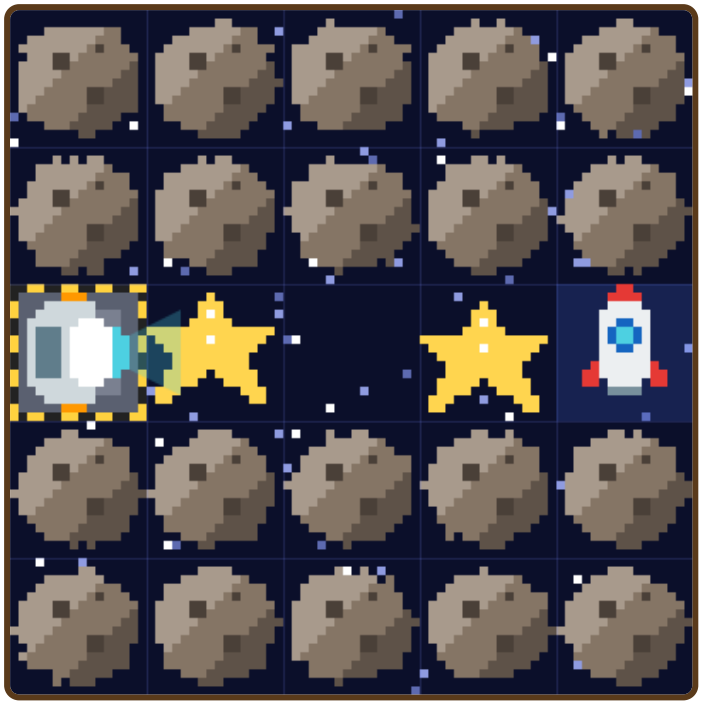
⚠ Common mistake

Forgetting turn_left() keeps changing your direction and you hit an asteroid.

🎬 Video: `videos/en/pyspace-04.mp4`

Stage 5: if has_star():

Conditions | Loops | Nesting | Python syntax (colon & indent)



The start

Problem

Some squares have stars and some don’t. has_star() checks the square you are on. Use if so you only pick up where there is a star.

🎯 **Goal**：Check with if before you act

💡 **Hint (look if you get stuck)**

for i in range(4): forward() → if has_star(): → (indent more) pick_up()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: if has_star():

```
for i in range(4):
    forward()
    if has_star():
        pick_up()
```

✨ **Key point**

Lines inside if are indented one more level (8 spaces in total).

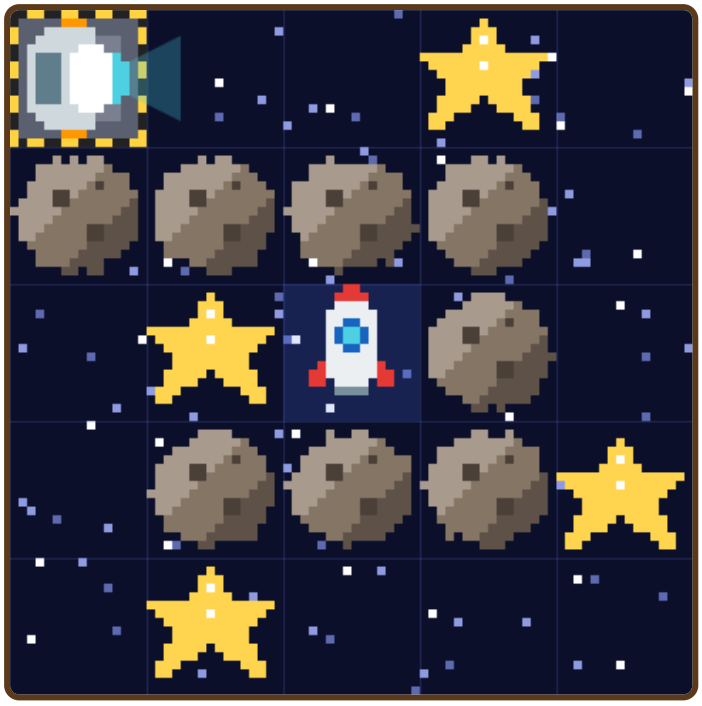
⚠ **Common mistake**

Forgetting the “:” after if has_star() causes a SyntaxError.

🎬 Video: `videos/en/pyspace-05.mp4`

Stage 6: while not mission_done():

Conditions | Repeat until | Nesting | Python syntax (colon & indent)



The start

Problem

Stars are scattered around a spiral. Repeat with “while not mission_done():” and check as you go.

🎯 **Goal**：Repeat with while not mission_done():

💡 **Hint (look if you get stuck)**

Inside while: if has_star(): pick_up() → if is_open('front'): forward() else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 6: while not mission_done():

```
while not mission_done():
    if has_star():
        pick_up()
    if is_open('front'):
        forward()
    else:
        turn_right()
```

✨ Key point

mission_done() becomes True when you have every star and are at the rocket. not repeats “while it is not done yet”.

⚠ Common mistake

If you join the two ifs with elif, you will not move on the turn you pick up a star (it can still clear).

🎬 Video: `videos/en/pyspace-06.mp4`

Column

How to read while not

`while not mission_done():` means “repeat while the mission is not done”. `not` means “is not”. Indent the lines inside by 4 spaces.

Well done!

You worked hard to the end! Let’s look back at the ideas you used.

Idea	Where did it appear?
Order	Line up move, turn and pick up (stages 1–2)
Loops & patterns	Pack “move → pick up” together (stages 3–4)
If (conditions)	Pick up only when there is a star (stage 5)
Repeat until	Until the mission is complete (stages 6–7)
Python	Write the same thing in words (code)

When you clear every stage, get your certificate from “⚙ Settings” → “📜 Certificate”.

PART 2

Part 2 Guide for teachers and parents

About this guide

This book covers “Space Star Hunt” (7 stages) and “Python Space” (6 stages). Part 1 is written in simple language for students.

The core idea is a condition: “check the situation, then act”. Picking up where there is no star fails, which makes conditions necessary. The final stage is tested on 30 random worlds, so students need a general method.

Aims

- Combine sequence, loops and conditions to reach a goal.
- Notice and explain the value of checking before acting.
- In Python, learn how to write if, while and function calls.

Overview of the stages

App	#	Stage	Ideas
🚀 Space Star Hunt	1	Pick up a star	Sequence
	2	Turn and collect	Sequence, Direction
	3	Collect with a loop	Loops
	4	Star stairs	Loops, Patterns
	5	If there is a star	Conditions, Loops
	6	Until mission complete	Conditions, Repeat until
	7	Any space!	Generalization, Conditions, Repeat until
🐍 Python Space	1	Pick up a star	Sequence, Python syntax (colon & indent)
	2	Turn and collect	Sequence, Direction, Python syntax (colon & indent)
	3	Collect with for	Loops, Python syntax (colon & indent)
	4	Star stairs	Loops, Patterns, Python syntax (colon & indent)
	5	if has_star():	Conditions, Loops, Nesting, Python syntax (colon & indent)
	6	while not mission_done():	Conditions, Repeat until, Nesting, Python syntax (colon & indent)

Suggested order and teaching points

Students	Stages	Lessons	Key ideas
Grades 3–4	1–4	1–2	Order, loops, patterns
Grades 4–6	5–7	2	Conditions, repeat until, generalization
Grade 6 – junior high	Python 1–6	2–3	if/while, function calls

Common difficulties and support

- **Picking up where there is no star:** in stage 5, show that a memorized route breaks when the layout changes, and draw out the idea of “checking”.
- **Nesting a condition in a loop:** have students say one square’s action in words (move → if star, pick up) before building.
- **Finding the pattern (stage 4):** trace the stairs and circle the repeating “set”.
- **Python:** confirm what `while not` means and that indentation decides what is inside.

Example 45-minute lesson (stage 5)

Time	Activity	Support
Intro 5 min	Notice that some squares have stars and some don’t; share the goal.	Recall “move → pick up”.
Main 30 min	Build a route for this layout; see it break when the layout changes; rebuild with “if”.	Record the star count on Worksheet 5.
Wrap-up 10 min	Summarize the value of conditions; talk about real-life examples.	Automatic doors, sensor lights.

Assessment (example)

Area	Meets expectations (B)	Exceeds (A), e.g.
Knowledge & skills	Combines blocks (code) correctly and completes the task.	Uses loops and conditions to complete it with a short, clear program.
Thinking, judgment & expression	Finds where the error is from the result and fixes it.	Explains the rule or steps in words and makes a program that works in other cases too.
Attitude toward learning	Keeps trying even when it doesn’t work.	Shares ideas with friends and looks for better ways on their own.

Use the worksheets (1 reflection; 2, 5, 7 etc.) and the app progress (8 check card, certificate) as evidence.

Using the apps and related materials

Requirements and tips

- Works in the browser on computers, tablets and smartphones. No installation needed. An internet connection is required.
- Cleared stages are saved in that device’s browser. If devices are shared, check progress at the start of each lesson.
- **Teacher mode:** add `?sensei` to the end of the address to show an “Answer” button.
- From “⚙ Settings” you can switch Japanese/English, use the pixel font, switch to the Python version and show the certificate.

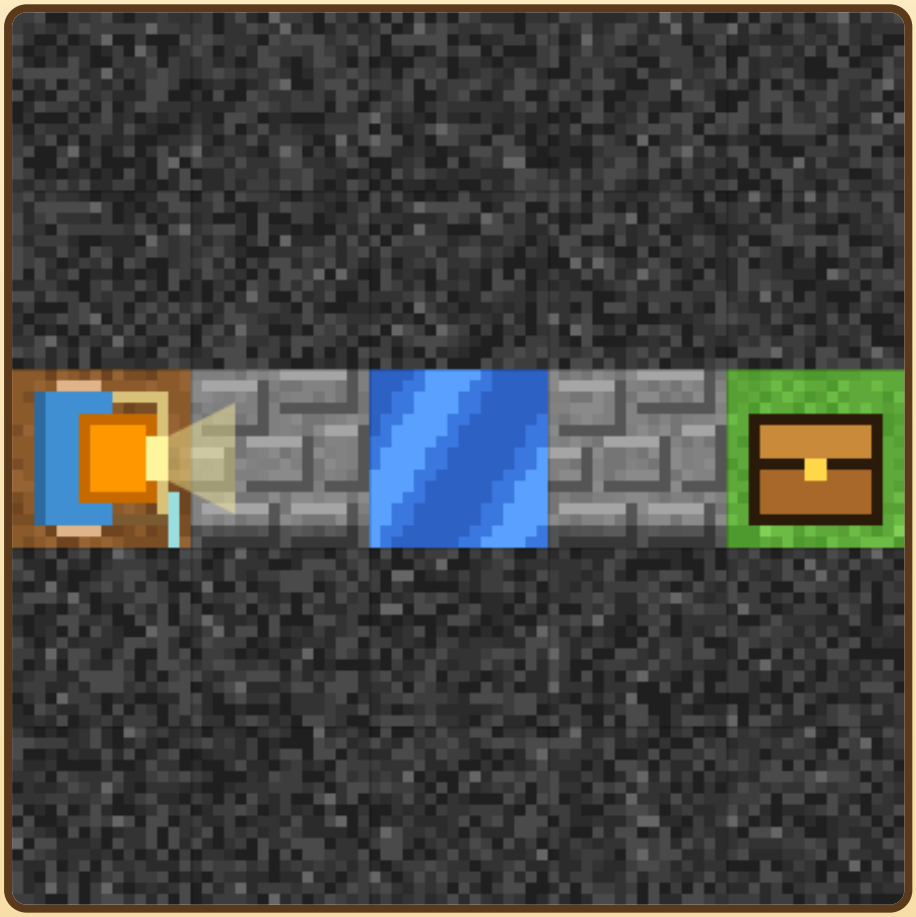
Related materials

Material	Contents
Guide (guide.html)	Goals, common mistakes, hints and answers for every task. This book is built from the same data.
Worksheets (PDF)	5 Space / Break & Build (order and count table), 7 Python, 1 goal & reflection, 8 check card.
Lesson plans (Japanese)	Example 45-minute lessons (open them from each task in the Guide).
Videos (videos.html)	Short videos showing how to solve each task.
Lesson slides	03 Space Star Hunt, 07 Python Space (PowerPoint).

For kids and teachers

Learn Programming by Breaking & Building

From blocks to Python — 14 adventure stages



National Institute of Technology, Maizuru College — Yasuhito Inoue

Contents

How to use this book 2

Part 1 The adventure book for kids 3

Chapter 1 Break, carry, place and go 4

Chapter 2 Break & Build Quest 5

Chapter 3 Break & Build in Python 21

Well done! 37

Part 2 Guide for teachers and parents 38

About this guide 39

Overview of the stages 40

Suggested order and teaching points 41

Assessment (example) 42

Using the apps and related materials 43

How to use this book

Use this book together with the “Break & Build Quest” app on a computer or tablet. You will learn how to think like a programmer by building programs with blocks and with Python.

1. Read the **problem** and look at the picture.
2. Think about the order: trace it with your finger or write it in your notebook.
3. Build the program in the app and press “▶ Run”. If it doesn’t work, find what was different and fix it.
4. Stuck? Look at the 💡 **Hint**. When you are done, turn the page and check the 📖 **Answer** and the ✨ **Key point**.

There is more than one right answer. If it works, your program is correct, even if it is different from the book. Mistakes are hints — try again and again!

Apps in this book

App	Tasks	Grade
Break & Build Quest	7	Grade 4+
Python Break & Build	7	Grade 5 – junior high

PART 1

Part 1 The adventure book for kids

Chapter 1

Break, carry, place and go

Move the explorer to the treasure chest. Break the stone blocks in the way, and place blocks on water to build bridges.

Command	What it does
move forward / turn right, left	Move one square / change direction on the spot
⛏️ break the block in front	Breaks the stone in front. **You carry one more block.**
🧱 place a block in front	Places a block in front (on water it becomes a bridge). **You carry one less block.**
if the front is ○○	Checks whether the front is stone, water or an open path
repeat until goal	Repeats until you reach the chest

The big idea in this app

Remember **how many you carry**. With no blocks, you cannot build a bridge, so “break first, then place” matters. Computers keep numbers like this in **variables**.

When it doesn’t work

- Look at “Blocks you have” on the screen.
- On the worksheet, write how many you carry after each move.

Chapter 2

🧱 Break & Build Quest

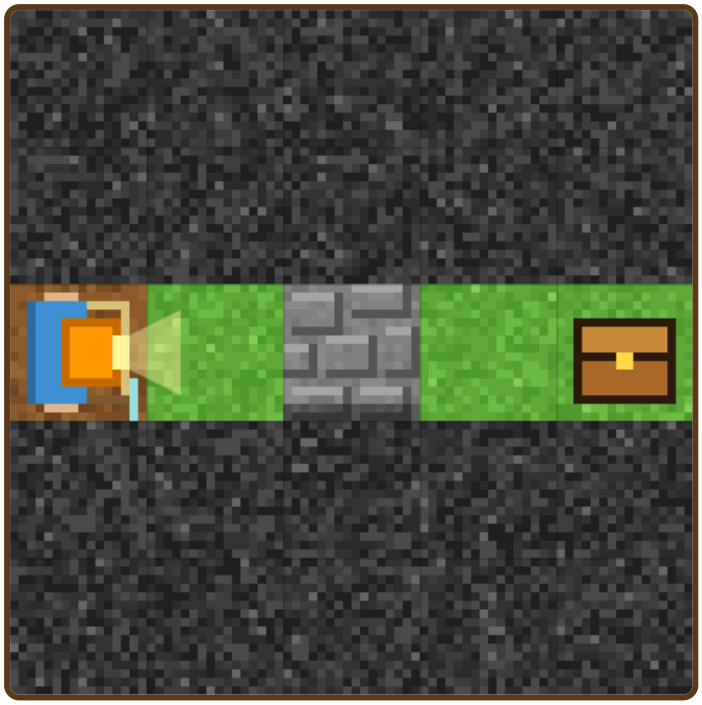
Break stone, carry it, and place it on water to build bridges. You will go from **order → loops → counting what you carry → if → two ifs**.

Commands in this chapter

move forward / turn right, left / ⛏️ break / 🧱 place / repeat ○ times / if the front is ○○ (else) / repeat until goal

Stage 1: Break and go

Sequence



The start

Problem

A stone block is in the way. Break it with "⛏️ break the block in front", then go to the treasure chest!

🎯 **Goal**：Use “break”

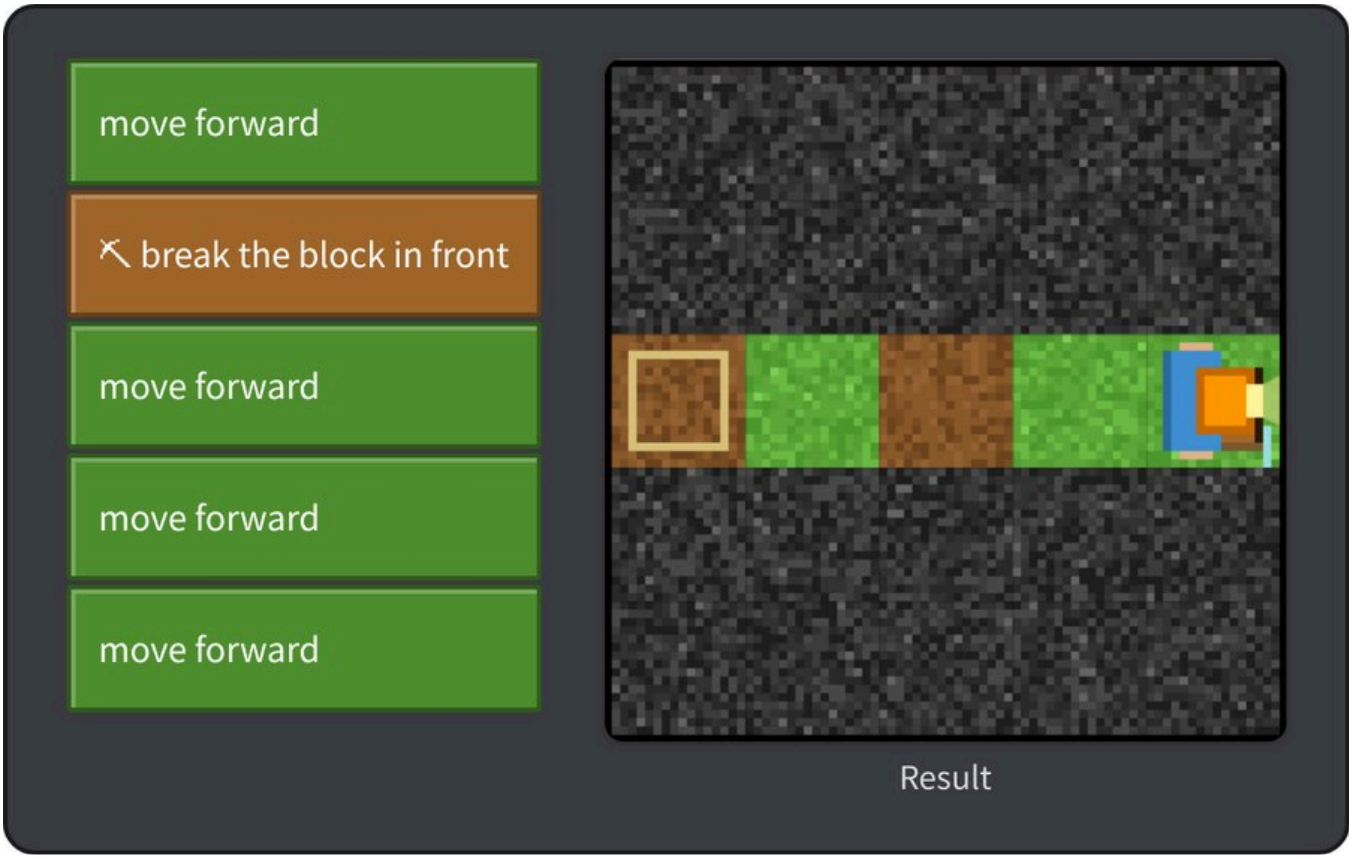
💡 **Hint (look if you get stuck)**

Forward 1 → break → forward 3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Break and go



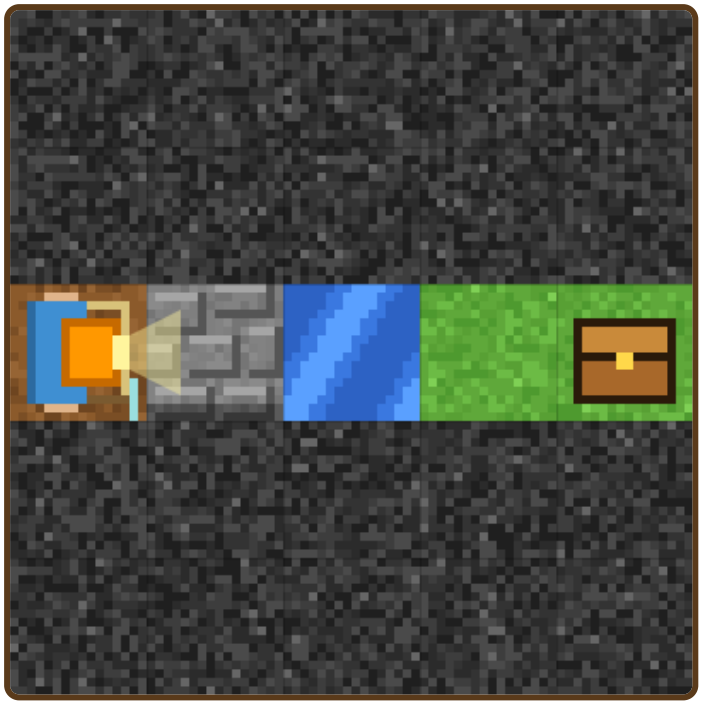
> ✨ **Key point**
>
> Break the stone in front, and it becomes a path.

> ⚠ **Common mistake**
>
> Moving without breaking bumps into the stone.

🎬 Video: `videos/en/craft-01.mp4`

Stage 2: Build a bridge

Sequence Counting



The start

Problem

There is water ahead. You need a block to make a bridge. Break the stone first — then you will have a block!

🎯 **Goal**：Break → hold → place

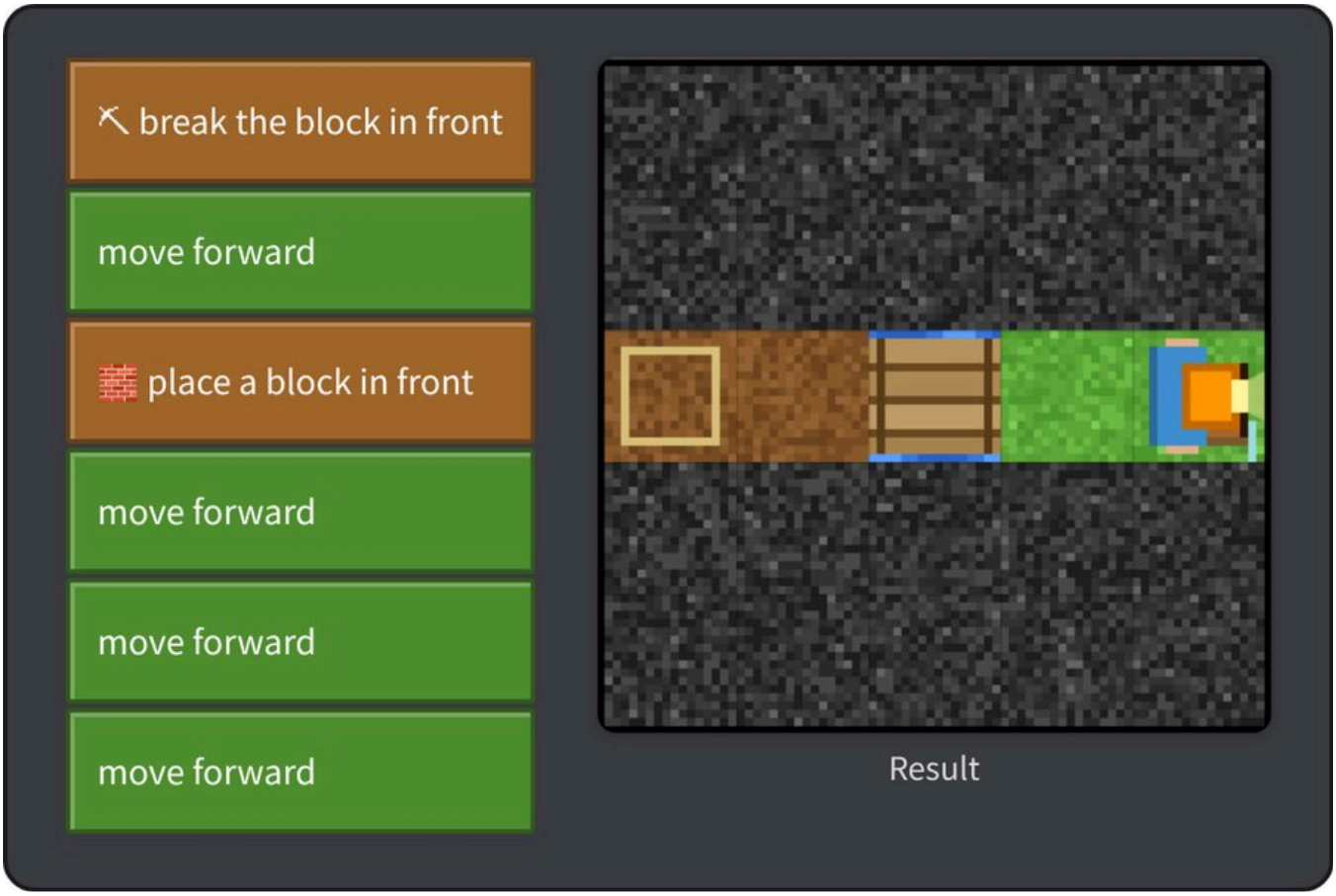
💡 **Hint (look if you get stuck)**

Break → forward → place → forward 3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Build a bridge



✨ **Key point**

You start with no blocks. Break first, then place.

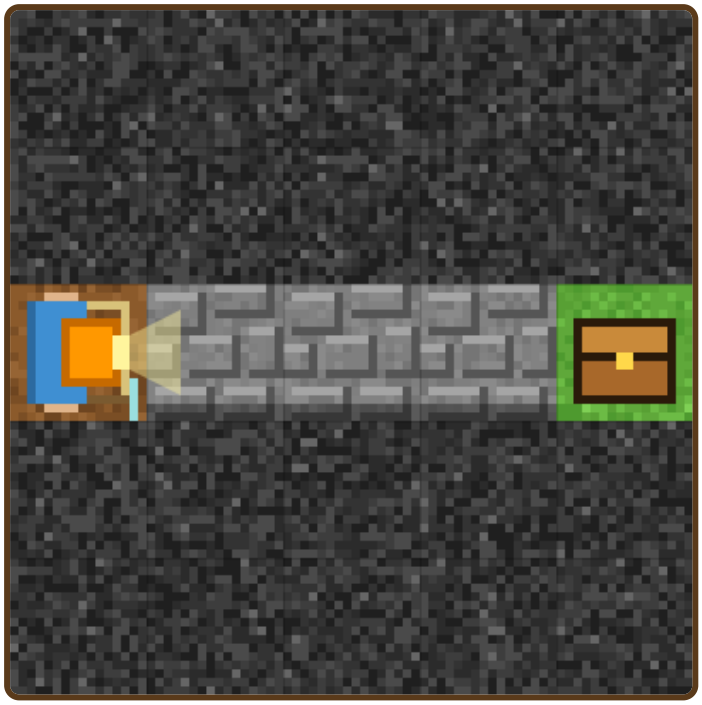
⚠ **Common mistake**

Placing first fails because you have no blocks yet.

🎬 Video: `videos/en/craft-02.mp4`

Stage 3: Break with a loop

Loops



The start

Problem

Stone blocks in a row! Put "break → move" into a loop.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Break with a loop

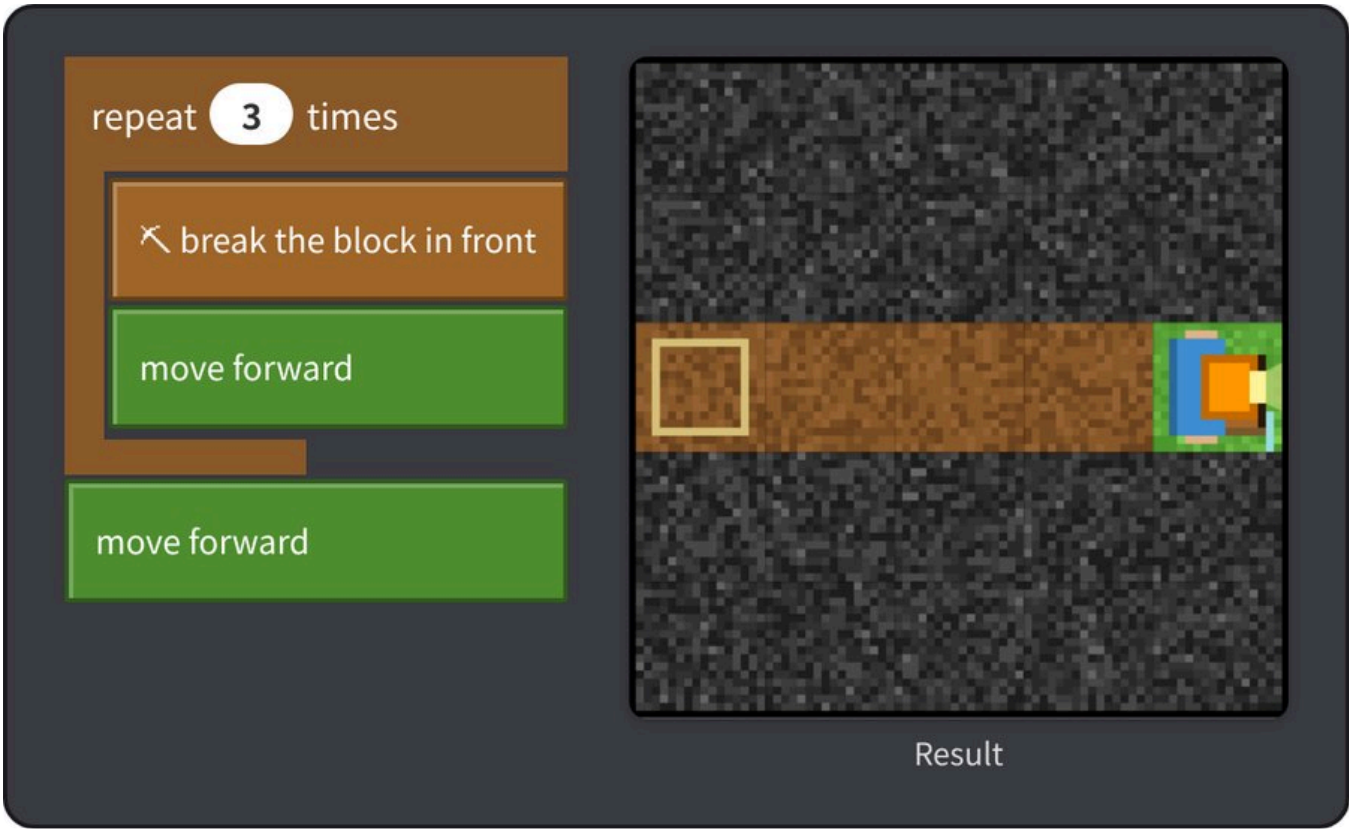
💡 **Hint (look if you get stuck)**

"repeat 3 (break → move forward)" → "move forward"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Break with a loop



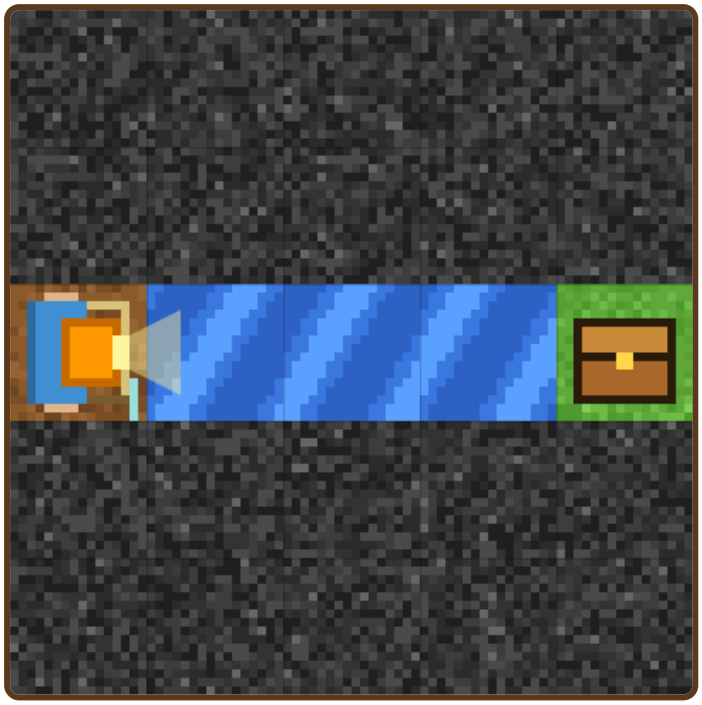
> ✨ **Key point**
>
> You repeated “break → move” 3 times.

> ⚠ **Common mistake**
>
> Moving first bumps into stone. Break first.

🎬 Video: `videos/en/craft-03.mp4`

Stage 4: A long bridge

Loops Counting



The start

Problem

You have 3 blocks. Make a long bridge over the river.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Use up the blocks you have

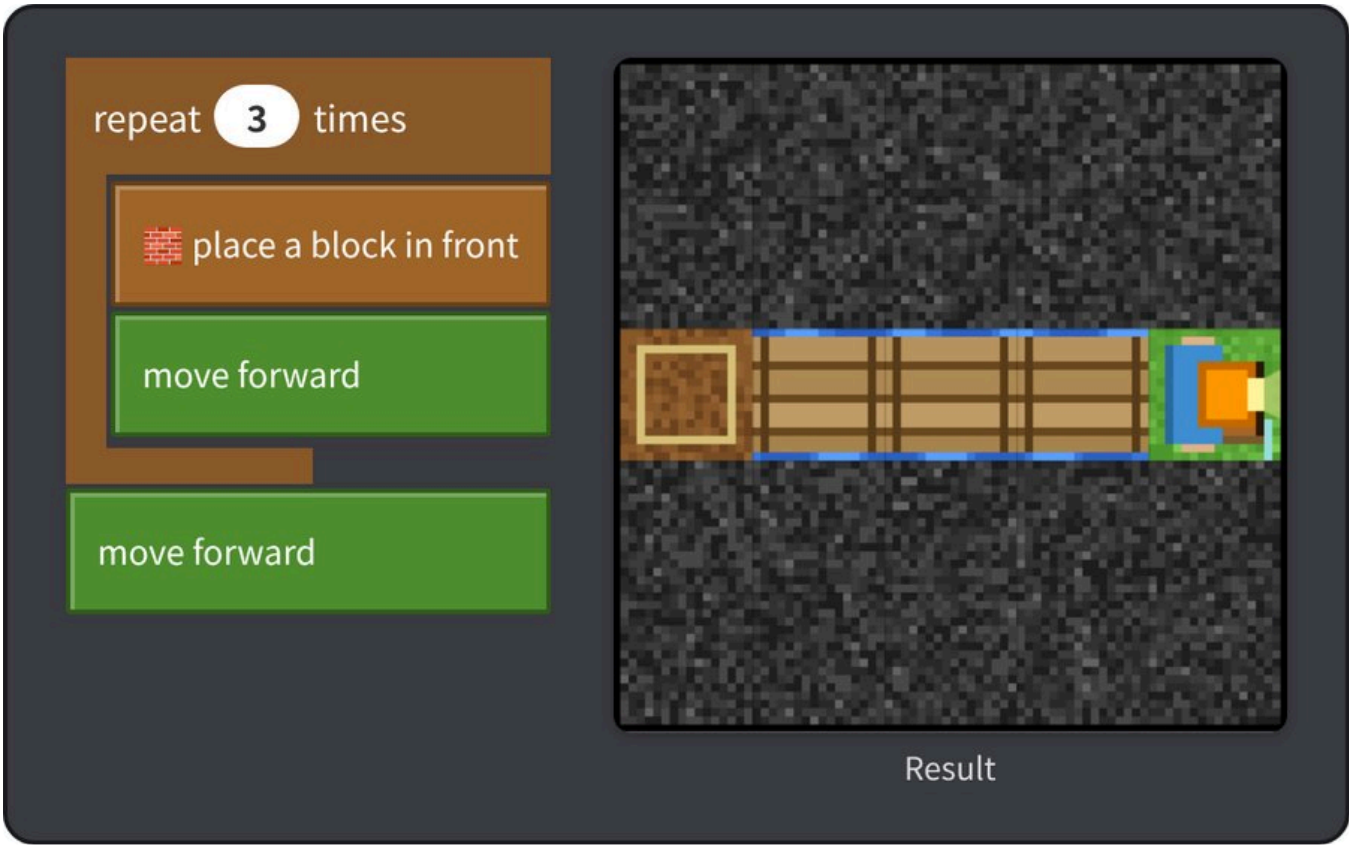
💡 **Hint (look if you get stuck)**

"repeat 3 (place → move forward)" → "move forward"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 4: A long bridge



✨ **Key point**

You had 3 blocks. After placing 3, you have exactly 0.

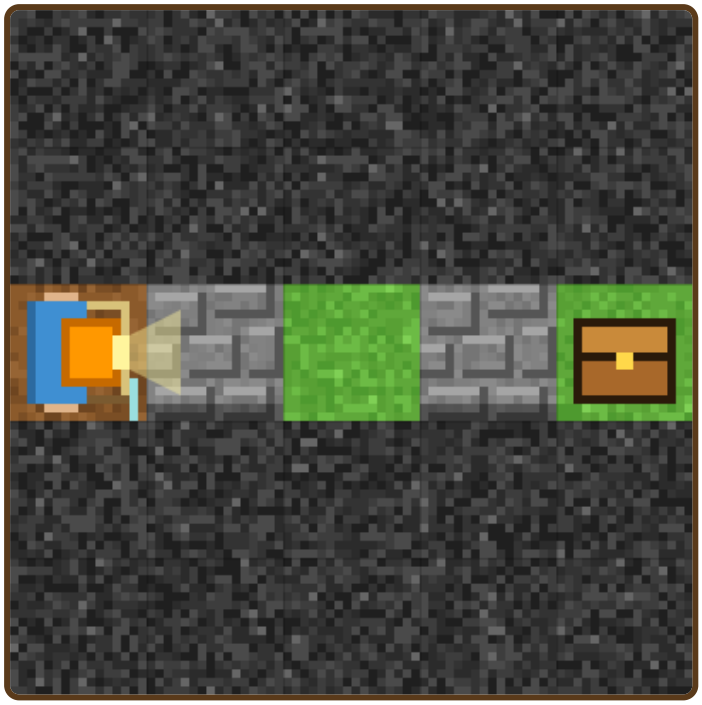
⚠ **Common mistake**

Moving before placing drops you into the water.

🎬 Video: `videos/en/craft-04.mp4`

Stage 5: If there is stone

Conditions Loops



The start

Problem

Some squares have stone and some don't. Breaking where there is no stone is a mistake! Use "if" to check first.

🎯 Target: 4 blocks or fewer

🎯 **Goal**：Check first, then break (if)

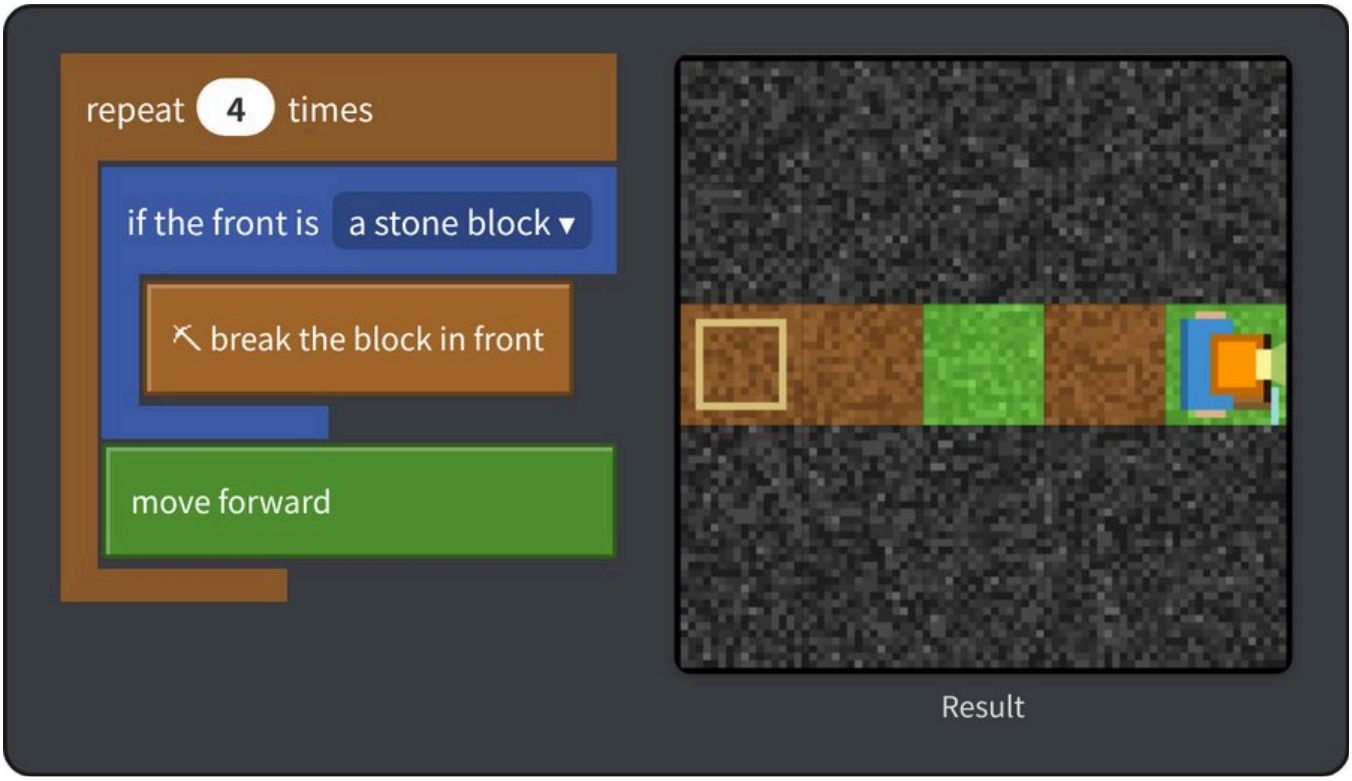
💡 **Hint (look if you get stuck)**

"repeat 4: if the front is a stone block, break it → move forward"

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: If there is stone



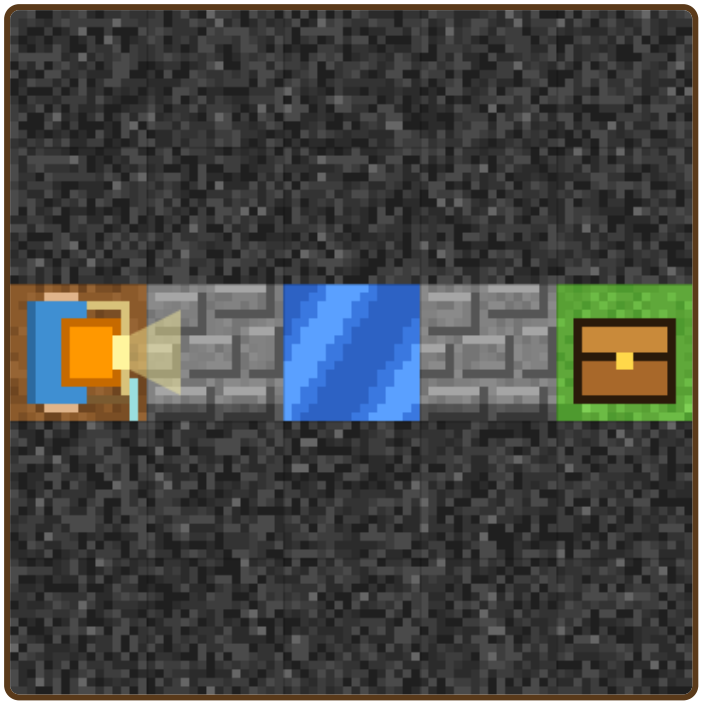
> ✨ **Key point**
>
> Break only when there is stone. Breaking where there is none is a mistake!

> ⚠ **Common mistake**
>
> Trying to break when there is no stone fails.

🎬 Video: `videos/en/craft-05.mp4`

Stage 6: Stone and water

Conditions Counting



The start

Problem

Stone and water are mixed. Break stone, bridge water. Breaking stone gives you a block, and placing one uses it up. Watch how many you have!

🎯 Target: 6 blocks or fewer

🎯 **Goal**：Stone and water: two “if”s

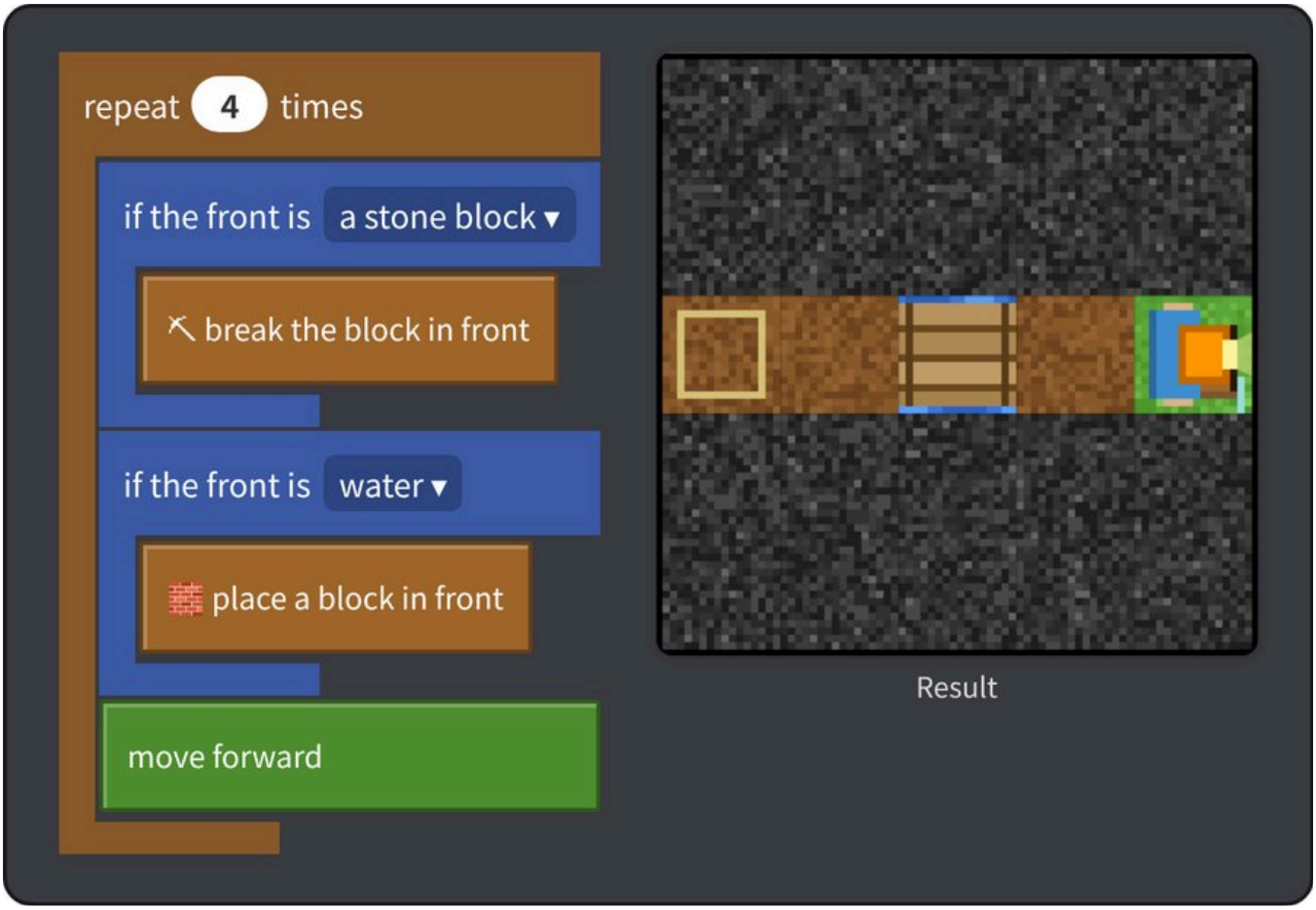
💡 **Hint (look if you get stuck)**

Repeat 4 times: if stone → break / if water → place / move forward

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Stone and water



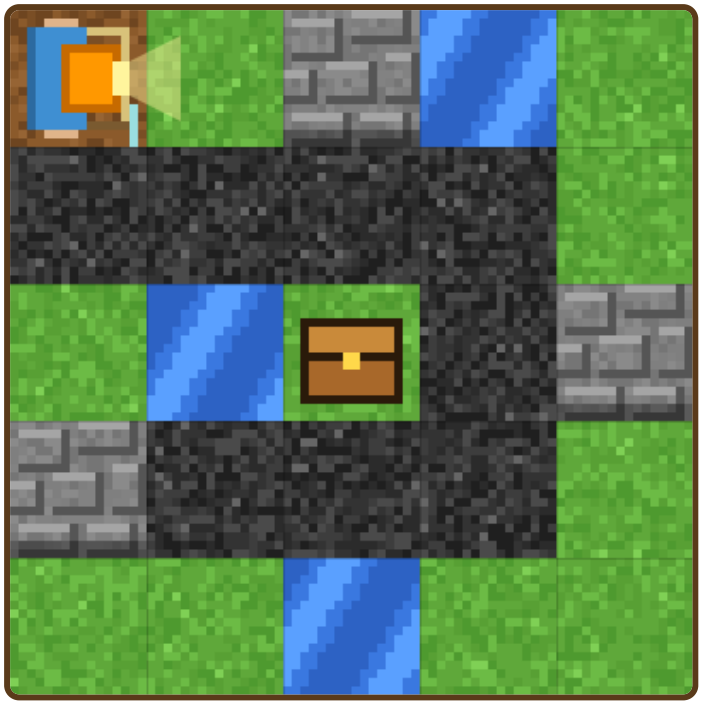
> ✨ **Key point**
>
> Breaking adds one, placing uses one: 0 → 1 → 0 → 1.

> ⚠ **Common mistake**
>
> Forgetting to check for water makes you fall in.

🎬 Video: `videos/en/craft-06.mp4`

Stage 7: Spiral cave

Conditions | Repeat until | Nesting



The start

Problem

A spiral cave full of stone and water. Use "repeat until goal" and decide what to do by checking the front.

🎯 Target: 8 blocks or fewer

🎯 **Goal**：Check the front as you go

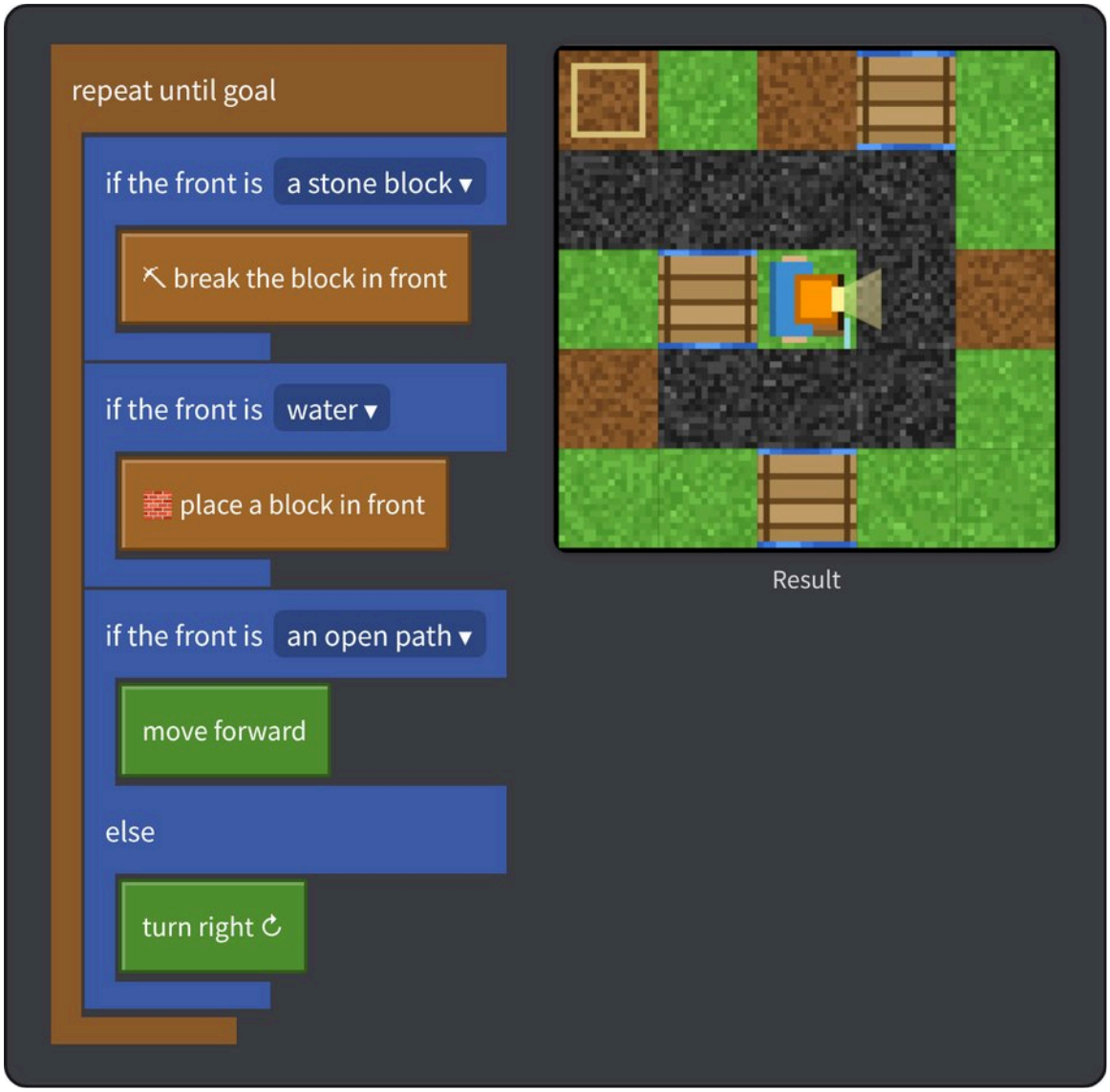
💡 **Hint (look if you get stuck)**

Inside the loop: if stone → break / if water → place / if the front is an open path → move, else turn right

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 7: Spiral cave



> ✨ **Key point**
>
> Stone? Break. Water? Place. Open? Move. Otherwise, turn right!

> ⚠ **Common mistake**
>
> Hard dark rock cannot be broken. If you cannot move, turn right.

🎬 Video: `videos/en/craft-07.mp4`

Column

Count up and down

Breaking is +1, placing is −1. When stone and water are mixed, as in stage 6, counting “how many do I have now?” as you go shows where you would run out. Counting is an important skill for reading programs.

Chapter 3

🐍 Break & Build in Python

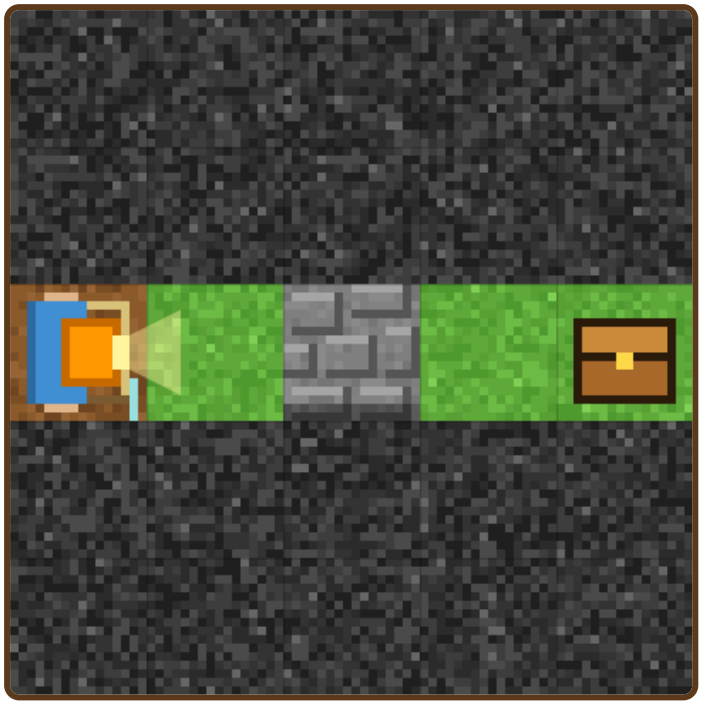
Write the adventure in **Python**: `mine()` breaks and `place()` places. Check with `if front_is('stone'):` and split into three or more cases with `elif`.

Commands in this chapter

forward() / turn_right() / mine() / place() / for i in range(3): / if front_is('stone'): / elif front_is('water'): / while not at_goal():

Stage 1: Break and go

Sequence | Python syntax (colon & indent)



The start

Problem

A stone block is in the way. mine() breaks the block in front of you. Break it and go to the treasure chest!

🎯 **Goal**：Use mine()

💡 **Hint (look if you get stuck)**

forward() → mine() → forward() ×3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 1: Break and go

```
forward()
mine()
forward()
forward()
forward()
```

✨ Key point

mine() breaks the stone in front, and it becomes a path.

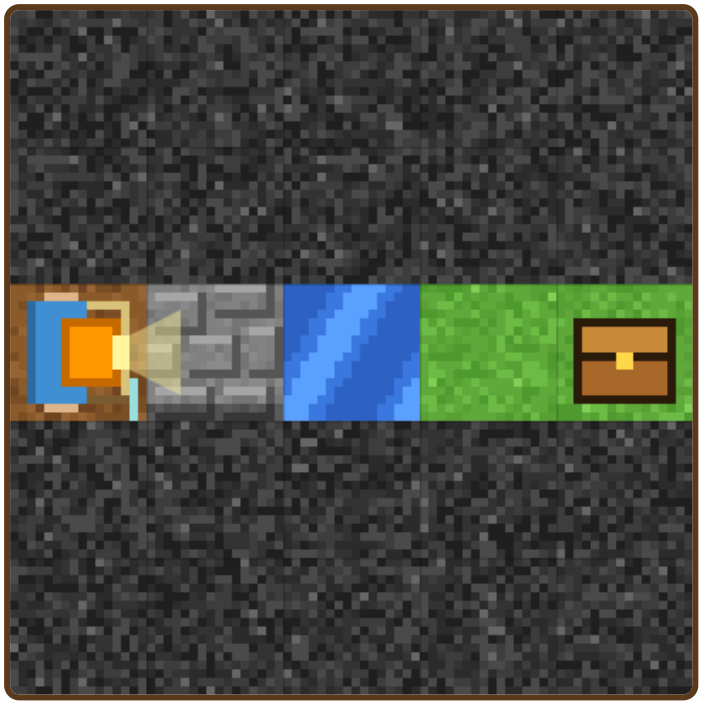
⚠ Common mistake

mine() where there is no stone fails. Move forward once before breaking.

🎬 Video: `videos/en/pycraft-01.mp4`

Stage 2: Build a bridge

Sequence | Python syntax (colon & indent)



The start

Problem

place() puts a block in front of you. On water it becomes a bridge. Break the stone first to get a block!

🎯 **Goal**：The order: mine() → place()

💡 **Hint (look if you get stuck)**

mine() → forward() → place() → forward() ×3

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 2: Build a bridge

```
mine()
forward()
place()
forward()
forward()
forward()
```

✨ Key point

You start with no blocks. mine() to get one, then place() to build a bridge.

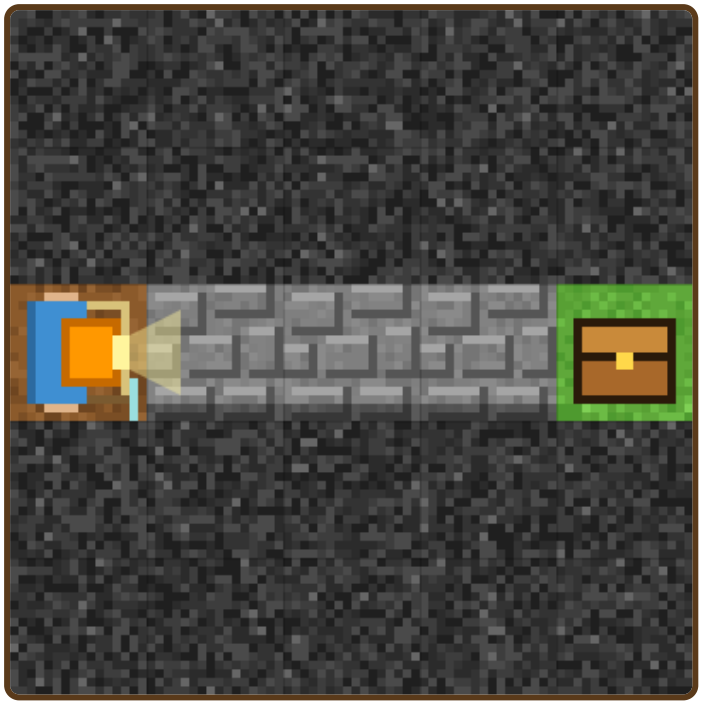
⚠ Common mistake

forward() first bumps into the stone. Move to just before the water before place().

🎬 Video: `videos/en/pycraft-02.mp4`

Stage 3: Break with for

Loops | Python syntax (colon & indent)



The start

Problem

Stone blocks in a row! Repeat “mine() → forward()” with for.

🎯 **Goal**：Break with for

💡 **Hint (look if you get stuck)**

for i in range(3): mine(), forward() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 3: Break with for

```
for i in range(3):
    mine()
    forward()
forward()
```

✨ **Key point**

“mine() → forward()” 3 times. The last forward() is outside the for (not indented).

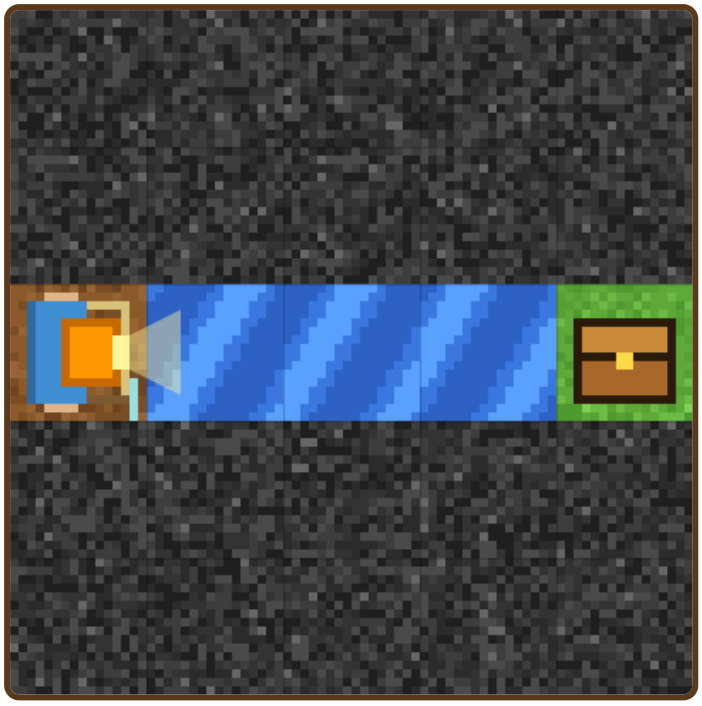
⚠ **Common mistake**

If forward() is not indented, mine() runs 3 times in a row and fails.

🎬 Video: `videos/en/pycraft-03.mp4`

Stage 4: A long bridge

Loops | Counting | Python syntax (colon & indent)



The start

Problem

You have 3 blocks. Repeat “place() → forward()” to make a long bridge.

🎯 **Goal**：Use up the blocks you have

💡 **Hint (look if you get stuck)**

for i in range(3): place(), forward() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 4: A long bridge

```
for i in range(3):
    place()
    forward()
forward()
```

✨ **Key point**

You have 3 blocks, so after 3 place() calls you have exactly 0.

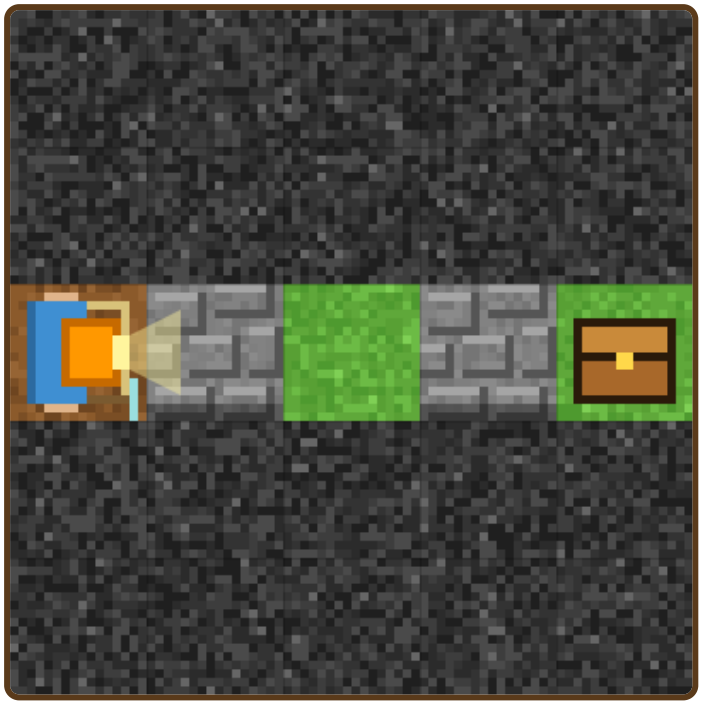
⚠ **Common mistake**

With range(4), the 4th place() runs out of blocks.

🎬 Video: `videos/en/pycraft-04.mp4`

Stage 5: if front_is('stone'):

Conditions | Loops | Nesting | Python syntax (colon & indent)



The start

Problem

front_is('stone') checks if the front is a stone block. Use if so you only break where there is stone.

🎯 **Goal**：Check with if front_is('stone'):

💡 **Hint (look if you get stuck)**

for i in range(4): if front_is('stone'): mine() → forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 5: if front_is('stone'):

```
for i in range(4):
    if front_is('stone'):
        mine()
    forward()
```

✨ Key point

front_is('stone') is True if the front is stone. Indent one more level inside if.

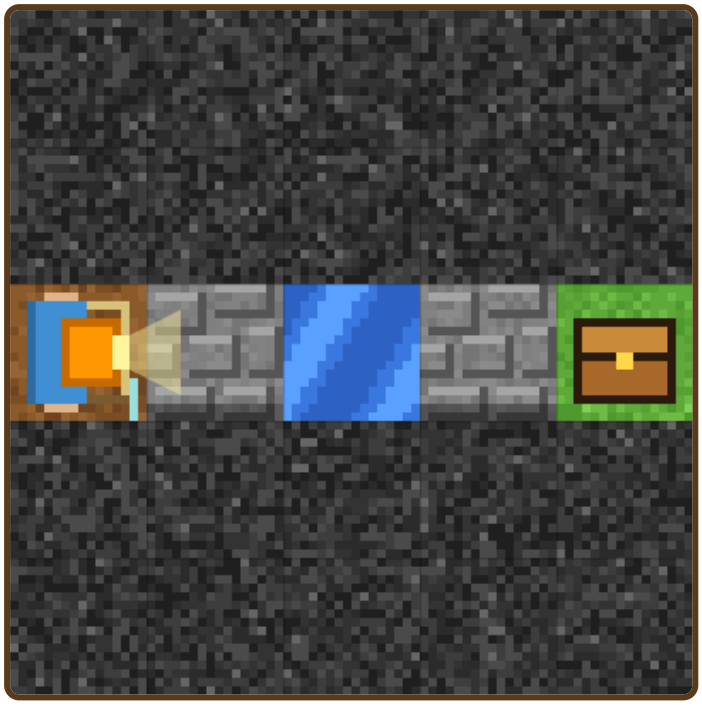
⚠ Common mistake

"stone" with double quotes is OK, but other words like 'rock' cause a ValueError.

🎬 Video: `videos/en/pycraft-05.mp4`

Stage 6: Stone and water with elif

Conditions | Loops | Nesting | Counting | Python syntax (colon & indent)



The start

Problem

Stone and water are mixed. If it is stone, mine(); elif it is water, place(). Breaking gives you a block, and placing uses one.

🎯 **Goal**：Split into two cases with elif

💡 **Hint (look if you get stuck)**

for i in range(4): if front_is('stone'): mine() / elif front_is('water'): place() / forward()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer　Stage 6: Stone and water with elif

```
for i in range(4):
    if front_is('stone'):
        mine()
    elif front_is('water'):
        place()
    forward()
```

✨ Key point

Stone → mine(), else if water → place(). Breaking adds one and placing uses one (0 → 1 → 0 → 1).

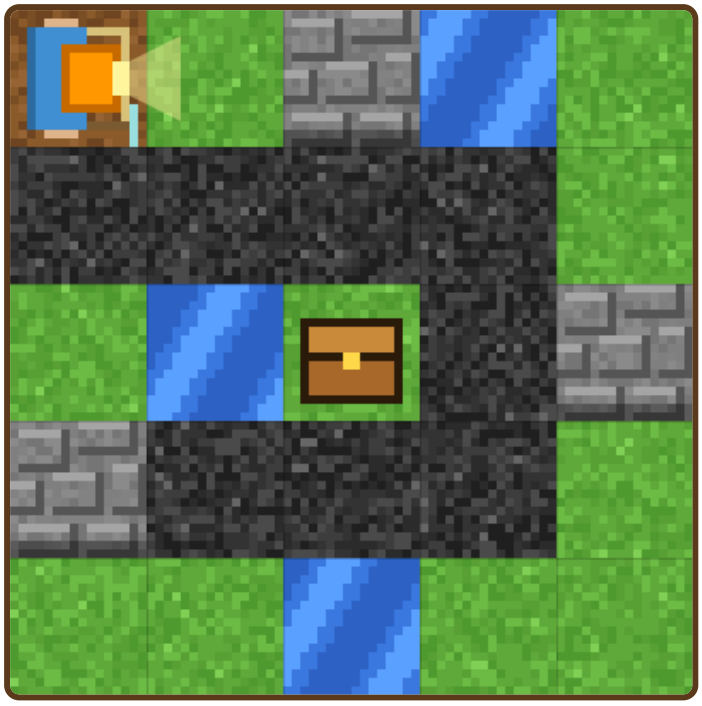
⚠ Common mistake

Writing “else if” causes a SyntaxError. Write elif at the same depth as if.

🎬 Video: `videos/en/pycraft-06.mp4`

Stage 7: Spiral cave

Conditions | Repeat until | Nesting | Generalization | Python syntax (colon & indent)



The start

Problem

Use “while not at_goal():” and decide by checking the front: stone → mine(), water → place(), path → forward(), otherwise turn_right().

🎯 **Goal**：Check the front as you go

💡 **Hint (look if you get stuck)**

while not at_goal(): if front_is('stone') … elif front_is('water') … elif front_is('path') … else: turn_right()

✏ **Write your own idea**

The answer is on the next page →

📖 Answer Stage 7: Spiral cave

```
while not at_goal():
    if front_is('stone'):
        mine()
    elif front_is('water'):
        place()
    elif front_is('path'):
        forward()
    else:
        turn_right()
```

✨ Key point

if, elif, elif and else split it into 4 cases. You check the front again on every loop.

⚠ Common mistake

Hard dark rock is not 'stone' for front_is. When you cannot move, else turns right.

🎬 Video: `videos/en/pycraft-07.mp4`

Column

How to use elif

`if` → `elif` → `else` checks from the top and runs only the first one that matches. Like “if stone, break; else if water, place; otherwise move”, you can split into three or more cases. Don’t forget the quotes in `'stone'`.

Well done!

You worked hard to the end! Let’s look back at the ideas you used.

Idea	Where did it appear?
Order	Break → carry → place (stages 1–2)
Loops	Pack “break → move” together (stages 3–4)
Counting what you carry	Up and down (stages 2, 4, 6)
If (conditions)	Break only stone, place on water (stages 5–6)
Repeat until	Check as you go to the goal (stage 7)
Python	Split cases with if / elif / else

When you clear every stage, get your certificate from “⚙ Settings” → “📜 Certificate”.

PART 2

Part 2 Guide for teachers and parents

About this guide

This book covers “Break & Build Quest” (7 stages) and “Python Break & Build” (7 stages). Part 1 is written in simple language for students.

Breaking stone adds one block; placing on water takes one away. Because the count (state) affects later commands, order really matters, which builds a basis for variables. Later stages combine cases (stone, water, path) with conditions.

Aims

- Plan the order of steps while thinking about how the count changes.
- Build programs that choose actions under several conditions.
- In Python, learn multi-way branching with if / elif / else.

Overview of the stages

App	#	Stage	Ideas
🧱 Break & Build Quest	1	Break and go	Sequence
	2	Build a bridge	Sequence, Counting
	3	Break with a loop	Loops
	4	A long bridge	Loops, Counting
	5	If there is stone	Conditions, Loops
	6	Stone and water	Conditions, Counting
	7	Spiral cave	Conditions, Repeat until, Nesting
🐍 Python Break & Build	1	Break and go	Sequence, Python syntax (colon & indent)
	2	Build a bridge	Sequence, Python syntax (colon & indent)
	3	Break with for	Loops, Python syntax (colon & indent)
	4	A long bridge	Loops, Counting, Python syntax (colon & indent)
	5	if front_is('stone'):	Conditions, Loops, Nesting, Python syntax (colon & indent)
	6	Stone and water with elif	Conditions, Loops, Nesting, Counting, Python syntax (colon & indent)
	7	Spiral cave	Conditions, Repeat until, Nesting, Generalization, Python syntax (colon & indent)

Suggested order and teaching points

Students	Stages	Lessons	Key ideas
Grades 4–5	1–4	1–2	Order, loops, counting up and down
Grades 5–6	5–7	2	Conditions (two cases), repeat until
Grade 6 – junior high	Python 1–7	2–3	if / elif / else, while

Common difficulties and support

- **Placing with nothing to place:** in stage 2, use the on-screen count to show you must break first.
- **Keeping count:** have students write the count after each move in the Worksheet 5 table.
- **Two conditions (stage 6):** have students say “when, and what to do” in words before building.
- **Python:** watch the quotes in `front_is('stone')` and the order of `elif` (checked from the top).

Example 45-minute lesson (stages 4–6)

Time	Activity	Support
Intro 5 min	Confirm that breaking adds and placing takes away; share the goal.	Point to the on-screen count.
Main 30 min	Build a long bridge; break only stone; split actions for stone and water.	Record the count in a table.
Wrap-up 10 min	Summarize the value of counting and cases.	Introduce the word “variable”.

Assessment (example)

Area	Meets expectations (B)	Exceeds (A), e.g.
Knowledge & skills	Combines blocks (code) correctly and completes the task.	Uses loops and conditions to complete it with a short, clear program.
Thinking, judgment & expression	Finds where the error is from the result and fixes it.	Explains the rule or steps in words and makes a program that works in other cases too.
Attitude toward learning	Keeps trying even when it doesn’t work.	Shares ideas with friends and looks for better ways on their own.

Use the worksheets (1 reflection; 2, 5, 7 etc.) and the app progress (8 check card, certificate) as evidence.

Using the apps and related materials

Requirements and tips

- Works in the browser on computers, tablets and smartphones. No installation needed. An internet connection is required.
- Cleared stages are saved in that device’s browser. If devices are shared, check progress at the start of each lesson.
- **Teacher mode:** add `?sensei` to the end of the address to show an “Answer” button.
- From “⚙ Settings” you can switch Japanese/English, use the pixel font, switch to the Python version and show the certificate.

Related materials

Material	Contents
Guide (guide.html)	Goals, common mistakes, hints and answers for every task. This book is built from the same data.
Worksheets (PDF)	5 Space / Break & Build (order and count table), 7 Python, 1 goal & reflection, 8 check card.
Lesson plans (Japanese)	Example 45-minute lessons (open them from each task in the Guide).
Videos (videos.html)	Short videos showing how to solve each task.
Lesson slides	04 Break & Build Quest, 08 Python Break & Build (PowerPoint).